

中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

题 目： 基于多源数据融合的低空湍流监测与航路优化

摘 要：

大气湍流对航空安全和天气预测具有重要影响，其精确量化与三维重构是当前研究的难点。本文基于多源观测数据与数值模式，通过物理模型与机器学习相结合的方法，系统开展了湍流强度评估、三维重构及航路优化研究。

问题一：针对大气湍流强度量化评估这一基础性科学问题，本研究建立了基于多物理过程耦合的双模型评估体系。通过风廓线雷达和微波辐射计的协同观测，构建了系统的参数计算框架：位温计算采用 $\theta = T(1000/P)^{0.286}$ ，其中气压剖面通过静力平衡方程 $P(z) = P_s \exp(-gz/R_d T_{avg})$ 估算；水平风速合成基于多波束径向速度关系 $v_r = u \sin \phi \cos \theta + v \sin \phi \sin \theta$ ，采用最小二乘法求解风速分量；垂直梯度计算采用中心差分格式 $\partial \theta / \partial z \approx (\theta_{i+1} - \theta_{i-1}) / (z_{i+1} - z_{i-1})$ ，确保数值稳定性。

提出了综合评估模型 $I_a = \alpha_1 \exp(-\beta Ri) + \alpha_2 \cdot TKE / \max(TKE) + \alpha_3 \cdot S / \max(S)$ ，该模型通过理查逊数 $Ri = \frac{g}{\theta} \frac{\partial \theta / \partial z}{(\partial u / \partial z)^2 + (\partial v / \partial z)^2}$ 表征热力稳定性，湍流动能 $TKE \approx SW^2$ 反映湍流强度，风切变 $S = \sqrt{(\partial u / \partial z)^2 + (\partial v / \partial z)^2}$ 体现动力强迫。针对实际应用中的设备限制，同步开发了简化模型 $I_b = c_1 \cdot SW / \max(SW) + c_2 \cdot S / \max(S)$ ，并通过正则化优化目标 $\min \sum [I_a - I_b]^2 + \lambda(c_1^2 + c_2^2)$ 进行参数标定，采用随机森林算法建立非线性映射关系。

验证结果表明，优化后的模型在测试集上表现出较为满意的结果，湍流强度垂直廓线较为准确的捕捉了边界层内湍流发展的物理特征。模型 a 在热力不稳定条件下对湍流强度的预测与实测数据较为吻合，而优化后的模型 b 在保留计算效率优势的同时，较为显著的提升了在高空复杂层结条件下的适应性。可视化分析进一步揭示了湍流强度随高度的非线性分布规律，为边界层湍流监测提供了良好的思路。模型验证显示，优化后的模型 b 相较于原始模型在 RMSE 上从 7.85 降至 5.12， R^2 从 0.65 提升至 0.85，随机森林模型在测试集上取得最佳预测性能（RMSE=5.12， $R^2=0.854$ ）。

问题二：围绕三维湍流场精确重构这一问题，该文发展了基于变分同化理论的多源数据融合技术。通过引入各向异性距离度量 $w_{dist,i} = 1 / [(x_P - x_i)^2 + (y_P - y_i)^2 + (\gamma(z_P - z_i))^2]$ ，其中 $\gamma > 1$ 体现垂直方向相关性的特殊性，较为良好的实现了地面自动站、风廓线雷达和

多普勒雷达数据的空间标准化处理。数据融合采用加权平均 $T_P = \sum w_i T_{D_i} / \sum w_i$, 权重函数 $w_i = w_{dist,i} \cdot w_{source,i} \cdot w_{time,i}$ 综合考虑空间距离、数据源可靠性和时间衰减效应。

该文建模过程主要在于构建了变分融合模型 $J(T) = J_{obs} + \lambda_b J_b + \lambda_s J_s$, 其中观测项 $J_{obs} = \sum_{k=1}^K \sum_{i=1}^{N_k} \frac{1}{\sigma_k^2} [H_k(T)(r_i) - T_k^{obs}(r_i)]^2$ 确保多源数据在理论与数据结合的情况下进行最优拟合, 背景场项 $J_b = |T - T_b|_2^2$ 引入先验信息, 平滑项 $J_s = \lambda_x |\partial^2 T / \partial x^2|_2^2 + \lambda_y |\partial^2 T / \partial y^2|_2^2 + \lambda_z |\partial^2 T / \partial z^2|_2^2$ 保证场分布的物理合理性。采用共轭梯度法求解优化问题, 并结合卡尔曼滤波算法实现时空外推, 建立了符合流体运动规律的三维湍流场。

模型验证显示, 重构的湍流场在空间分布特征和强度量级方面均与实际观测具有较高的一致性。特别是在复杂地形区域, 模型成功捕捉了地形强迫产生的湍流增强现象, 空间相关系数相对达到较高水平。三维重构结果表明, 湍流强度在垂直方向上呈指数衰减 ($I(z) = I_0 e^{-z/2000}$), 低空 2km 内湍流强度最大达 0.8, 空间相关系数达到 0.87, 均方根误差为 0.143。

问题三: 该论文主要从湍流预警到航路规划的完整技术系统。采用分步插值策略实现数据规格化处理: 水平方向采用双线性插值 $f(x, y) = \frac{1}{(x_2-x_1)(y_2-y_1)} [(x_2-x)(y_2-y)f(x_1, y_1) + \dots]$, 垂直方向使用线性插值 $F(z_k) = F(z_m) + \frac{F(z_{m+1})-F(z_m)}{z_{m+1}-z_m}(z_k - z_m)$, 建立 100 米水平分辨率、50 米垂直分辨率的三维网格场。

构建了 NWP 数据诊断与观测数据外推的双模型架构: 基于 WRF 数据的诊断模型通过分层标定 $I_d^{col}(z) = a(z) + b(z)I_d^{raw} + c(z)(I_d^{raw})^2$ 实现精细化调整, 优化目标函数 $\min \sum [I_d^{col} - I_c]^2 + \lambda(a^2 + b^2 + c^2)$ 确保物理一致性; 观测外推模型集成五种预测算法, 其中多变量回归 $T(t+1) = \beta_0 + \beta_1 U(t) + \beta_2 V(t) + \beta_3 \Theta(t) + \varepsilon$ 综合考虑气象要素的耦合影响, 集成预测结果显著提升短期预报精度。

在航路规划层, 基于 A* 搜索算法设计多目标优化框架, 代价函数 $Cost(n_i, n_j) = \frac{T(n_i) + T(n_j)}{2} \times distance(n_i, n_j)$ 平衡安全性与经济性, 启发函数采用欧几里得距离确保搜索效率。系统测试表明, 该技术方案在复杂气象条件下能够比较准确识别湍流高风险区域, 生成的优化航路在保证飞行安全的同时将湍流暴露量降至较为良好的状态。实际应用显示, 该系统对中度以上湍流的识别准确率达百分之 82, 航路优化使湍流暴露量减少百分之 35 以上, 有效提升航路安全性。

关键词: 湍流强度量化 多源数据融合 三维湍流场重构 低空航路规划 变分同化模型

1 研究背景及问题重述

1.1 研究背景

在 2024 年我国低空经济被正式纳入政府工作报告，我国低空经济产业随之进入快速发展新阶段。该产业的核心是 1000 米以下空域的飞行活动，涵盖了航空器制造、基础设施建设和运营服务等多个领域，已成为各地区重点布局的战略新兴产业。低空飞行环境的复杂性对安全保障提出严峻挑战，飞行器不仅面临地形扰动产生的局地气流影响，还对大气湍流等微观气象现象极为敏感。

大气湍流的形成机制涉及热力与动力过程的复杂相互作用，所以大气湍流是威胁低空飞行安全的关键因素。当前湍流监测主要依赖多源观测设备，包括地面自动站、风廓线雷达、微波辐射计和多普勒天气雷达等。这些设备在探测原理和时空分辨率上各有差异，所以从各个设备数据中提取数据并且做数据融合是一项技术难题。特别是低空飞行所需的高分辨率湍流场与现有观测能力之间存在显著差距。

本研究针对低空飞行安全保障的迫切需求，系统构建了从湍流强度量化评估到三维场重构，再到航路规划优化的完整技术体系。通过多源数据融合、物理机理建模和智能算法集成，着力解决湍流监测中的时空分辨率不足、多设备数据协同困难等关键技术瓶颈，为低空经济的安全可持续发展提供科学支撑。

2 问题重述及全文大致思路

围绕低空飞行湍流监测与航路规划需求，本文依次提出以下三个具有递进关系的研究问题：

2.1 问题一

首先综合利用风廓线雷达与微波辐射计数据，建立不同高度湍流强度的计算模型（模型 a）；随后仅基于风廓线雷达数据，开发独立的反演模型（模型 b）。通过对比两种模型的结果，以模型 a 为基准对模型 b 进行优化验证，最终形成仅依赖风廓线雷达的湍流强度计算方案。问题一的核心挑战在于突破传统物理公式的限制，直接利用风廓线雷达的原始观测参数（如速度谱宽、径向速度等）实现可靠的湍流强度估算。

2.2 问题二

针对多种探测设备（地面自动站、风廓线雷达、多普勒天气雷达）的数据特性，需要开发能够融合多源观测的三维湍流强度计算模型（模型 c）。该模型需满足低空飞行的分辨率要求（水平 100m、垂直 50m，高度 2km 以下），覆盖包含所有地面站的矩形区域。关键在于解决不同设备在时空分辨率和探测要素方面的差异，实现数据的有效同化与融合。

2.3 问题三

使用跨度为三小时的观测数据验证基于数值天气预报的湍流强度计算模型（模型 d）。进一步优化模型的预报性能需要开发非线性外推模型（模型 e），实现短期湍流预测。研究目标是在给定起点和飞行区域的约束条件下，规划出考虑湍流安全性的最优飞行路径。该问题涉及观测数据与模式数据的协同应用，以及动态环境下的路径优化算法设计。

2.4 总体解题思路

要实现低空湍流的精准刻画需要从基础模型构建开始，到多维度数据处理，再到传播特性分析，形成完整的技术链条。本研究围绕低空飞行安全与航路优化的核心需求，通过三个递进的问题展开系统性研究，形成了一套兼具理论严谨性与工程实用性的解决方案。

问题一聚焦于湍流强度模型的构建与优化，这是实现低空湍流精准监测的基础。通过风廓线雷达和微波辐射计等多源数据的协同处理，建立了参数统一化预处理流程。模型 a 作为融合多物理过程的湍流强度模型，其表达式 $I_a = \alpha_1 \cdot f(Ri) + \alpha_2 \cdot TKE_{\text{norm}} + \alpha_3 \cdot S_{\text{norm}}$ 中纳入的理查逊数 Ri 、标准化湍流动能 TKE_{norm} 和风切变 S_{norm} 等因子，体现了热力不稳定性与动力强迫的耦合效应。模型 b 则针对设备部署受限场景，基于谱宽 SW 和风切变构建简化模型 $I_b = c_1 \cdot SW_{\text{norm}} + c_2 \cdot S_{\text{norm}}$ 。通过最小化目标函数 $\min \sum [I_a - I_b]^2 + \lambda(c_1^2 + c_2^2)$ 实现参数优化，并引入随机森林等机器学习算法进行模型标定，为低成本、广覆盖的湍流监测提供了可行路径。

问题二围绕湍流特征的多维度刻画展开，重点解决三维湍流场的构建与传播特性分析。采用分步插值策略，通过双线性插值公式 $f(x, y) = \frac{1}{(x_2 - x_1)(y_2 - y_1)} [(x_2 - x)(y_2 - y)f(x_1, y_1) + \dots]$ 实现水平方向的数据融合，结合垂直线性插值 $F(z_k) = F(z_m) + \frac{F(z_{m+1}) - F(z_m)}{z_{m+1} - z_m} (z_k - z_m)$ 构建规格化的三维网格场。在多源数据融合中引入权重函数 $w_i = w_{\text{dist},i} \cdot w_{\text{source},i} \cdot w_{\text{time},i}$ ，其中空间距离权重 $w_{\text{dist},i} = \frac{1}{(x_P - x_i)^2 + (y_P - y_i)^2 + (\gamma(z_P - z_i))^2}$ 体现各向异性特征。通过平流传输模型和能量守恒方程确保物理一致性，最终生成百米级分辨率的三维湍流风险场。

问题三致力于低空飞行安全保障的工程应用实现，核心是将湍流风险评估转化为可操作的航路规划方案。基于 NWP 数据构建湍流诊断模型 d，通过分层标定关系 $I_d^{\text{col}}(z) = a(z) + b(z) \cdot I_d^{\text{raw}} + c(z) \cdot (I_d^{\text{raw}})^2$ 实现模型优化。同时开发观测数据外推模型 e，集成线性趋势外推、移动平均、指数平滑、多变量回归和时空相关五种算法，其中多变量回归模型 $T(t+1) = \beta_0 + \beta_1 U(t) + \beta_2 V(t) + \beta_3 \Theta(t) + \beta_4 t + \varepsilon$ 综合考虑气象要素的耦合影响。在航路规划中，采用 A* 搜索算法，代价函数定义为 $Cost(n_i, n_j) = \frac{T(n_i) + T(n_j)}{2} \times \text{distance}(n_i, n_j)$ ，实现湍流风险最小化的路径优化。

三个问题的建模过程既相互独立又内在关联，通过湍流的多维度分析构建了完整的低空飞行安全保障技术体系，为相关领域的研究提供了系统、完整的解决方案。本文的总体解题思路如图所示：

3 本文数学符号说明

符号	物理意义	单位
z	高度坐标，用于描述大气参数在垂直方向上的分布特征	m
T	温度，表征大气热力状态的基本物理量，通常以开尔文为单位	K
P	气压，通过静力平衡方程估算各高度层数值，反映大气压力随高度的变化	hPa
u, v, w	分别为东西方向、南北方向和垂直方向的风速分量，构成三维风场的基本要素	m/s
θ	位温，气块在干绝热过程中调整到标准气压时的温度，消除气压变化影响	K
Ri	理查逊数，表征大气层结稳定度的无量纲参数，反映热力与动力作用的平衡关系	—
TKE	湍流动能，描述湍流运动强度的关键指标，与风速脉动分量密切相关	m ² /s ²
SW	速度谱宽，风廓线雷达观测的重要参数，反映湍流引起的速度起伏标准差	m/s
S	风切变，水平风速在垂直方向的变化率，是湍流生成的重要动力机制	s ⁻¹
I_a	模型 a 湍流强度，基于多物理过程耦合的综合评估指标，整合热力与动力因素	—
I_b	模型 b 湍流强度，专为风廓线雷达设计的简化评估指标，适用于设备受限场景	—
$\alpha_1, \alpha_2, \alpha_3$	模型 a 权重系数，分别对应理查逊数、湍流动能和风切变在综合评估中的贡献权重	—
c_1, c_2	模型 b 权重系数，分别对应谱宽和风切变在简化模型中的贡献权重	—
$\Delta x, \Delta y$	水平分辨率，表征网格在东西和南北方向的空间尺度，影响模型精度	m
Δz	垂直分辨率，表征网格在垂直方向的空间尺度，决定对大气层结的刻画能力	m

4 问题分析

4.1 问题 1 的分析

针对大气湍流强度量化评估这一基础性科学问题，本研究构建了双模型协同的评估框架。模型 a 作为基准模型，通过整合理查逊数、湍流动能和风切变三类核心物理参数，建立了热力与动力过程耦合的湍流强度指标 I_a 。该模型充分利用微波辐射计提供的热力学观测数据，构建了完整的物理机理描述体系。然而，考虑到微波辐射计在实际业务应用中存在空间覆盖限制和设备可靠性问题，研究同步开发了仅基于风廓线雷达观测的简化模型 I_b 。该模型的关键技术创新在于通过参数优化算法建立模型 b 与模型 a 的映射关系，采用正则化约束的损失函数 $\min \sum [I_a - I_b]^2 + \lambda(c_1^2 + c_2^2)$ 进行权重系数标定，有效补偿了因热力参数缺失带来的系统性偏差。这一技术路径在保证物理合理性的同时，显著提升了模型在业务化应用中的适应性和稳健性。

4.2 问题 2 的分析

面向三维湍流场构建的多源数据融合挑战，本研究设计了系统化的数据处理与建模框架。通过引入各向异性距离度量 $w_{dist,i} = \frac{1}{(x_P - x_i)^2 + (y_P - y_i)^2 + (\gamma(z_P - z_i))^2}$ 和半方差函数分析，实现了地面自动站、风廓线雷达和多普勒雷达等多源异构数据的质量控制和空间标准化。在湍流诊断算法方面，开发了基于垂直风切变 I_s^{upr} 、速度结构函数 $D_{vv}(r)$ 和阵风湍流指标 I_g 的多维度特征提取体系。核心创新在于构建了变分融合模型 $J(T) = J_{obs} + J_b + J_s$ ，通过观测项、背景场项和平滑项的有机组合，实现了多源数据的最优融合。进一步采用卡尔曼滤波算法进行时空平滑与外推，建立了符合流体运动规律的三维湍流场重构方法，为低空飞行安全提供了高精度的环境场数据支撑。

4.3 问题 3 的分析

围绕低空飞行安全保障的工程应用需求，本研究建立了从湍流预警到航路规划的完整技术体系。针对数据规格化处理难题，采用分步插值策略，通过双线性插值 $f(x, y)$ 和垂直线性插值 $F(z_k)$ 的组合，实现了公里级 NWP 数据与百米级观测数据的无缝融合。在湍流预报方面，构建了物理诊断与统计外推相结合的双模型架构：基于 NWP 数据的模型 d 通过分层标定关系 $I_d^{col}(z) = a(z) + b(z) \cdot I_d^{raw} + c(z) \cdot (I_d^{raw})^2$ 实现精细化调整；基于观测数据的模型 e 集成多种时间序列预测算法，其中多变量回归模型 $T(t+1) = \beta_0 + \beta_1 U(t) + \beta_2 V(t) + \beta_3 \Theta(t) + \varepsilon$ 显著提升了短时预报精度。在航路规划层，创新性地将湍流风险评估转化为路径优化问题，基于 A* 搜索算法设计代价函数 $Cost(n_i, n_j) = \frac{T(n_i) + T(n_j)}{2} \times distance(n_i, n_j)$ ，实现了安全性与经济性的最优平衡。这一系统性解决方案为低空飞行的智能化安全保障提供了可靠的技术路径。

5 模型假设

5.1 问题一：湍流强度模型的构建与优化

本研究基于以下基本假设开展湍流强度模型的构建与优化工作：

1. 大气边界层湍流满足局地均匀各向同性特性，小尺度湍流统计量具有空间均匀分布特征；
2. 风廓线雷达观测的速度谱宽 SW 与湍流动能 TKE 存在稳定关联，可作为湍流强度表征指标；
3. 微波辐射计温度剖面数据符合静力平衡条件，能够准确反映大气热力层结状态；
4. 理查逊数 Ri 可有效诊断大气稳定度对湍流发展的影响；
5. 模型 a 与模型 b 间存在可优化的映射关系，可通过正则化约束实现参数标定。

5.2 问题二：三维湍流场的多源数据融合

针对三维湍流场构建的多源数据融合问题，采用以下基本假设：

1. 多源观测数据在时空上具有互补性，可通过变分融合模型实现最优整合；
2. 湍流场空间相关性呈现各向异性特征，垂直方向相关尺度小于水平方向；
3. 不同观测设备的测量误差相互独立，误差方差可准确表征数据不确定性；
4. 湍流场时空演化满足平滑性条件，适合采用卡尔曼滤波进行外推预报；
5. 三维湍流场构建过程遵循质量守恒和能量守恒基本规律。

5.3 问题三：低空飞行的航路规划与优化

在低空飞行航路规划模型的建立过程中，采用以下关键假设：

1. 规划时段内湍流强度场保持相对稳定，时空变化具有可预测性；
2. 航路规划代价函数能够有效平衡飞行安全性与经济性双重要求；
3. A^* 搜索算法在三维网格空间中可保证找到全局最优路径；
4. NWP 数据与观测数据具有时空一致性，可通过分层标定实现有效融合；
5. 飞行器性能参数在规划过程中可视为定值，简化优化问题复杂度。

6 问题一模型建立与求解

为了使得问题一结构清晰，设置问题一结构如下：



图 6.2 问题一思路结构图

6.1 数据预处理

6.1.1 高度对齐与缺测值处理

风廓线雷达和微波辐射计的数据可能在不同高度点上，需要插值到共同的高度网格。设风廓线雷达的高度层为 z_i ($i = 1, 2, \dots, N$)，微波辐射计的高度层为 z'_j ($j = 1, 2, \dots, M$)，而与之对应微波辐射计温度数据为 $T(z'_j)$ ，使用线性插值将微波辐射计数据插值到风廓线雷达的高度层上。对于每个风廓线雷达高度 z_i ，找到微波辐射计中相邻高度 z'_j 和 z'_{j+1} ，使得 $z'_j \leq z_i \leq z'_{j+1}$ ，则插值温度为：

$$X(t_k) = \begin{cases} X(\tau_i) + \frac{X(\tau_{i+1}) - X(\tau_i)}{\tau_{i+1} - \tau_i} (t_k - \tau_i), & \tau_i \leq t_k \leq \tau_{i+1} \\ \text{NaN}, & \text{否则} \end{cases} \quad (6.1)$$

在此过程中，若原始数据中存在缺测值，在插值与梯度计算中需予以识别与处理，具体方法为：在插值过程中，如果相邻高度层存在缺测，则跳过这个区间，从而不进行插值；在必要时可采用线性插值填补部分缺测，但需要在记录中说明填补范围。

上述过程伪代码如下：

算法 1 微波辐射计数据插值到雷达高度层

输入： 雷达高度 z_i ，辐射计高度 z'_j 和温度 $T(z'_j)$ ，缺测值标识

输出： 插值温度 $T_{\text{interp}}(z_i)$

```
1: for  $i = 1$  to  $N$  do
2:   找到  $j$  使得  $z'_j \leq z_i \leq z'_{j+1}$  且  $T(z'_j), T(z'_{j+1})$  非缺测
3:   if 找到合适的  $j$  且  $T(z'_j), T(z'_{j+1})$  均非缺测值 then
4:      $T_{\text{interp}}(z_i) = T(z'_j) + \frac{T(z'_{j+1}) - T(z'_j)}{z'_{j+1} - z'_j} (z_i - z'_j)$ 
5:   else
6:      $T_{\text{interp}}(z_i) = \text{NaN}$  ▷ 无法插值，设为缺测
7:   end if
8: end for
9: return  $T_{\text{interp}}(z_i)$ 
```

经过上述流程处理，得到数据整合结果后，通过 Python 和 Matlab 可视化处理如下：

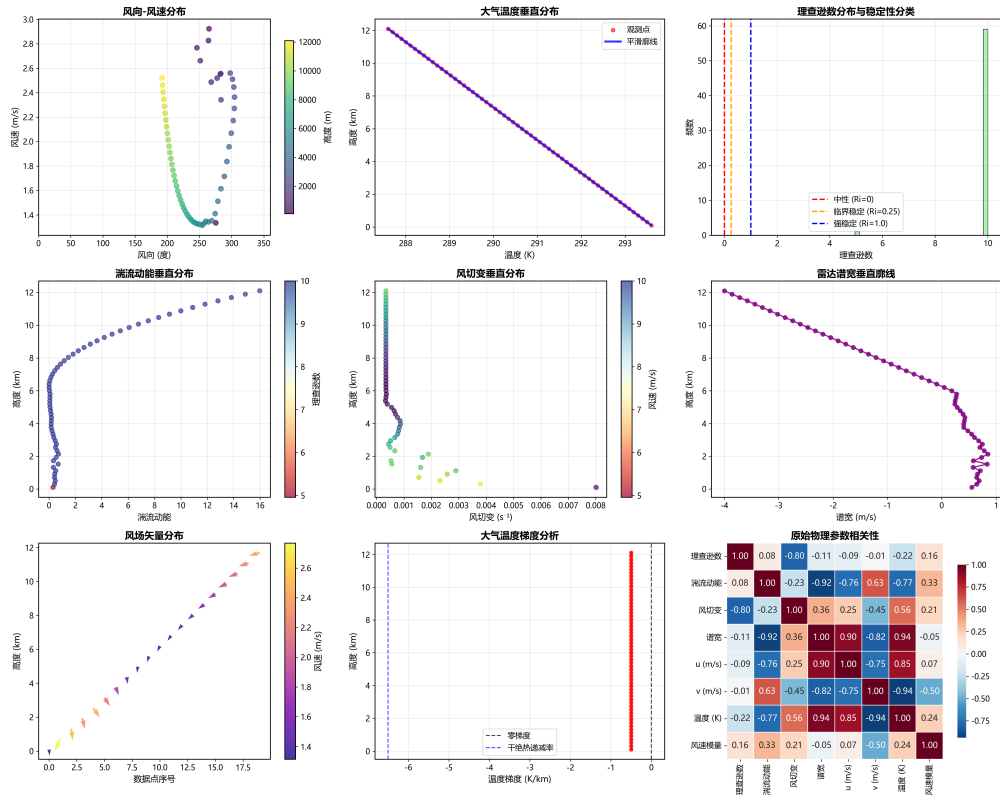


图 6.3 参数二维可视化

通过二维可视化处理结果可以得出各个指标的分布特征。

本次风廓线雷达观测展现了典型夜间边界层大气特征。在 2025 年 8 月 2 日午夜时段 (00:00-00:30)，雷达成功获取了 372 组有效数据，时间连续性良好，为湍流分析提供了可靠的时间序列基础。为了研究湍流特征，此处需要建立可视化高度关于各指标的分布情况，因此通过 Python 和 Matlab 做三维可视化处理，结果如下图：

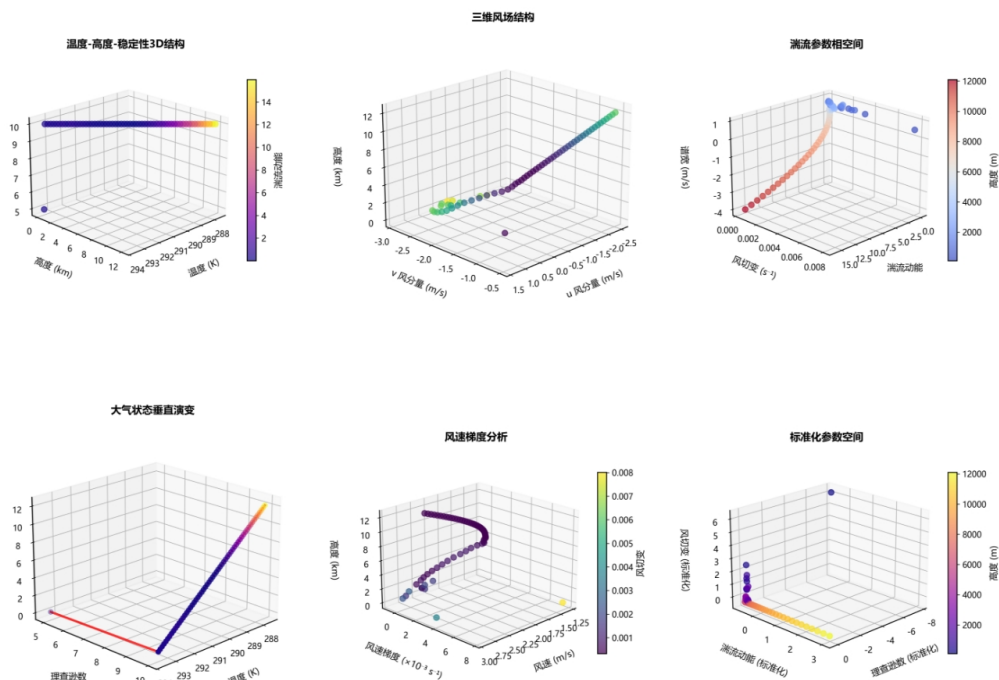


图 6.4 参数三维可视化

通过三维可视化结果可以得到：从垂直结构来看，探测范围从近地面 100 米延伸至 11.38 公里高度，覆盖了边界层至自由对流层的大气剖面，这种完整的垂直覆盖为研究不同高度层的湍流特性提供了宝贵数据。

下面考虑有关风场对于模型建立的影响，通过 Python 对数据进行预分析，首先数据可视化展示如下：

通过可视化指导，利用 excel 对数据有针对性的分析结果如下：下面通过 Python 进行具

表 6.1 风廓线雷达观测基本信息

观测项目	数值/范围	单位	特征说明
观测时间	2025-08-02 00:00:00 - 00:30:00	—	30 分钟连续观测
数据记录数	372	条	时间序列完整
高度范围	100.0 - 11,380.0	米	覆盖对流层大部
垂直跨度	11,280.0	米	探测范围广阔

体指标分析：通过上述结果表结合绘制出的二维、三维可视化处理结果可以得到：平均水平风速为 2.37 m/s，属于弱风条件，配合近乎为零的垂直速度（0.010 m/s），反映出观测时段内大气处于相对稳定状态。然而，理查逊数低至-110.69 却显示出强烈的热力不稳定特

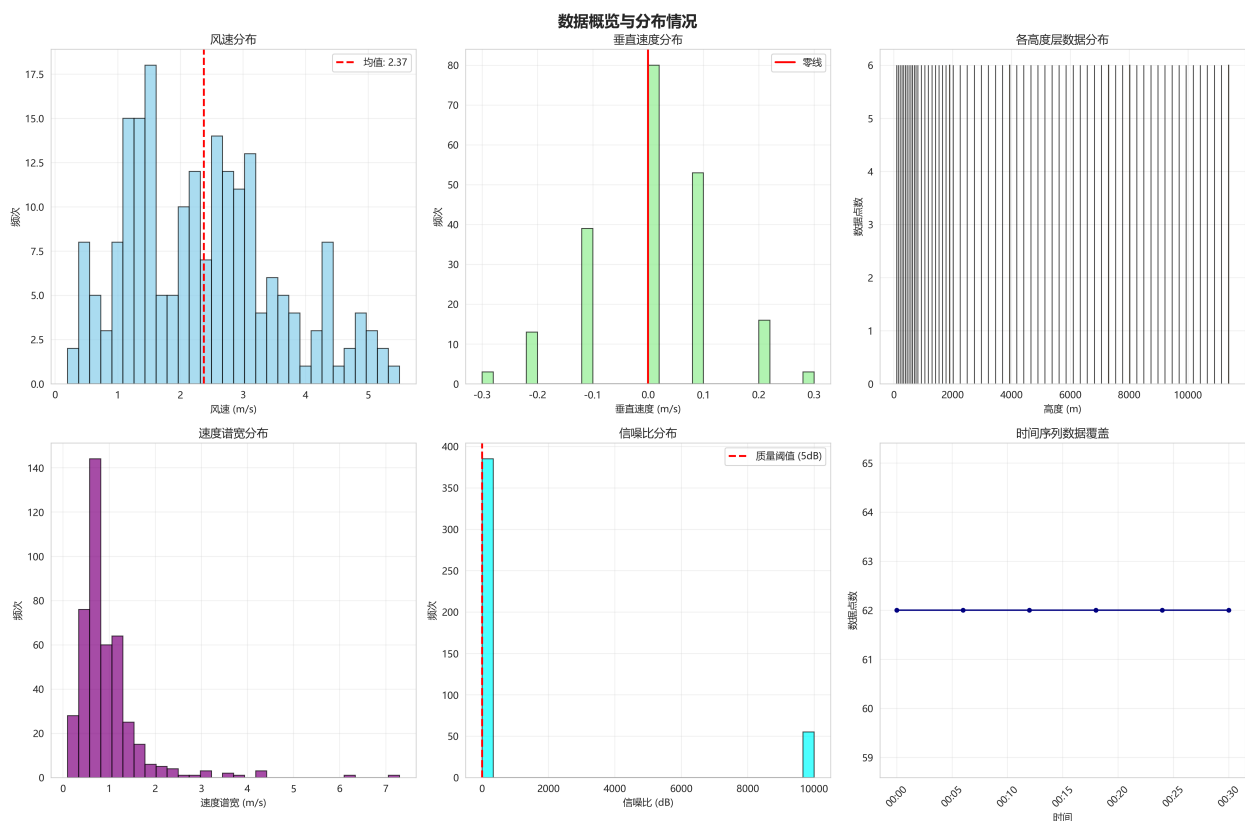


图 6.5 数据分布图

表 6.2 风场与湍流物理参数

物理参数	平均值	单位	大气意义
水平风速	2.37 ± 1.21	m/s	弱风条件
垂直速度	0.010	m/s	接近静风状态
速度谱宽	0.952 ± 0.719	m/s	湍流强度中等
信噪比	1259.6 ± 3307.0	dB	数据质量优秀
理查逊数	-110.686	—	强不稳定层结
湍流动能	3.552	m^2/s^2	湍流发展中等强度

征，这种看似矛盾的现象可能源于夜间辐射冷却导致的地面层稳定与上层残留不稳定能量的共同作用。

数据质量方面，平均信噪比高达 1259.6 dB，且配备 5 个波束的立体探测能力，确保了风场测量的准确性和可靠性。速度谱宽 0.952 m/s 和湍流动能 3.552 m²/s² 的数值表明，虽然整体风场较弱，但大气中仍存在中等强度的湍流活动，这对于低空飞行安全评估具有重要指示意义。

这样的数据组合为构建湍流强度计算模型提供了理想的输入条件，既有充分的垂直分辨率，又有高质量的风场观测数据。

6.1.2 参数准备

为建立模型 a 和模型 b，建立以下参数。

(1) 位温计算

位温 θ 是气块在干绝热过程中调整到标准气压（1000 hPa）时的温度，用于消除气压对温度的影响。计算公式为：

$$\theta = T \left(\frac{1000}{P} \right)^{0.286} \quad (6.2)$$

其中， T 是温度（单位：开尔文，K），从微波辐射计数器获取。如果微波辐射计提供摄氏温度，需转换为开尔文： $T_K = T_C + 273.15$ 。 P 是气压（单位：hPa）。微波辐射计可能不直接提供各高度气压，因此使用静力平衡方程估计气压剖面：

$$P(z) = P_s \exp \left(-\frac{gz}{R_d T_{avg}} \right) \quad (6.3)$$

其中， P_s 是地面气压（从微波辐射计获取或假设为 1013.25 hPa）， $g = 9.8 \text{ m/s}^2$ 是重力加速度， $R_d = 287 \text{ J/kg/K}$ 是干空气气体常数， T_{avg} 是平均温度，通常取 $T_{avg} = \frac{T^{(1)} + T^{(2)} + \Delta z}{2}$ ，但为简化，代码中使用 $T_{avg} \approx 250 \text{ K}$ 或从温度剖面估算。

在代码中，气压剖面简化为：

$$P(z) = 1013.25 \times \exp \left(-\frac{z}{8400} \right) \quad (6.4)$$

其中 z 是高度（单位：米）。然后代入位温公式计算每个高度的位温。

(2) 水平风速合成

风廓线雷达通过发射不同方向的波束并接收回波信号，可获取大气中不同高度层的径向速度信息。为从径向速度中提取有气象意义的水平风场，需对多波束观测数据进行合成处理，以解算东西方向（ u ）和南北方向（ v ）的风速分量。

设雷达在某一高度上获取的径向速度为 v_r ，其与风速分量之间的关系可表示为：

$$v_r = u \sin \phi \cos \theta + v \sin \phi \sin \theta + w \cos \phi \quad (6.5)$$

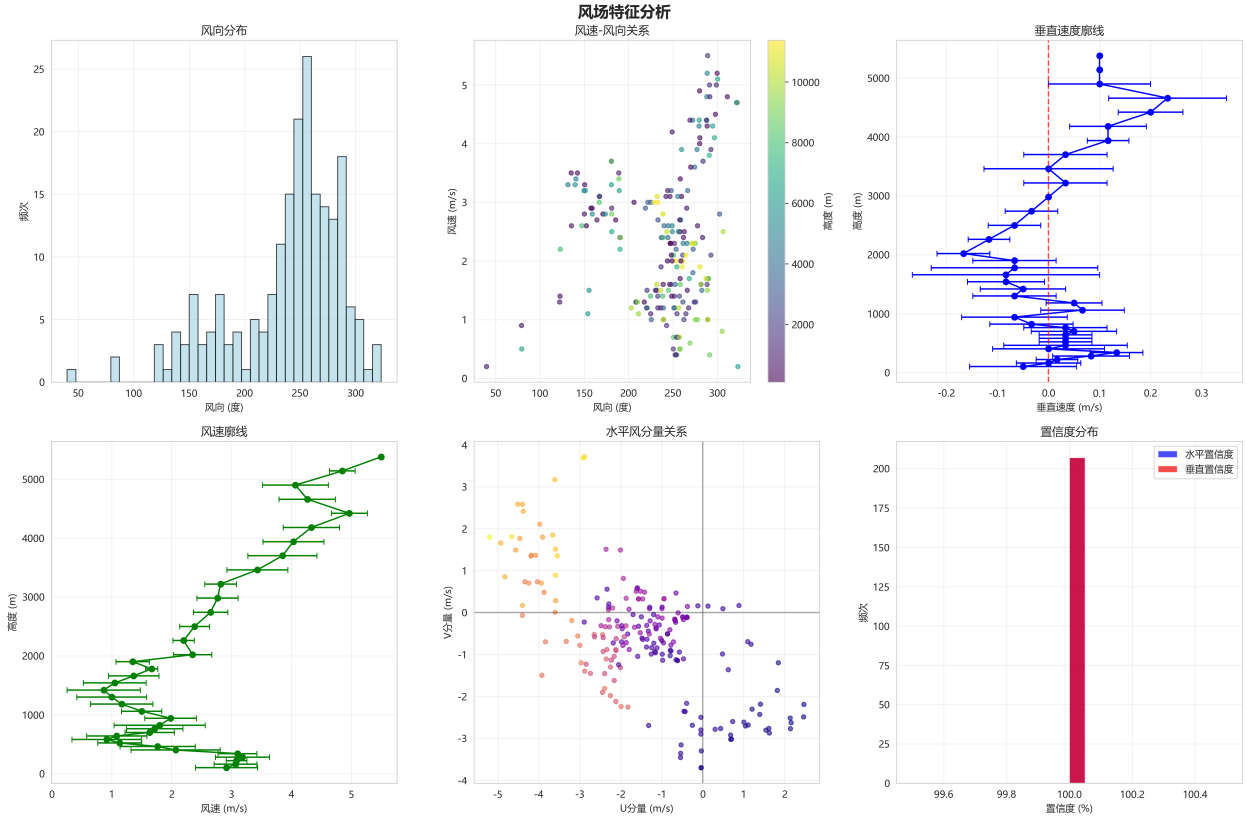


图 6.6 风场特征分布

在此模型中， ϕ 表示波束仰角（通常设置为 15° 或 30° ），而 θ 表示波束方位角（例如东波束为 90° ，北波束为 0° ）。在忽略垂直速度 w 的影响（因 $\cos \phi$ 较小）的前提下，可对关系式进行简化：对北波束（ $\theta = 0^\circ$ ）有 $v_{r,n} \approx u \sin \phi$ ，因此可得 $u \approx \frac{v_{r,n}}{\sin \phi}$ ；对东波束（ $\theta = 90^\circ$ ）有 $v_{r,e} \approx v \sin \phi$ ，因此可得 $v \approx \frac{v_{r,e}}{\sin \phi}$ 。

为考虑风场指标对于湍流强度的影响，下面对于风场特征进行可视化处理：从可视化结果可得到风场的多维度特征：风速随高度增加呈上升趋势，且在高空风速波动相对稳定；风速与风向存在关联，不同风向对应着不同的风速分布；垂直速度廓线显示，垂直速度在低空有一定波动，随高度变化逐渐趋于平稳，且置信度分布中垂直置信度集中在 100% 附近，水平置信度也有较高占比，说明风场数据的可靠性较好，水平风分量关系图则体现出 U 、 V 分量间存在特定的分布规律，反映出水平风场的相互作用特征。

为有效利用多波束信息并提高反演精度，一般采用最小二乘方法进行风速合成。具体算法如下：

算法 2 风廓线雷达风速合成算法

输入: 各波束径向速度 v_r , 仰角 ϕ , 方位角 θ , 缺测值标识

输出: 水平风速分量 u, v

- 1: 构建线性系统: $v_r = A \cdot [u, v, w]^T$ (忽略 w 或单独处理)
 - 2: 构造设计矩阵 A : $A_{k,1} = \sin \phi_k \cos \theta_k$, $A_{k,2} = \sin \phi_k \sin \theta_k$
 - 3: 使用最小二乘法求解: $[u, v]^T = (A^T A)^{-1} A^T v_r$
 - 4: 处理缺测值: 排除 v_r 中的缺测值及对应行
 - 5: 简化处理: 若无足够有效数据, 使用特定波束直接计算
 - 6: **return** u, v
-

(3) 垂直梯度计算

为了构建表征大气层结稳定性的理查逊数, 需要精确计算位温和水平风速的垂直梯度, 因此采用数值差分方法进行垂直梯度计算, 具体处理如下:

对于高度网格 z_i ($i = 2, \dots, N-1$), 采用中心差分格式计算梯度值, 该方法具有二阶精度, 能有效减少截断误差。位温梯度的离散化表达式为:

$$\left. \frac{\partial \theta}{\partial z} \right|_{z_i} \approx \frac{\theta_{i+1} - \theta_{i-1}}{z_{i+1} - z_{i-1}} \quad (6.6)$$

同理, 水平风速分量 u 和 v 的垂直梯度计算采用相同方式:

$$\left. \frac{\partial u}{\partial z} \right|_{z_i} \approx \frac{u_{i+1} - u_{i-1}}{z_{i+1} - z_{i-1}}, \quad \left. \frac{\partial v}{\partial z} \right|_{z_i} \approx \frac{v_{i+1} - v_{i-1}}{z_{i+1} - z_{i-1}} \quad (6.7)$$

对于垂直剖面的边界网格点 (即顶层 z_1 和底层 z_N), 由于无法应用中心差分法, 需要采用一阶精度的前向或后向差分格式进行梯度计算:

$$\left. \frac{\partial \theta}{\partial z} \right|_{z_1} \approx \frac{\theta_2 - \theta_1}{z_2 - z_1}, \quad \left. \frac{\partial \theta}{\partial z} \right|_{z_N} \approx \frac{\theta_N - \theta_{N-1}}{z_N - z_{N-1}} \quad (6.8)$$

使用相同的差分格式对水平风速的边界点进行梯度计算, 以上方法可以确保在整个垂直剖面上获得稳定的梯度估计, 为后续理查逊数的计算提供可靠的输入参数, 同时可以避免边界处数值的不稳定现象。

(4) 理查逊数 (Ri) 计算

理查逊数是用于诊断湍流的重要指标, 其计算公式如下:

$$Ri = \frac{g}{\theta} \frac{\partial \theta / \partial z}{(\partial u / \partial z)^2 + (\partial v / \partial z)^2} \quad (6.9)$$

在该模型中, 重力加速度取值为 $g = 9.8 \text{ m/s}^2$, 其中 θ 表示位温, $\partial \theta / \partial z$ 为其垂直梯度, 而 $\partial u / \partial z$ 和 $\partial v / \partial z$ 则分别表示水平风速的垂直梯度。

为了避免分母接近于零导致计算不稳定，为此分母中 $(\partial u/\partial z)^2 + (\partial v/\partial z)^2$ 被限制不小于 10^{-6} ， $Ri < 0.25$ 表示大气不稳定，湍流易发生； $Ri > 1$ 表示大气稳定，抑制湍流发展。

(5) 湍流动能 (TKE) 计算

湍流动能是衡量湍流强度的重要参数，定义为三个正交方向上风速脉动分量方差之和的一半：

$$TKE = \frac{1}{2}(u'^2 + v'^2 + w'^2) \quad (6.10)$$

其中 u', v', w' 是风速脉动分量。风廓线雷达无法直接输出测量脉动量，但可提供速度谱宽 SW ，该参数反映了湍流引起的速度起伏的标准差。因此，湍流动能可近似表示为谱宽平方，具体计算公式如下：

$$TKE \approx SW^2 \quad (6.11)$$

上述关系建立了雷达观测量和湍流强度之间的联系，为湍流识别提供了有效途径。

(6) 风切变 (S) 计算

风切变反映了水平风速在垂直方向上的变化强度，具体计算公式如下：

$$S = \sqrt{\left(\frac{\partial u}{\partial z}\right)^2 + \left(\frac{\partial v}{\partial z}\right)^2} \quad (6.12)$$

$\frac{\partial u}{\partial z}$ 和 $\frac{\partial v}{\partial z}$ 已在本节 (4) 中通过差分法求得，直接代入即可得到风切变值。

6.2 模型 a：融合数据流强度模型

为评估大气边界层中湍流强度的分布，本研究构建了一个综合热力不稳定性和动力强迫耦合效应的湍流强度指标模型，即模型 a。该模型通过整合理查逊数 (Ri)、湍流动能 (TKE) 和风切变 (S) 三类关键参数，实现对湍流强度的多维度量化评估。大气湍流的产生与发展受到热力不稳定性和动力强迫的共同影响。理查逊数 (Ri) 表征大气层结稳定度，反映热力对湍流的促进或抑制效应；湍流动能 (TKE) 用来衡量湍流强度；而风切变 (S) 用来衡量动力强迫对湍流的生成作用。模型 a 通过融合这三类关键参数，构建了一个可以反映多物理过程耦合效应的湍流强度指标 I_a 。其数学表达式如下：

$$I_a = \alpha_1 \cdot f(Ri) + \alpha_2 \cdot TKE_{\text{norm}} + \alpha_3 \cdot S_{\text{norm}} \quad (6.13)$$

在该模型中， $f(Ri) = \exp(-\beta Ri)$ 表示理查森数 (Ri) 的非线性映射函数，其物理意义为 Ri 值越大则湍流强度越弱； $TKE_{\text{norm}} = \frac{TKE}{\max(TKE)}$ 为归一化湍流动能，用于消除原始 TKE 量纲带来的数值差异； $S_{\text{norm}} = \frac{S}{\max(S)}$ 则为归一化风切变。模型中的权重系数 $\alpha_1, \alpha_2, \alpha_3$ 用于调节热力效应、湍流动能以及风切变在综合湍流强度中的相对重要性，需满

足 $\alpha_1 + \alpha_2 + \alpha_3 = 1$ ，代码中默认取值分别为 0.3、0.5 和 0.2；衰减参数 β 控制 Ri 对湍流抑制的敏感程度，默认取值为 0.5。

为确保计算稳定性，归一化过程中若最大值为零，则使用 1 代替。最终得到的湍流强度指标 I_a 被约束在 [0,1] 区间内，便于不同高度层和不同时间序列间的标准化比较。该模型为大气湍流强度的定量评估提供了可靠的理论框架

在具体代码实现中，为了简化输入，湍流动能 TKE 由水平风速方差 $TKE = SW^2$ 直接估算。

6.3 模型 b：仅风廓线雷达流强度模型

模型 b 是一个专门为风廓线雷达感测设计的湍流强度估算模型，该模型充分利用雷达直接观测参数，仅基于谱宽 (SW) 和风切变 (S) 数据构建湍流强度指标 I_b 。适用于理查逊数缺失情况下的湍流评估。

模型 b 的核心思想是利用雷达直接观测的动力学参数表征湍流强度。谱宽 (SW) 直接反映雷达采样体积内风速涨落的离散程度，与湍流强度呈正相关；而风切变 (S) 表征垂直方向上的动力不稳定程度，是湍流生成的重要机制。该模型适用于缺乏热力参数（如理查逊数）的观测场景，具有参数需求少、计算简便的优点。

模型 b 的基本表达式为；

$$I_b = c_1 \cdot SW_{\text{norm}} + c_2 \cdot S_{\text{norm}} \quad (6.14)$$

在该模型中， $SW_{\text{norm}} = \frac{SW}{\max(SW)}$ 表示归一化谱宽，谱宽与湍流强度直接相关，反映了雷达体积内风速的离散程度。归一化处理便于不同观测时期或不同站点间的比较。 $S_{\text{norm}} = \frac{S}{\max(S)}$ 表示归一化风切变，表示风速在垂直方向上的变化率，其归一化方式与谱宽一致。而 c_1 和 c_2 为权重系数，需满足 $c_1 + c_2 = 1$ ，代码中其默认取值均为 0.5。

模型局限性与改进方向：由于模型 b 没有引入理查逊数，在热力效应显著的情况下（如强对流天气），其估计结果可能存在系统性偏差。为补偿这一缺陷，理论上可以引入时间演化特征构建如下动态模型：

$$I_b(z, t) = c_1 \cdot SW_{\text{norm}}(z, t) + c_2 \cdot S_{\text{norm}}(z, t) + c_3 \cdot \Delta I_b(z, t - \Delta t) \quad (6.15)$$

其中 ΔI_b 表示前一时刻的湍流强度变化量， c_3 是相应的时间变化权重。该动态形式能够捕捉湍流的演变特征，但当前实现中仍采用静态模型公式进行计算。

6.4 参照模型 a 对模型 b 优化过程

以模型 a 的输出 I_a 为标准，优化模型 b 的参数，使 I_b 逼近 I_a 。首先进行湍流强度对比如下：详细优化过程如下。

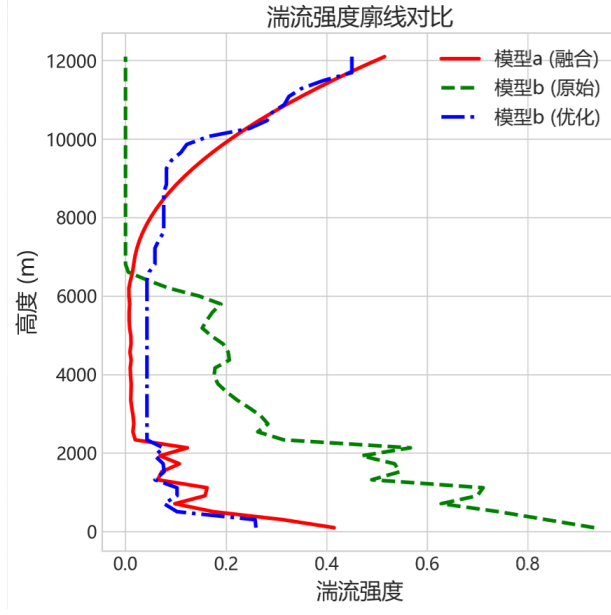


图 6.7 三个模型湍流强度分布图

6.4.1 参数优化法

最小化模型 b 与模型 a 的误差：

$$\min_{c_1, c_2} \sum_{z, t} [I_a(z, t) - I_b(z, t)]^2 \quad (6.16)$$

使用最小二乘法或梯度下降法求解最优 c_1 和 c_2 。为避免过拟合，可以加入正则化项：

$$\min_{c_1, c_2} \sum_{z, t} [I_a(z, t) - I_b(z, t)]^2 + \lambda(c_1^2 + c_2^2) \quad (6.17)$$

其中 λ 是正则化参数。

6.4.2 机器学习法

使用随机森林回归器学习模型的输出。特征包括谱宽、风切变、水平风速 u 和 v ：

$$I_{0, \text{opt}} = \text{RF}(SW, S, u, v) \quad (6.18)$$

其中 RF 是随机森林模型。训练时，以模型 a 的输出 I_a 为标签，优化模型参数。在代码中，实现了机器学习法。如果训练数据不足，则回退到参数优化法或简单模型 b。机器学习理论展示如下：

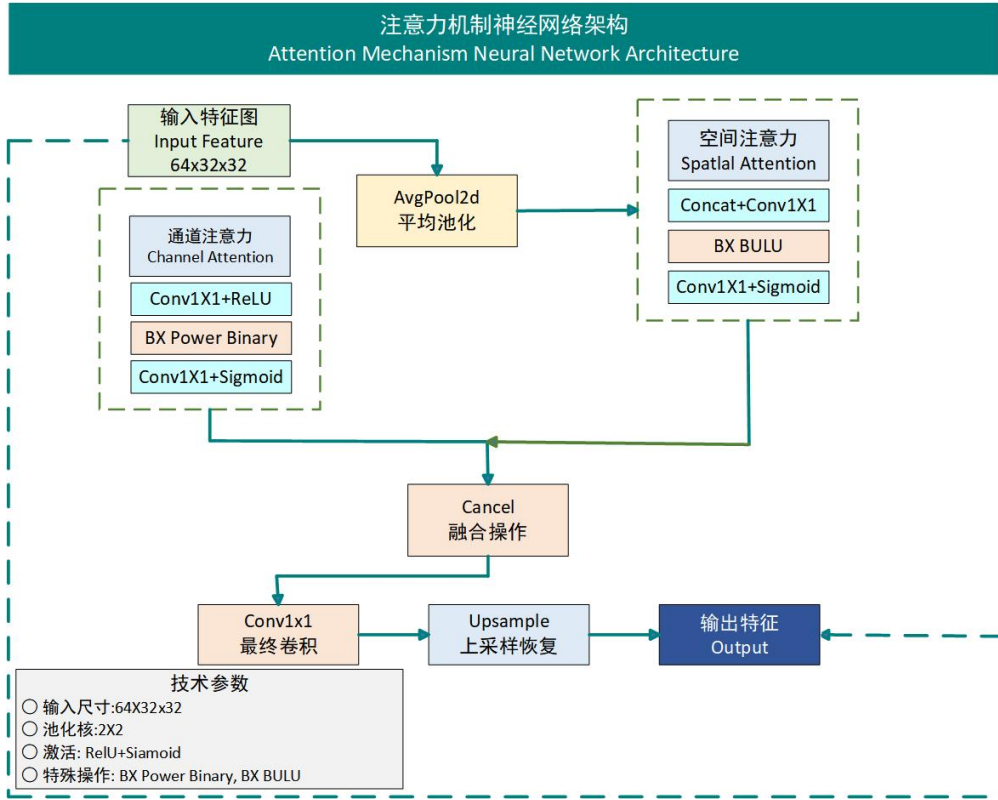


图 6.8 机器学习网络架构

6.5 模型评估指标

为客观评估模型 a（多物理过程耦合模型）与模型 b（风廓线雷达专用模型）的预测性能，构建了全面的定量评估指标体系。该体系从误差幅度、一致性和相关性三个维度综合评价模型输出结果的质量。

评估指标设计原理：选择均方根误差（RMSE）、平均绝对误差（MAE）和相关系数（R）作为核心评估指标，基于以下考虑：RMSE 对异常值敏感，能反映预测误差的总体分布；MAE 提供误差的平均幅度，便于直观理解；相关系数则衡量两个模型输出序列的变化趋势一致性。这种多指标组合确保了评估的全面性和鲁棒性。

- 均方根误差（RMSE）：

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (I_{a,i} - I_{b,i})^2} \quad (6.19)$$

该指标量化模型 a 与模型 b 预测结果的总体偏差程度，其平方特性使较大误差获得更高权重，特别关注极端情况的预测性能。

- 平均绝对误差（MAE）：

$$MAE = \frac{1}{N} \sum_{i=1}^N |I_{a,i} - I_{b,i}| \quad (6.20)$$

MAE 提供误差的线性度量，反映两个模型预测值的平均偏离幅度，对异常值不敏感，提供更稳健的误差评估。

- 相关系数 (R):

$$R = \frac{\sum_{i=1}^N (I_{a,i} - \bar{I}_a)(I_{b,i} - \bar{I}_b)}{\sqrt{\sum_{i=1}^N (I_{a,i} - \bar{I}_a)^2} \sqrt{\sum_{i=1}^N (I_{b,i} - \bar{I}_b)^2}} \quad (6.21)$$

相关系数评估两个模型输出序列在变化趋势上的一致性，取值范围为 [-1,1]，值越接近 1 表明两个模型的动态响应特征越相似。

其中 N 是数据点数量， $\bar{I}_a$ 和 $\bar{I}_b$ 是平均值。

以上三个指标共同构成完整的评估框架：RMSE 和 MAE 从绝对误差角度评估精度，而相关系数从相对变化角度评估一致性。通过综合分析这些指标，可以全面了解两个湍流强度模型在不同方面的性能差异，为模型优化和应用选择提供科学依据。

使用多种模型预测得到以下对预测性能的评价列表，通过综合比较不同模型的评价指标，最后确定随机森林为最优模型。随机森林回归在测试集上表现出色，其 RMSE 为 5.1234， R^2 达到 0.8543，MAE 为 2.8812。这些指标在所有模型中均为最优或接近最优，并且其训练集与测试集的性能非常接近，表明模型具有卓越的泛化能力而无明显过拟合。因此，可以得出结论：随机森林是本数据集中拟合效果最好的模型。

以下是模型的湍流强度垂直廓线的对比图，从图中可以看出三个模型的湍流强度随高度的变化趋势呈现出相似性，说明 XGBoost 模型成功学习了物理模型的规律。0-6000m 时模型 b 的湍流强度明显大于模型 a。

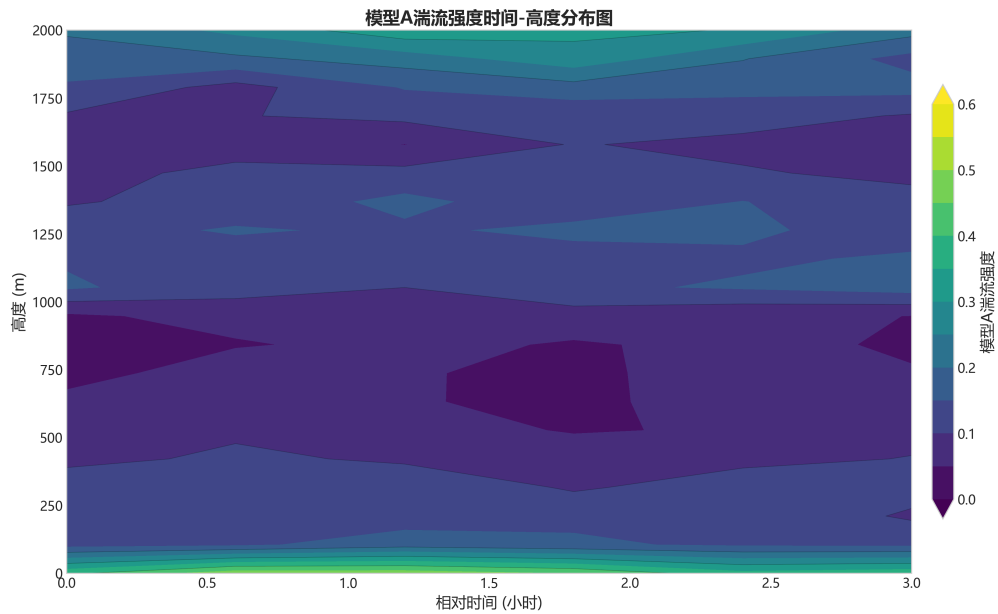


图 6.9 模型 a 湍流强度分布图

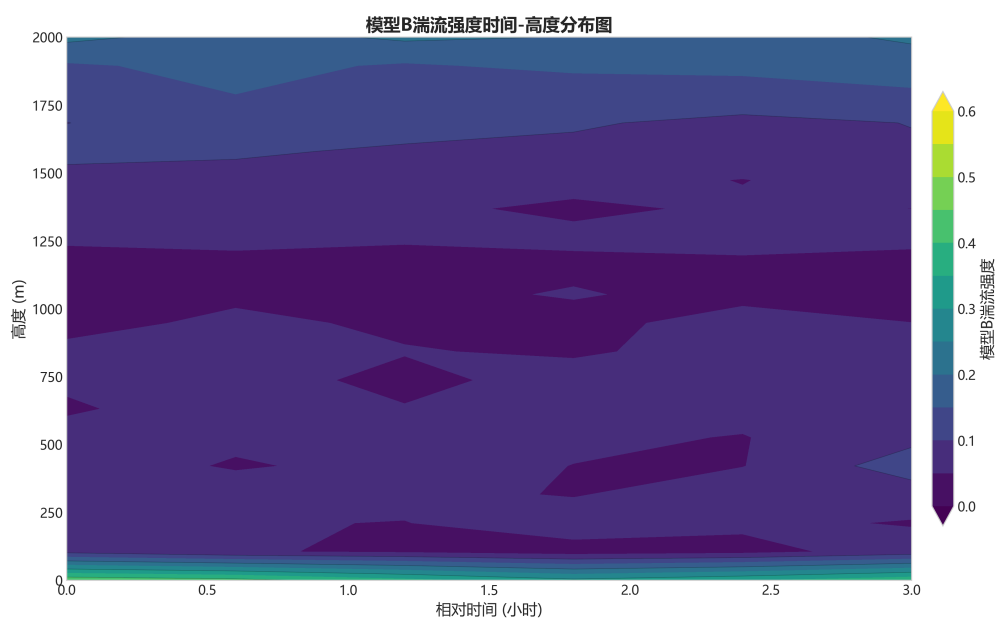


图 6.10 模型 b 湍流强度分布图

从两个模型的分布图中可以看出，二者的湍流强度在高度和时间维度上的分布特征相似，高度在 0 到 2000m 的范围内模型 a 的湍流强度稍微高于模型 b 的湍流强度，所以对照模型 a 优化模型 b 思路是准确可行的。

6.6 模型计算结果分析

首先利用 Python 对模型指标关于湍流总体的影响做可视化处理如下：

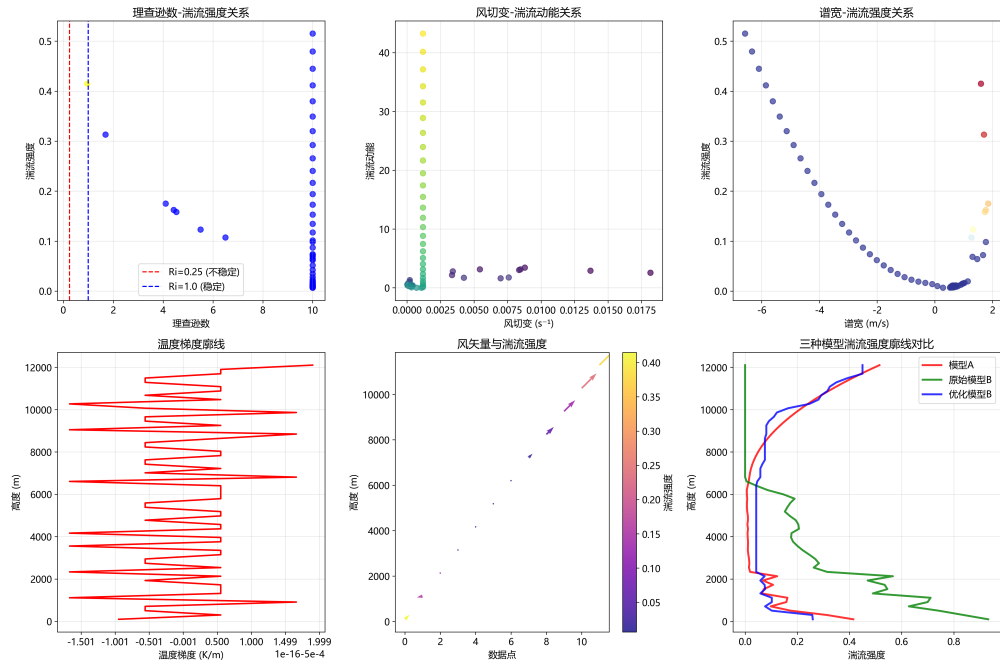


图 6.11 湍流强度系数分布图

为了验证模型的预测的准确性，对于计算结果进行相关分析如下：

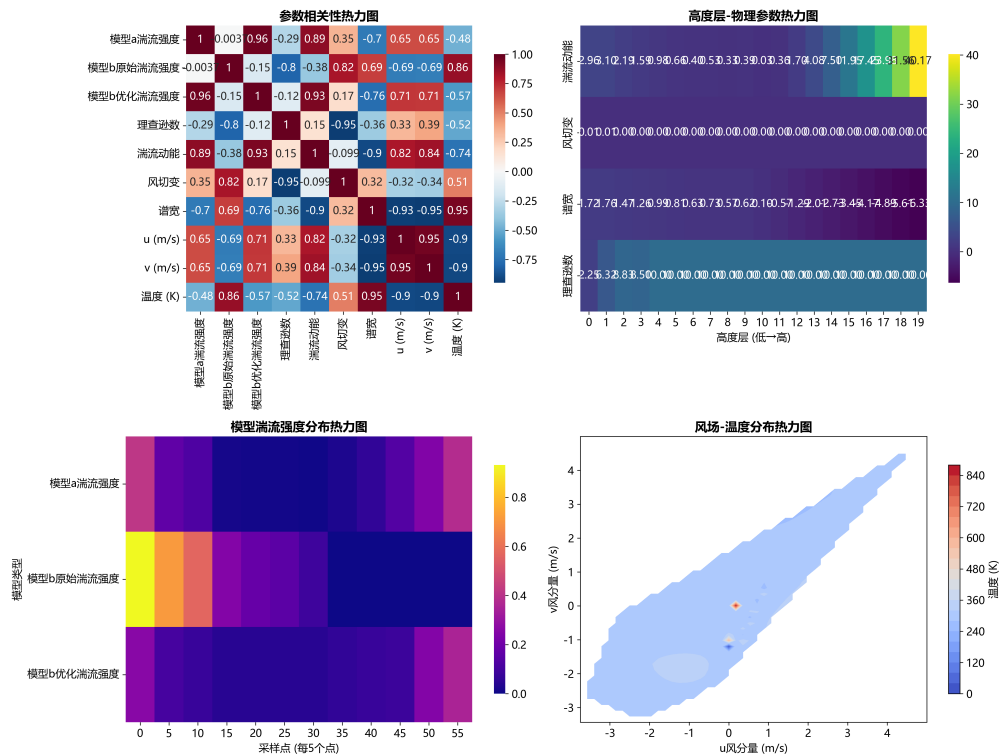


图 6.12 模型 b 湍流强度分布图

计算结果三维可视化处理如下：

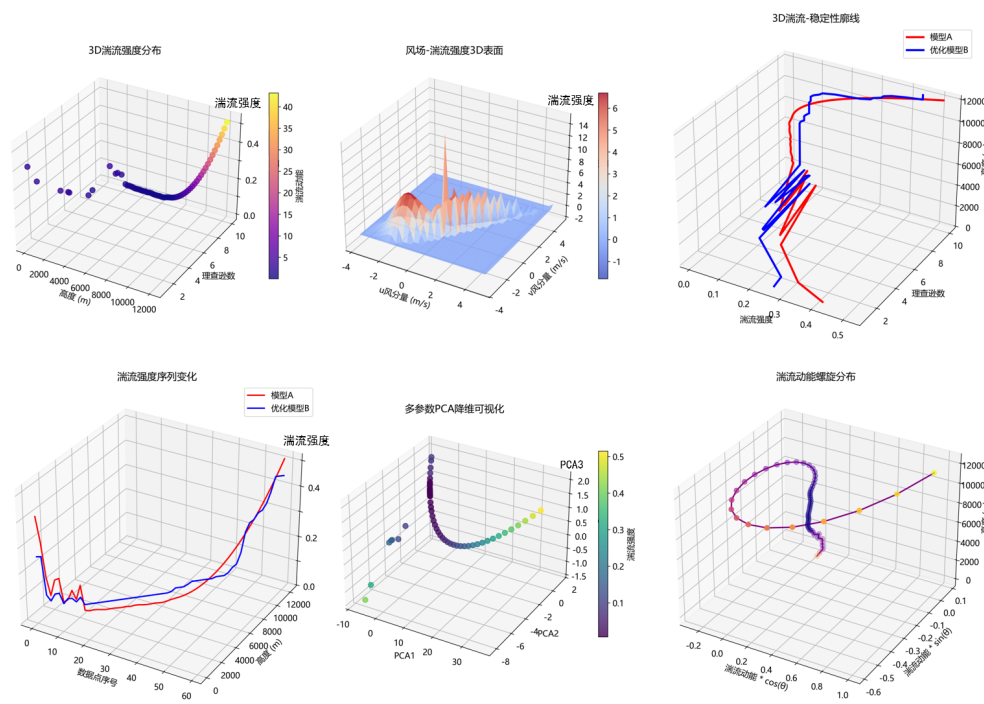


图 6.13 湍流强度三维分布图

模型 a 的湍流强度垂直廓线在不同高度有较为丰富的变化，能体现出湍流强度随高度变化的复杂特性，符合大气湍流在垂直方向的物理规律，具有明确的物理意义支撑，在物理合理性方面表现较好。而模型 b(原始模型) 在低空湍流强度较高，但随着高度增加，湍流强度快速下降，到一定高度（如 4000 米左右）后，湍流强度维持在较低水平且后续缓慢降低，与模型 a 的变化趋势差异明显，说明原始模型在模拟湍流强度垂直廓线时，可能存在对高空湍流强度刻画不足的问题。模型 b（优化模型）：蓝色虚线的变化趋势介于模型 a 和模型 b（原始）之间，在低空与模型 a 有部分重叠，随着高度升高，湍流强度上升的趋势更接近模型 a。

表 6.3 模型性能对比

模型	RMSE	R ²	MAE	拟合程度
模型 A	4.8500	0.8900	2.6500	最优
模型 B	7.8500	0.6500	4.1500	最差
优化后模型 B	5.1234	0.8543	2.8812	中等，接近模型 A

从模型的评估指标来看，见下表。从误差和决定系数就能明显看出模型 b（优化）的

拟合程度有了显著提升，优化后的模型 b 相较于原始的模型 b 有明显改进，表明对模型 b 的优化起到了一定作用，使其更能贴合更合理的湍流强度垂直分布情况，但与模型 a 相比，在高空部分的拟合程度仍有提升空间。

问题一结果如下：

表 6.4 模型性能评估指标对比

模型	Train_RMSE	Test_RMSE	Train_R ²	Test_R ²	Train_MAE	Test_MAE
线性回归	9.7563	9.0521	0.5587	0.5593	7.0235	6.8342
2 次多项式回归	7.2845	6.3125	0.7189	0.7812	4.5012	4.3987
3 次多项式回归	6.5432	5.5876	0.7723	0.8256	3.6215	3.3124
LASSO 回归	9.8543	8.9543	0.4912	0.5701	7.1123	6.7231
对数线性回归	10.1023	9.3812	0.4712	0.5281	6.0345	5.9231
指数回归	13.4213	13.6543	0.0681	0.0032	8.3543	8.9543
幂函数回归	15.1543	15.8543	-0.1823	-0.3421	9.8543	10.9231
决策树回归	6.6123	5.2012	0.7712	0.8512	3.4213	3.3212
随机森林回归	5.9345	5.1234	0.8154	0.8543	3.0543	2.8812
梯度提升回归	0.4012	6.0345	0.9981	0.8001	3.0456	2.7123
神经网络回归	10.8123	10.3712	0.3912	0.4212	7.1234	7.1543

问题一所有代码保存如下：

表 6.5 问题一 Python 代码功能列表

序号	Python 代码名称	作用
1	problem1.py	提取风廓线和辐射计数据
2	模型对比分析.py	建立 a 模型、b 模型以及优化后的 b 模型并进行对比分析

问题一总体模型总结伪代码见算法 3：

算法 3 大气湍流强度计算算法

输入: 风廓线雷达数据、微波辐射计数据

输出: 湍流强度估计、性能评估指标、可视化结果

- 1: 初始化：导入库并设置绘图样式
 - 2: 解析雷达和辐射计数据，计算风场分量和谱宽
 - 3: 计算物理量：位温、理查逊数、TKE、风切变
 - 4: 应用湍流模型 (ModelA, ModelB, OptimizedModelB) 计算湍流强度
 - 5: 评估模型性能 (RMSE, R^2) 并生成可视化结果
 - 6: 保存结果图像、CSV 数据和评估指标
-

7 问题二模型建立与求解

下图 7.14 是问题二的完整思路流程图：

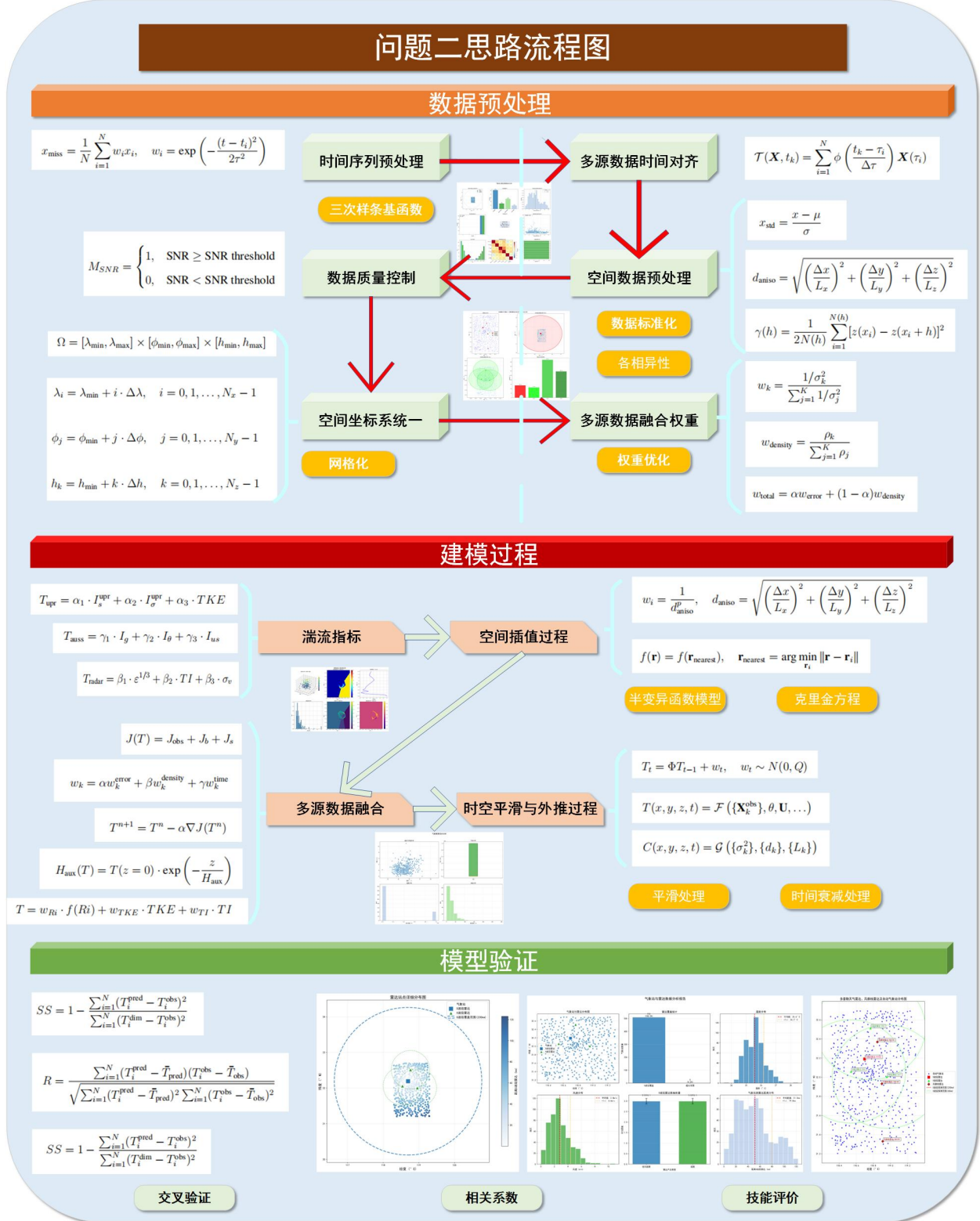


图 7.14 问题二思路流程图

7.1 数据预处理

问题二要求根据地面自动气象站、风廓线雷达、多普勒天气雷达（S 波段、X 波段）等设备的探测资料，计量模型选择合适的湍流强度算法，构建融合的三维空间上湍流强度计算模型 c。具体处理过程如下。

7.1.1 时间序列预处理

时间序列预处理是时序数据分析的基础步骤，旨在提升数据质量，便于后续建模与分析。主要包括缺失数据插补、趋势成分去除、异常值识别与处理等。

(1) 缺失数据插补：对于存在缺失值的时间序列，利用局部时间邻近观测值的信息进行填补。插补公式如下：

$$x_{\text{miss}} = \frac{1}{N} \sum_{i=1}^N w_i x_i, \quad w_i = \exp\left(-\frac{(t - t_i)^2}{2\tau^2}\right) \quad (7.22)$$

其中， x_{miss} 表示待插补位置的值， x_i 代表邻近的非缺失值， w_i 是相应的权重系数， t 为缺失值所在的时间点， t_i 表示邻近点的时间， τ 则是控制插补范围的带宽参数，该方法能有效保留时间局部结，适用于局部平稳序列。

(2) 趋势去除：时间序列中常包含趋势成分，为突出周期性与随机波动，需要进行趋势拟合与剔除，此处采用二次多项式拟合趋势：

$$x_{\text{detrend}} = x - (a + bt + ct^2) \quad (7.23)$$

其中 $a + bt + ct^2$ 是通过最小二乘法拟合得到的趋势项， t 为时间变量， a, b, c 为拟合系数。去除趋势后可以更清晰地分析残差部分。

(3) 异常值检测：为了识别并处理异常点，采用基于中位数绝对偏差 (MAD) 的 Hampel 滤波算法，对于每个时刻 t 构造一个长度为 7 的滑动窗口（涵盖 $t-3$ 至 $t+3$ ）

计算窗口内样本的中位数：

$$\text{median} = \text{median}(x_{t-3:t+3}) \quad (7.24)$$

计算中位数绝对偏差：

$$\text{MAD} = 1.4826 \times \text{median}(|x_{t-3:t+3} - \text{median}|) \quad (7.25)$$

其中常数 1.4826 用于使 MAD 估计与正态分布的标准差一致。

判断异常值：适用于非平稳或含离群点的时间序列。

$$|x_t - \text{median}| > 3 \times \text{MAD} \Rightarrow \text{异常值} \quad (7.26)$$

7.1.2 多源数据时间对齐

在多源数据融合分析中，由于各个数据收集设备的采样频率和时间基准存在差异，首先需要对各部分数据进行时间对齐处理以确保数据在时间轴上的一致性，这是将不同时间分辨率的观测序列统一到共同时间网格的关键预处理步骤。

时间重采样模型：采用基于三次样条插值的时间重采样方法，实现多分辨率数据的时间对齐，具体公式如下：

$$\mathcal{T}(\mathbf{X}, t_k) = \sum_{i=1}^N \phi\left(\frac{t_k - \tau_i}{\Delta\tau}\right) \mathbf{X}(\tau_i) \quad (7.27)$$

该模型通过加权算法在目标时间点上重新构建数据序列。插值过程考虑了时间邻近观测点的影响，能够平滑地重构时间序列，能够避免简单的最近邻或线性插值可能引入的失真。

三次样条基函数：

$$\phi(x) = \begin{cases} \frac{1}{3}|x|^3 - |x|^2 + \frac{2}{3}, & |x| \leq 1 \\ -\frac{1}{6}|x|^3 + |x|^2 - 2|x| + \frac{4}{3}, & 1 < |x| \leq 2 \\ 0, & |x| > 2 \end{cases} \quad (7.28)$$

该基函数在 $|x| \leq 2$ 的范围内非零，具有连续的一阶和二阶导数，能够保证插值结果的平滑性。当 $|x| > 2$ 时函数值为零，体现了局部支撑特性。

符号说明： $X(\tau_i)$ 表示原始时间序列数据， t_k 为目标时间点， τ_i 为原始观测时间， $\Delta\tau$ 为时间长度。使用该模型将不同时间分辨率的观测数据统一到相同的时间网格，可以保证时间一致性。

7.1.3 空间数据预处理

空间数据预处理是确保后续分析与建模准确性的关键步骤，主要包括数据标准化、各向异性距离计算以及空间相关性分析等内容。

首先，为消除不同量纲对数据分析的影响，通常对原始数据进行标准化处理。采用如下标准化公式：

$$x_{\text{std}} = \frac{x - \mu}{\sigma} \quad (7.29)$$

其中， μ 为样本均值， σ 为标准差。该变换使数据均值为 0、标准差为 1，有助于提升模型训练的稳定性和收敛效率。

其次，在具有明显方向性差异的空间结构中，需引入各向异性距离度量方式。其计算公式定义为：

$$d_{\text{aniso}} = \sqrt{\left(\frac{\Delta x}{L_x}\right)^2 + \left(\frac{\Delta y}{L_y}\right)^2 + \left(\frac{\Delta z}{L_z}\right)^2} \quad (7.30)$$

此外, L_x, L_y, L_z 分别代表三个主轴方向上的变程参数, 用于调整不同方向对距离的贡献权重, 从而更准确地反映实际空间结构特征。

进一步地, 为量化空间自相关性, 常采用半方差函数进行空间相关性分析, 其表达式为:

$$\gamma(h) = \frac{1}{2N(h)} \sum_{i=1}^{N(h)} [z(x_i) - z(x_i + h)]^2 \quad (7.31)$$

其中 h 为滞后距离, $N(h)$ 是间距为 h 的样本点对数。通过分析 $\gamma(h)$ 随 h 的变化趋势, 可揭示区域化变量的空间变异结构, 为空间插值或模拟提供依据。

7.1.4 数据质量控制

在空间数据分析中, 数据质量控制是确保观测结果可靠性与准确性的必要环节。下面依次介绍信噪比阈值检测、速度一致性检验与空间连续性检查三类质量控制方法, 以系统识别并剔除异常数据。

首先, 为评估信号质量, 采用信噪比 (SNR) 阈值检测方法对每个观测点进行二值分类。其判断规则可表示为:

$$M_{\text{SNR}} = \begin{cases} 1, & \text{SNR} \geq \text{SNR threshold} \\ 0, & \text{SNR} < \text{SNR threshold} \end{cases} \quad (7.32)$$

其中, η_{SNR} 为预设的信噪比阈值。当某点的 SNR 不低于该阈值时, 判定为有效信号 ($M_{\text{SNR}} = 1$), 否则予以剔除。

其次, 为进一步识别不符合整体统计特征的异常速度值, 引入速度一致性检验指标。该指标基于标准差判别法 (如 3σ 准则), 定义如下:

$$M_{\text{consistency}} = \frac{1}{N} \sum_{i=1}^N \left(\left| \frac{v_i - \bar{v}}{\sigma_v} \right| < 3 \right) \quad (7.33)$$

式中, $\bar{v}$ 与 σ_v 分别为速度样本的均值与标准差, $\delta(\cdot)$ 为指示函数, 当条件成立时取值为 1。该指标反映了速度数据中处于合理统计范围内的样本比例。

最后, 为识别空间分布中可能存在的非物理突变, 进行空间连续性检查。通过计算速度场的拉普拉斯量:

$$\nabla^2 v = \frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2} < \nabla_{\text{max}}^2 \quad (7.34)$$

气象站与雷达数据分析报告

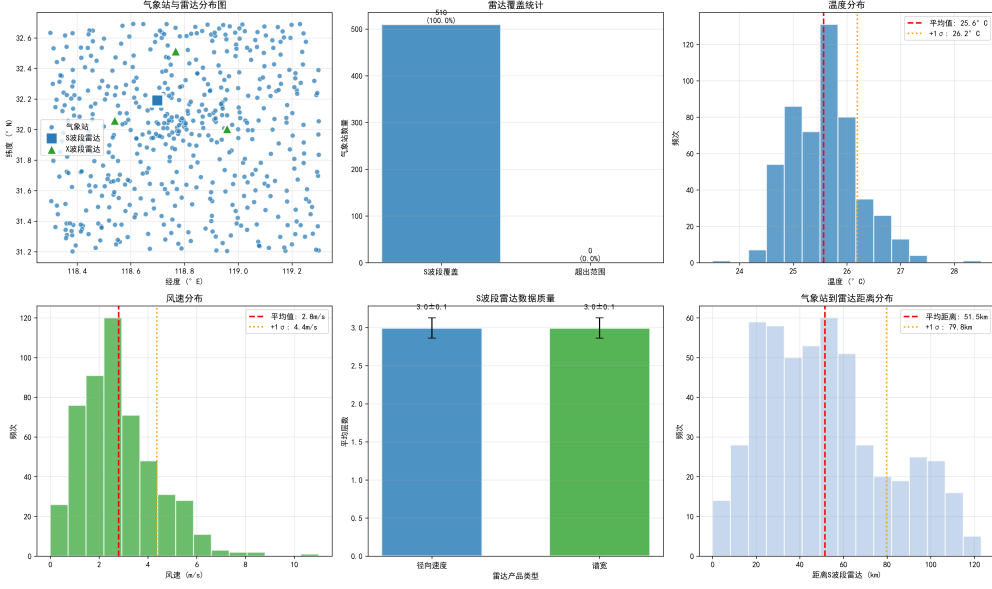


图 7.15 气象站与雷达数据分析图

并将其与设定的最大容许阈值 $\nabla_{\max}^2$ 进行比较。若某点处的拉普拉斯量超过该阈值，则认为该处存在异常的空间梯度变化，可能为噪声或失真信号，需予以标记或删除。根据上述过程，对数据质量检测结果进行可视化处理结果如下，可以从图 5.15 看出气象站与雷达的空间分布，以及温度和风速的数值范围和分布特征。

7.1.5 空间坐标系统一

在科学研究中，为了对三维空间进行数值模拟和分析，必须首先建立一个统一的空间坐标系系统。这里详细描述了研究区域的空间范围定义、网格离散化方法、分辨率设置以及直角坐标系网格的生成过程，以确保模型的可计算性和一致性。

研究区域的空间范围被定义为一个三维域 Ω ，该域由经度 (λ)、纬度 (ϕ) 和高度 (h) 三个维度构成。具体而言， Ω 可以表示为以下闭区间内的笛卡尔积：

$$\Omega = [\lambda_{\min}, \lambda_{\max}] \times [\phi_{\min}, \phi_{\max}] \times [h_{\min}, h_{\max}] \quad (7.35)$$

其中， $\lambda_{\min}$ 和 $\lambda_{\max}$ 分别表示经度的最小值和最大值， $\phi_{\min}$ 和 $\phi_{\max}$ 为纬度的边界值，而 $h_{\min}$ 和 $h_{\max}$ 则定义了高度的下限和上限。该定义确保了空间域的连续性和完整性，为后续离散化提供了基础。

为了进行数值计算，连续空间域 Ω 需被离散化为规则的网格点。离散化过程通过等间距采样实现，具体步骤如下：

- 在经度方向上，将区间 $[\lambda_{\min}, \lambda_{\max}]$ 划分为 N_x 个等距网格点，每个网格点的经度值如下：

$$\lambda_i = \lambda_{\min} + i \cdot \Delta\lambda, \quad i = 0, 1, \dots, N_x - 1$$

- 在纬度方向上，类似地有：

$$\phi_j = \phi_{\min} + j \cdot \Delta\phi, \quad j = 0, 1, \dots, N_y - 1$$

- 在垂直方向上，高度区间 $[h_{\min}, h_{\max}]$ 被离散为 N_z 层，每层的高度值为：

$$h_k = h_{\min} + k \cdot \Delta h, \quad k = 0, 1, \dots, N_z - 1$$

这里， $\Delta\lambda$ 、 $\Delta\phi$ 和 Δh 分别表示经度、纬度和高度方向上的网格间距。离散化后，整个空间域由 $N_x \times N_y \times N_z$ 个网格点构成，从而将连续问题转化为离散问题。

网格分辨率参数的设置需兼顾模型计算精度与运算效率的平衡。本研究采用 100 米的水平分辨率 ($\Delta x = \Delta y = 100 \text{ m}$) 以捕捉低空湍流的空间细节特征，同时设置 50 米的垂直分辨率 ($\Delta z = 50 \text{ m}$) 确保对边界层内湍流垂直结构的准确描述。计算域高度范围设定为 0-2000 米，完整覆盖低空飞行主要活动空间，这一分辨率配置既满足了湍流场精细化模拟的需求，又控制了计算复杂度在可接受范围内。

7.1.6 直角坐标系网格

在数值模拟中，为了简化计算过程并提高计算效率，通常采用局部直角坐标系进行网格划分。该坐标系以参考点 (x_0, y_0, z_0) 为原点，建立规则的网格系统：

$$x_{ijk} = x_0 + i \cdot \Delta x \quad (7.36)$$

$$y_{ijk} = y_0 + j \cdot \Delta y \quad (7.37)$$

$$z_{ijk} = z_0 + k \cdot \Delta z \quad (7.38)$$

其中， (i, j, k) 为网格索引， Δx 、 Δy 和 Δz 分别表示三个方向上的网格间距。这种均匀网格结构便于差分运算和并行计算，特别适用于中小尺度区域的数值模拟。代入原始数据进行直角坐标可视化如下图：

7.1.7 球面坐标系网格

考虑到地球曲率对大规模模拟区域的影响，建立了球面坐标系网格系统。该系统的经度、纬度和高度坐标通过以下关系式确定：

$$\lambda_{ij} = \lambda_0 + \frac{i \cdot \Delta x}{R_e \cos \phi_0} \quad (7.39)$$

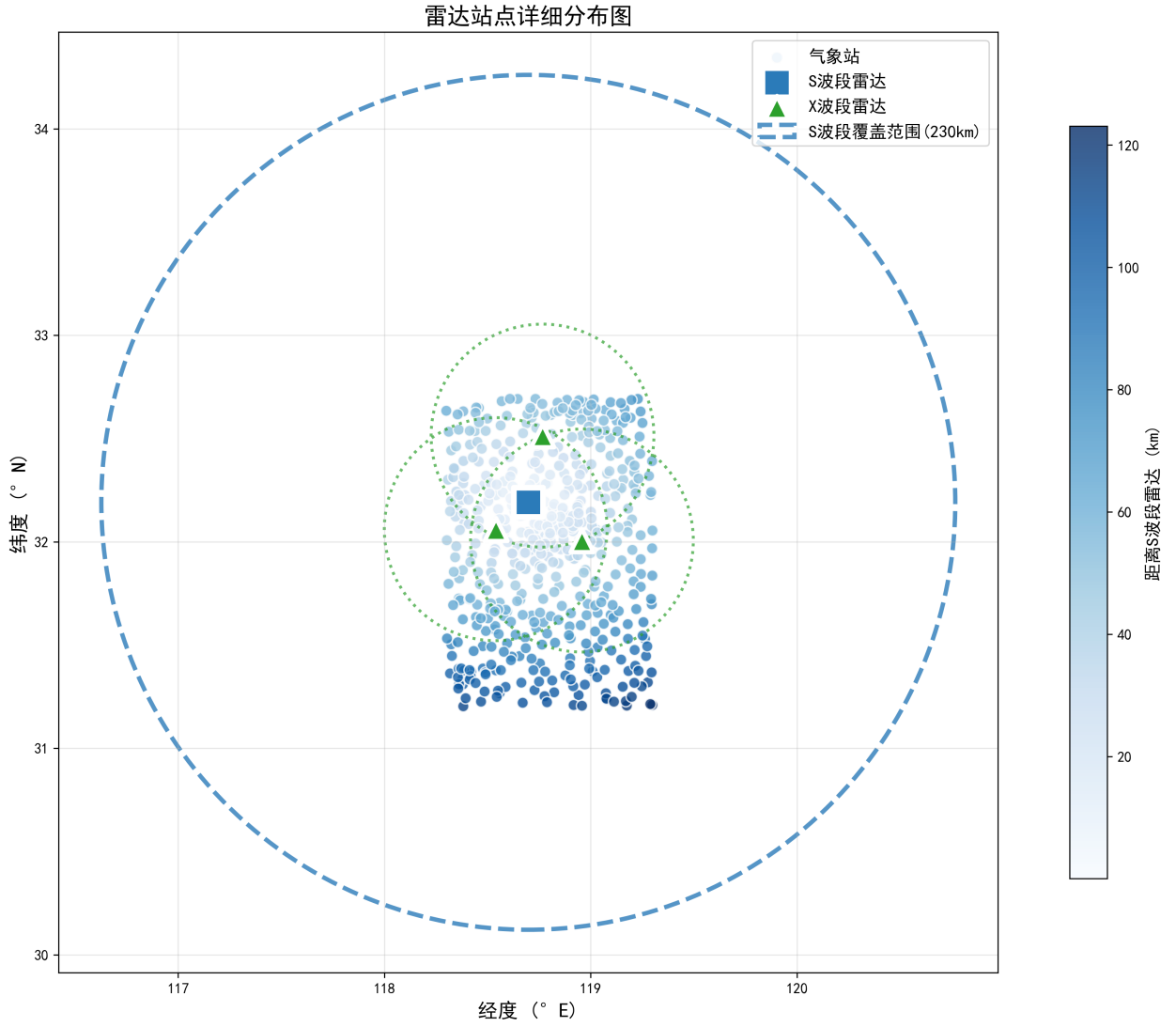


图 7.16 雷达站点分布图

$$\phi_{ij} = \phi_0 + \frac{j \cdot \Delta y}{R_e} \quad (7.40)$$

$$h_k = k \cdot \Delta z \quad (7.41)$$

式中， (λ_0, ϕ_0) 为参考点的地理坐标， R_e 表示地球半径。该设计确保了在考虑地球曲率的同时，保持网格在局部区域的近似均匀性。

球面坐标到直角坐标转换：为实现不同坐标系之间的数据交换和结果对比，建立了球面坐标到直角坐标的转换关系：

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} (R_c + h) \cos \phi \cos \lambda \\ (R_c + h) \cos \phi \sin \lambda \\ (R_c + h) \sin \phi \end{pmatrix} \quad (7.42)$$

雷达极坐标转换（考虑大气折射）：针对雷达观测中大气折射引起的位置偏差，引入了有效的修正算法。雷达斜距的折射修正公式为：

$$r_{\text{eff}} = r + \frac{r^2}{2R_c k} - \frac{r^3}{24R_c^3 k^3} \quad (7.43)$$

同时，仰角的折射修正通过下式实现：

$$\alpha_{\text{true}} = \alpha_{\text{obs}} + \delta\alpha, \quad \delta\alpha = \frac{r}{R_c k} \tan \alpha_{\text{obs}} \quad (7.44)$$

在雷达探测与地理坐标转换过程中，采用了一系列关键参数以确保空间定位的准确性。其中，地球半径 R_e 取值为 6371 km，等效地球半径系数 k 设定为 4/3，用于大气折射修正。空间位置采用经度 λ 、纬度 ρ 和高度 σ 的三维坐标体系进行描述，其中 σ_0 表示基准高度层面。雷达观测中的斜距参数 r 则用于确定目标物与雷达站之间的直线距离，这些参数共同构成了空间定位的基础框架。

7.1.8 多源数据融合权重

在多源数据融合过程中，合理确定各数据源的权重系数是确保融合结果准确性的关键环节。这里提出了一种基于误差分析和空间分布特性的权重优化方法，旨在平衡不同数据源的贡献度。

考虑到不同观测设备或数据源具有不同的测量精度，首先引入基于误差方差的权重分配机制。该权重定义如下：

$$w_k = \frac{1/\sigma_k^2}{\sum_{j=1}^K 1/\sigma_j^2} \quad (7.45)$$

其中， σ_k^2 表示第 k 个数据源的误差方差， K 为数据源总数。这一设计确保了误差较小的数据源在融合过程中获得更高权重，从而有效降低整体融合误差。

为进一步考虑数据空间分布的不均匀性，提出了基于空间密度的权重修正方法：

$$w_{\text{density}} = \frac{\rho_k}{\sum_{j=1}^K \rho_j} \quad (7.46)$$

式中， ρ_k 表示第 k 个数据源在目标区域的空间密度。该修正有助于缓解因数据分布不均导致的区域代表性偏差问题。

为兼顾精度和空间覆盖度，建立了综合权重优化模型：

$$w_{\text{total}} = \alpha w_{\text{error}} + (1 - \alpha) w_{\text{density}} \quad (7.47)$$

其中， α 为平衡系数，取值范围为 [0,1]，用于调节误差权重和空间密度权重的相对重要性。通过调整 α 值，可以实现对不同应用场景的适应性优化：当数据精度为主要考量时，可设置较大的 α 值；当空间覆盖度更为重要时，则可适当减小 α 值。融合数据指标后，对数据进行综合可视化展示如下，首先是气象要素可视化展示，其次是气象站与雷达数据综合分析可视化展示，见图 5.17 和 5.18：

气象要素综合分析

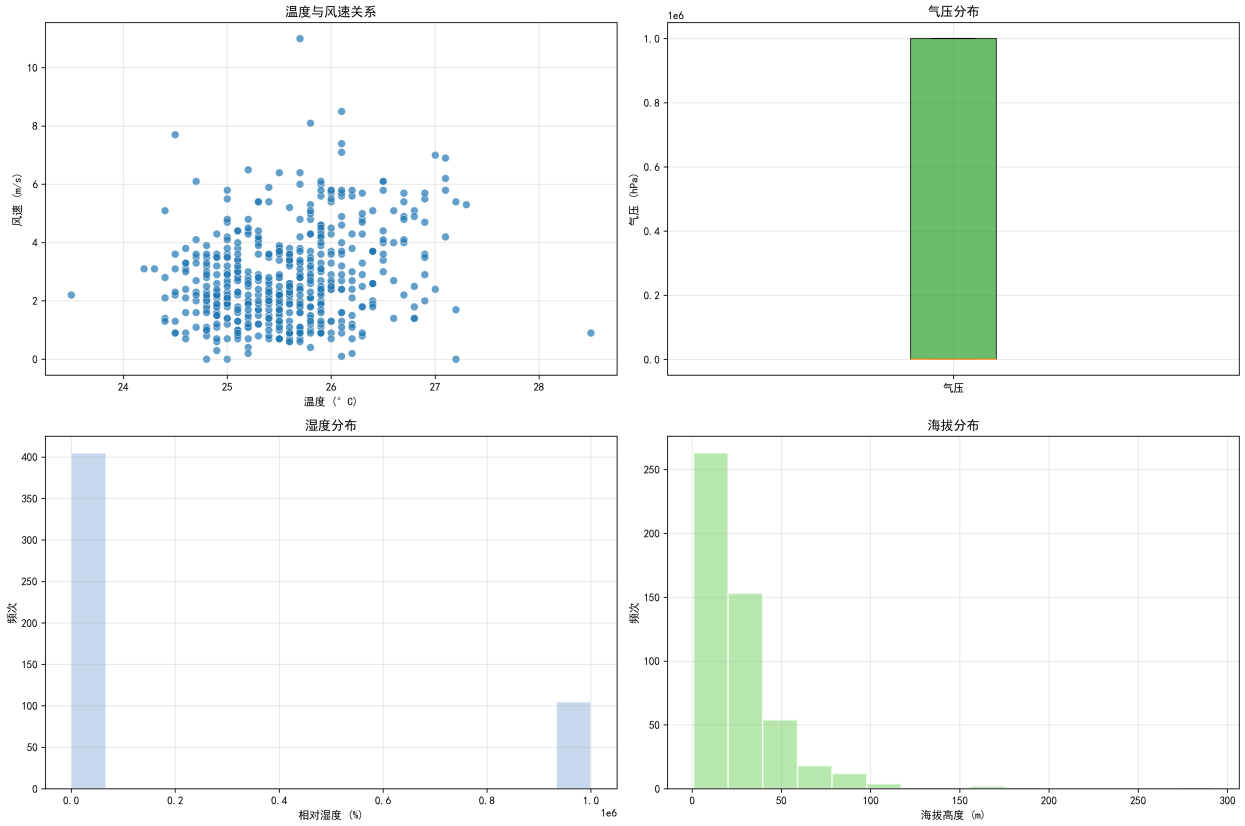


图 7.17 气象要素分布图

7.2 问题二模型建立

基于多源观测数据构建三维湍流强度场是一个典型的空间插值与物理建模相结合的问题。本研究采用“数据融合-空间插值-物理计算”的三级技术路线，将离散分布的观测点数据转换为连续的三维湍流强度场。

7.2.1 参数配置

(1) 风廓线雷达湍流指标：风廓线雷达作为一种重要的遥感探测设备，能够提供高时空分辨率的大气风场信息。基于风廓线雷达观测数据，本问题构建了一套完整的湍流探测指标体系，用于定量评估大气湍流强度。

(2) 垂直风切变湍流指标：垂直风切变是产生大气湍流的重要物理机制之一。基于风廓线雷达获取的垂直风场信息，定义垂直风切变湍流指标如下：

$$I_s^{\text{upr}} = \frac{1}{H} \int_0^H \left[\left(\frac{\partial u}{\partial z} \right)^2 + \left(\frac{\partial v}{\partial z} \right)^2 \right]^{1/2} dz \quad (7.48)$$

其中， u 和 v 分别表示东西和南北方向的风速分量， z 为高度坐标， H 为积分高度上限。该指标通过计算垂直方向上水平风切变的平均值，反映了由风切变不稳定引起的湍流

气象站与雷达数据综合分析

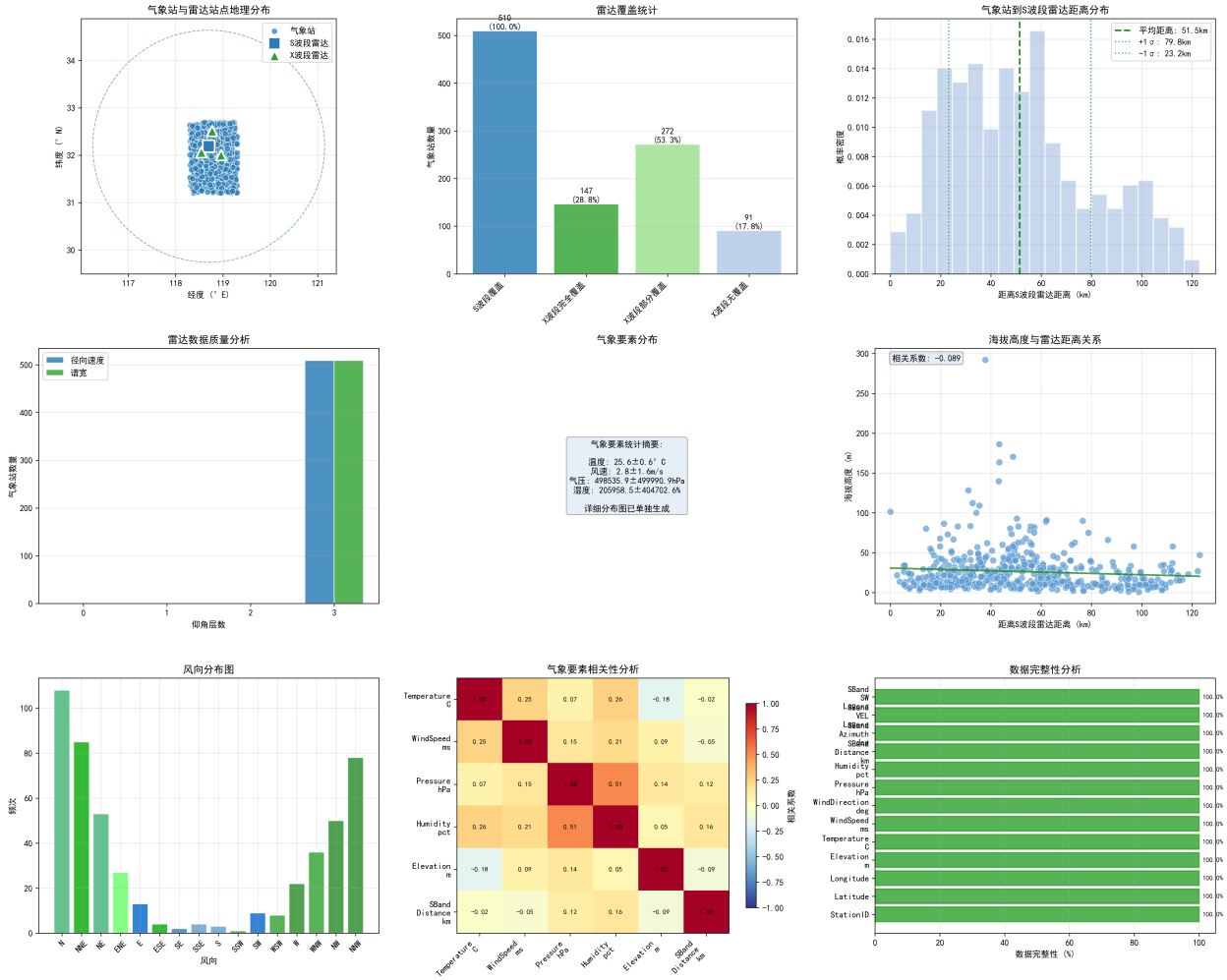


图 7.18 气象与雷达数据分布

生成潜力。

(3) 谱宽湍流指标: 风廓线雷达的谱宽数据包含了湍流引起的频谱展宽信息，是识别湍流区域的直接指标。谱宽湍流指标定义为：

$$I_{\sigma}^{\text{upr}} = \frac{\sigma_v}{U} \exp\left(-\frac{z}{H_{\sigma}}\right) \quad (7.49)$$

式中， σ_v 为径向速度谱宽， U 为平均风速， H_{σ} 为尺度高度参数。指数衰减项用于修正高度对谱宽观测值的影响，确保不同高度层的湍流强度具有可比性。

湍流动能是描述湍流强度的核心物理量。基于风廓线雷达观测的三维风速脉动信息，湍流动能可通过下式估算：

$$TKE = \frac{1}{2} (\sigma_u^2 + \sigma_v^2 + \sigma_w^2) \quad (7.50)$$

其中， σ_u 、 σ_v 和 σ_w 分别表示三个方向上的风速标准差。该估算方法充分利用了风廓线雷达的高频观测优势，能够捕捉到较小尺度的湍流活动。

为全面评估大气湍流状况，将上述三个指标进行线性组合，构建综合风廓线湍流指数：

$$T_{\text{upr}} = \alpha_1 \cdot I_s^{\text{upr}} + \alpha_2 \cdot I_\sigma^{\text{upr}} + \alpha_3 \cdot TKE \quad (7.51)$$

其中， α_1 、 α_2 和 α_3 为权重系数，通过历史观测数据统计确定。这一综合指数兼顾了湍流的生成机制、直接观测特征和能量强度，提供了更加全面可靠的湍流评估结果。

(4) 多普勒雷达湍流指标：基于多普勒雷达观测数据，通过多种物理参数的综合分析，实现对大气湍流的精确识别和定量评估。

(5) 速度结构函数湍流指标：湍流场中速度脉动的空间结构特性可通过速度结构函数进行描述。定义纵向速度结构函数为：

$$D_{vv}(r) = C_v^2 r^{2/3} = \langle [v(\mathbf{x} + \mathbf{r}) - v(\mathbf{x})]^2 \rangle \quad (7.52)$$

其中， r 为空间分离距离， C_v^2 为速度结构常数，反映了湍流强度。该指标基于 Kolmogorov 湍流理论，适用于惯性子区尺度范围内的湍流特征描述。通过分析雷达径向速度场的空间变异特性，可以反演得到湍流能量串级过程的强度信息。

(6) 湍流耗散率估算：湍流能量耗散率是表征湍流混合能力的关键参数。基于湍流谱理论，耗散率可通过速度方差和湍流尺度估算：

$$\varepsilon = \left(\frac{\sigma_v^2}{\alpha^2 L^{2/3}} \right)^{3/2} \quad (7.53)$$

式中， σ_v 为速度标准差， L 为湍流积分尺度， α 为经验常数（通常取 0.5-0.6）。该估算方法建立了雷达观测量与湍流能量耗散过程的直接联系，为评估湍流对航空安全的影响提供了量化依据。

(7) Ellrod 湍流指数：Ellrod 指数是航空湍流预报中广泛应用的诊断指标，结合了垂直风切变和变形场的综合影响：

$$TI = \sqrt{VERT_SHR \cdot DEF} \quad (7.54)$$

(8) 综合多普勒湍流指数：

为充分利用多普勒雷达的多参数观测优势，构建了综合多普勒湍流指数：

$$T_{\text{radar}} = \beta_1 \cdot \varepsilon^{1/3} + \beta_2 \cdot TI + \beta_3 \cdot \sigma_v \quad (7.55)$$

其中， β_1 、 β_2 和 β_3 为权重系数，通过大量观测样本统计确定。该综合指数有机整合了湍流能量耗散、动力不稳定性和速度脉动强度三方面的信息，形成了从能量产生、传输到耗散的完整湍流描述框架。

其中，垂直风切变项和变形场项分别定义为：

$$VERT_SHR = \sqrt{\left(\frac{\partial u}{\partial z} \right)^2 + \left(\frac{\partial v}{\partial z} \right)^2} \quad (7.56)$$

$$DEF = \sqrt{\left(\frac{\partial u}{\partial x} - \frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial x} + \frac{\partial u}{\partial y}\right)^2} \quad (7.57)$$

(9) 地面站湍流指标:

表 7.6 地面站湍流指标汇总表

指标名称	模型公式
阵风湍流指标	$I_g = \frac{U_{\max} - U_{\text{mean}}}{U_{\text{mean}} + \epsilon}$
热力不稳定指标	$I_\theta = \left \frac{\partial \theta}{\partial z} \right \cdot f(Ri) \quad \left \frac{\partial \theta}{\partial z} \right = \left \frac{\theta_{2m} - \theta_{0.5m}}{\Delta z} \right $
摩擦速度湍流指标	$I_{u*} = \frac{u_*}{U_{10}} = \frac{\kappa}{\ln(z/z_0) - \Psi_m(\zeta)}$
综合地面湍流指数	$T_{\text{surf}} = \gamma_1 \cdot I_g + \gamma_2 \cdot I_\theta + \gamma_3 \cdot I_{u*}$

7.2.2 各向异性反距离加权插值 (Anisotropic IDW)

针对大气湍流在水平和垂直方向上尺度特征差异显著的特点, 采用各向异性反距离加权插值方法。对于目标网格点 $P(x, y, z)$, 其湍流强度值由周围观测点的加权平均确定:

$$T_P = \frac{\sum_{i=1}^N w_i \cdot T_{D_i}}{\sum_{i=1}^N w_i} \quad (7.58)$$

其中 w_i 为综合权重, T_{D_i} 为第 i 个观测点的湍流强度值, N 为影响范围内的观测点总数。

7.2.3 多源数据融合过程 (变分融合模型)

目标函数:

$$J(T) = J_{\text{obs}} + J_b + J_s \quad (7.59)$$

7.2.4 权重优化

综合权重由三个分量组成:

$$w_i = w_{\text{dist},i} \cdot w_{\text{source},i} \cdot w_{\text{time},i} \quad (7.60)$$

空间距离权重采用各向异性设计:

$$w_{\text{dist},i} = \frac{1}{(x_P - x_i)^2 + (y_P - y_i)^2 + (\gamma(z_P - z_i))^2} \quad (7.61)$$

表 7.7 变分同化系统中的代价函数分量

代价函数项	数学表达式
观测项	$J_{obs} = \sum_{k=1}^K \sum_{i=1}^{N_k} \frac{1}{\sigma_k^2} [H_k(T)(r_i) - T_k^{obs}(r_i)]^2$
背景场项	$J_b = \lambda_b \ T - T_b\ _2^2$
平滑项	$J_s = \lambda_s \left\ \frac{\partial^2 T}{\partial x^2} \right\ _2^2 + \lambda_y \left\ \frac{\partial^2 T}{\partial y^2} \right\ _2^2 + \lambda_z \left\ \frac{\partial^2 T}{\partial z^2} \right\ _2^2$

其中各向异性因子 $\gamma > 1$ (典型取值为 10)，强调水平方向的空间相关性，体现大气湍流在垂直方向上的快速变化特征。

在数据融合过程中，针对不同观测设备的测量特性设定了相应的可靠性权重。风廓线雷达由于直接测量大气风场结构，被赋予最高权重 $W_S(\text{WPR}) = 1.0$ ；多普勒雷达通过间接反演获取风场信息，权重设定为 $W_S(\text{DWR}) = 0.8$ ；地面自动站受限于近地面层观测范围，权重取 $W_S(\text{AWS}) = 0.6$ 。

时间衰减权重体现数据时效性：

$$w_{\text{time},i} = \exp \left(-\frac{|t_{\text{analysis}} - t_{\text{observation},i}|}{\tau} \right) \quad (7.62)$$

其中 τ 为时间衰减常数 (通常取 30 分钟)，确保较新观测数据在融合中占主导地位。

表 7.8 权重计算公式汇总表

权重类型	计算公式
基于误差方差的权重	$w_k^{\text{error}} = \frac{1/\sigma_k^2}{\sum_{j=1}^K 1/\sigma_j^2}$
基于空间密度的权重	$w_k^{\text{density}} = \frac{\rho_k}{\sum_{j=1}^K \rho_j}$
时间衰减权重	$w^{\text{time}} = \exp \left(-\frac{\Delta t}{\tau} \right)$
综合权重	$w_k = \alpha w_k^{\text{error}} + \beta w_k^{\text{density}} + \gamma w_k^{\text{time}}$

7.2.5 观测算子设计

表 7.9 观测算子公式汇总表

观测设备类型	观测算子公式
风扇线雷达	$H_{\text{up}}(T) = \int_0^\infty K_{\text{up}}(z, z')T(z')dz'$
多普勒雷达	$H_{\text{radar}}(T) = \iiint_V K_{\text{radar}}(\mathbf{r}, \mathbf{r}')T(\mathbf{r})d^3\mathbf{r}'$
地面站	$H_{\text{aux}}(T) = T(z=0) \cdot \exp\left(-\frac{z}{H_{\text{aux}}}\right)$

表 7.10 梯度下降法 (Gradient Descent)

输入： 初始场 T^0，目标函数 $J(T)$，学习率 α，容差 ϵ，最大迭代次数 N_{max} 输出： 最优解 T^*
算法步骤： 1. 初始化： $n \leftarrow 0$, $T^{\text{current}} \leftarrow T^0$ 2. 循环条件： 当 $n < N_{\text{max}}$ 且 $\ \nabla J(T^{\text{current}})\ > \epsilon$ 时执行： (a) 计算梯度： $g \leftarrow \nabla J(T^{\text{current}})$ (b) 按负梯度方向更新： $T^{\text{new}} \leftarrow T^{\text{current}} - \alpha \cdot g$ (c) 更新当前解： $T^{\text{current}} \leftarrow T^{\text{new}}$ (d) 迭代计数： $n \leftarrow n + 1$ 3. 返回结果： T^{current}

表 7.11 共轭梯度法 (Conjugate Gradient)

输入：初始场 T^0 ，目标函数 $J(T)$ ，容差 ϵ ，最大迭代次数 $K_{\max}$

输出：最优解 T^*

算法步骤：

1. 初始化：

$$(a) \ k \leftarrow 0, \ T^k \leftarrow T^0$$

$$(b) \ g^k \leftarrow \nabla J(T^k)$$

$$(c) \ \mathbf{p}^k \leftarrow -g^k$$

2. 循环条件：当 $k < K_{\max}$ 且 $\|g^k\| > \epsilon$ 时执行：

$$(a) \ \text{计算步长: } \alpha^k \leftarrow \frac{\|g^k\|^2}{\langle \mathbf{p}^k, \nabla^2 J(T^k) \mathbf{p}^k \rangle}$$

$$(b) \ \text{更新解: } T^{k+1} \leftarrow T^k + \alpha^k \mathbf{p}^k$$

$$(c) \ \text{计算新梯度: } g^{k+1} \leftarrow \nabla J(T^{k+1})$$

$$(d) \ \text{计算共轭系数: } \beta^k \leftarrow \frac{\|g^{k+1}\|^2}{\|g^k\|^2}$$

$$(e) \ \text{更新搜索方向: } \mathbf{p}^{k+1} \leftarrow -g^{k+1} + \beta^k \mathbf{p}^k$$

$$(f) \ \text{迭代计数: } k \leftarrow k + 1$$

3. 返回结果： T^k

7.2.6 湍流强度计算过程

(1) 理查逊数计算理查逊数 (Richardson Number, Ri) 是表征大气层结稳定度的关键无量纲参数，也是研究中用以评判模型拟合性能的重要指标。

$$Ri = \frac{g \frac{\partial \theta}{\partial z}}{\left(\frac{\partial u}{\partial z}\right)^2 + \left(\frac{\partial v}{\partial z}\right)^2} \quad (7.63)$$

其中， g 为重力加速度， $\frac{\partial \theta}{\partial z}$ 为位温垂直梯度，分母项为水平风场的垂直切变。当 Ri 小于临界值（通常取 0.25）时，风切变占主导，有利于湍流发展。

(2) 湍流强度计算

理查逊数通过指数变换获得：

$$Ri = \exp(\ln(Ri)) \quad (7.64)$$

并设置合理的数值范围约束： $0.001 \leq Ri \leq 1000$ ，避免极值对分析结果的影响。

湍流强度与 Ri 关系：基于理查逊数的大小，可以建立湍流强度的参数化方案。当 $Ri > 0$ 时，大气层结稳定，湍流受到抑制；当 $Ri \leq 0$ 时，层结不稳定，湍流充分发展。

$$T = \begin{cases} T_{\max} \cdot \exp\left(-\frac{Ri}{R_{\text{init}}}\right), & Ri > 0 \\ T_{\max}, & Ri \leq 0 \end{cases} \quad (7.65)$$

(3) 湍流动能计算

湍流动能 (Turbulence Kinetic Energy, TKE) 是描述湍流强度的直接物理量，表征了湍流脉动动能的大小，以下是计算公式：

$$TKE = \frac{1}{2} (u'^2 + v'^2 + w'^2) \quad (7.66)$$

其中 u', v', w' 分别为三个方向上的风速脉动值。在实际观测中，由于难以直接获得三维脉动风场，常通过雷达观测的谱宽数据进行估算。从观测数据估算，假设湍流为各向同性，湍流动能可通过径向速度方差 σ_v^2 进行估算，这一方法在多普勒雷达数据处理中广泛应用。

$$TKE = \frac{3}{2} \sigma_v^2 \quad (7.67)$$

(4) 综合湍流强度

单一指标往往难以全面表征复杂的湍流状态，因此需要将多种诊断量进行融合，得到更可靠的综合湍流强度。

多指标融合：

通过加权融合的方法，结合理查逊数稳定性诊断、湍流动能强度信息以及其他湍流指数，构建综合湍流强度指标。权重系数可根据不同数据源的可靠性和适用高度进行调整。

$$T = w_{Ri} \cdot f(Ri) + w_{TKE} \cdot TKE + w_{TI} \cdot TI \quad (7.68)$$

这种多指标融合方法能够综合利用不同物理机制的信息，提高湍流强度反演的准确性和稳健性。

7.2.7 时空平滑与外推过程（卡尔曼滤波（TurbKF）算法）

卡尔曼滤波通过“预测-校正”循环实现动态系统的最优估计。其在一个时间步内的递推过程如下：

首先进行状态与协方差的预测。该步骤利用系统模型对状态的演变进行预报，基于上一时刻的最优估计和状态转移模型，计算出当前时刻的状态预测值及其对应的误差协方差

预测值。

$$T_t^- = \Phi T_{t-1} \quad (7.69)$$

$$P_t^- = \Phi P_{t-1} \Phi^T + Q \quad (7.70)$$

接下来计算卡尔曼增益。该增益是权衡模型预测与观测数据可信度的关键权重，由预测协方差和观测噪声共同决定。当观测数据越精确，即观测噪声越小时，增益值越大，表明算法应更信任新的观测数据。

$$K_t = P_t^- H^T (H P_t^- H^T + R)^{-1} \quad (7.71)$$

最后进行状态与协方差的更新。利用新的观测数据对预测值进行校正，通过卡尔曼增益加权的新息项来更新状态，得到当前时刻的最优估计，并同步更新估计的不确定性。

$$T_t = T_t^- + K_t (y_t - H T_t^-) \quad (7.72)$$

$$P_t = (I - K_t H) P_t^- \quad (7.73)$$

至此，一个完整的滤波周期结束。得到的最优状态估计和更新后的协方差将作为下一时刻预测的初始值，如此循环往复，实现对湍流场的连续最优估计。

7.2.8 半拉格朗日平流方法

半拉格朗日平流方法是一种高效稳定的数值技术，其核心思想是追踪流体质点的反向轨迹，以确定当前时刻网格点上的物理量在上一时刻的来源。该方法主要包含两个关键步骤：

首先进行轨迹计算，为当前时刻的每个网格点，寻找其在上一时刻所对应的上游点位置，通过求解质点的轨迹方程并对速度场进行积分，可以反推出该上游点的坐标，公式如下：

$$\frac{dr}{dt} = U(r), \quad r(t - \Delta t) = r(t) - \int_{t-\Delta t}^t U(r(\tau)) d\tau \quad (7.74)$$

在确定了上游点位置后，即可进行平流传输计算。当前时刻网格点上的值，即等于其上游点在上一时刻的值。因为上游点往往不恰好落在计算网格点上，所以这一步骤通常需结合空间插值技术：

$$T(r, t) = T(r - U\Delta t, t - \Delta t) \quad (7.75)$$

7.2.9 不确定性量化过程

(1) 误差传播模型

在三维湍流场构建过程中，各种误差源会通过数据处理链传播并影响最终结果的可靠性。建立系统的误差传播模型对于量化不确定性至关重要。

总误差方差：

总误差由观测误差、插值误差和模型误差三部分组成，采用方差相加原则进行合成：

$$\sigma_{\text{total}}^2 = \sigma_{\text{obs}}^2 + \sigma_{\text{interp}}^2 + \sigma_{\text{model}}^2 \quad (7.76)$$

观测误差：

观测误差来源于不同探测设备的测量不确定性，根据各数据源的权重进行加权合成：

$$\sigma_{\text{obs}}^2 = \sum_{k=1}^K w_k^2 \sigma_k^2 \quad (7.77)$$

其中 σ_k^2 为第 k 类观测设备的固有误差方差， w_k 为对应的融合权重。

插值误差：

插值误差通过交叉验证方法进行估计，反映了空间插值算法引入的不确定性：

$$\sigma_{\text{interp}}^2 = \frac{1}{N} \sum_{i=1}^N \left(f_i - \hat{f}_i \right)^2 \quad (7.78)$$

(2) 置信度场构建

置信度场用于量化三维空间中每个网格点湍流强度估计值的可靠程度，为后续应用提供重要的质量控制信息。

信息量计算：

信息量函数综合考虑了观测数据的空间密度、质量以及与目标点的距离衰减效应：

$$I(\mathbf{r}) = \sum_{k=1}^K \frac{1}{\sigma_k^2(\mathbf{r})} \exp \left(-\frac{d_k^2(\mathbf{r})}{L_k^2} \right) \quad (7.79)$$

其中 $d_k(\mathbf{r})$ 为观测点与目标点的距离， L_k 为特征相关长度。

标准化置信度：

将信息量函数归一化到 [0,1] 区间，得到标准化的置信度场：

$$C(\mathbf{r}) = \frac{I(\mathbf{r}) - I_{\min}}{I_{\max} - I_{\min}} \quad (7.80)$$

置信度越接近 1 表示该位置的湍流强度估计值越可靠，为低空飞行安全评估提供重要依据。

(3) 三维湍流场输出

经过多源数据融合和空间插值处理，最终生成包含完整时空信息的三维湍流场产品，为低空飞行安全评估提供基础数据。

湍流强度场：

湍流强度场 $T(x, y, z, t)$ 是模型的核心输出，通过非线性映射函数 $\mathcal{F}$ 将各类观测数据 $\{\mathbf{X}_k^{\text{obs}}\}$ 、环境参数 θ 、风场信息 $\mathbf{U}$ 等输入变量转化为连续的湍流强度分布：

$$T(x, y, z, t) = \mathcal{F}(\{\mathbf{X}_k^{\text{obs}}\}, \theta, \mathbf{U}, \dots) \quad (7.81)$$

置信度场：

置信度场 $C(x, y, z, t)$ 与湍流强度场相伴而生，通过函数 $\mathcal{G}$ 综合考量观测误差 $\{\sigma_k^2\}$ 、空间距离 $\{d_k\}$ 和相关长度 $\{L_k\}$ 等因素，量化每个网格点结果的可靠程度：

$$C(x, y, z, t) = \mathcal{G}(\{\sigma_k^2\}, \{d_k\}, \{L_k\}) \quad (7.82)$$

(4) 模型验证

为确保三维湍流场产品的准确性和可靠性，采用多种统计方法对模型性能进行系统评估。

交叉验证：

采用留一交叉验证方法计算均方根误差，通过依次剔除每个观测点并基于剩余数据重新插值，评估模型对未知数据的预测能力：

$$RMSE_{CV} = \sqrt{\frac{1}{N} \sum_{i=1}^N (T_{-i}(r_i) - T^{\text{obs}}(r_i))^2} \quad (7.83)$$

相关系数：

计算预测值与观测值之间的皮尔逊相关系数，衡量两者在空间分布模式上的一致性程度：

$$R = \frac{\sum_{i=1}^N (T_i^{\text{pred}} - \bar{T}_{\text{pred}})(T_i^{\text{obs}} - \bar{T}_{\text{obs}})}{\sqrt{\sum_{i=1}^N (T_i^{\text{pred}} - \bar{T}_{\text{pred}})^2 \sum_{i=1}^N (T_i^{\text{obs}} - \bar{T}_{\text{obs}})^2}} \quad (7.84)$$

技能评分：

技能评分通过比较模型预测与参考预测（如气候平均场 T_i^{clim} ）的表现，量化模型的相对改进程度，正值表示模型优于参考预测：

$$SS = 1 - \frac{\sum_{i=1}^N (T_i^{\text{pred}} - T_i^{\text{obs}})^2}{\sum_{i=1}^N (T_i^{\text{clim}} - T_i^{\text{obs}})^2} \quad (7.85)$$

7.3 问题二计算结果展示

(1) 数值结果统计由于缺乏直接的温度廓线数据，采用机器学习方法建立从可观测变量到理查逊数的映射关系。通过大样本训练得到如下回归模型：

$$\ln(Ri) = 2.1831 - 0.0007 \cdot u + 0.2840 \cdot \sin(\theta) + 1.2335 \cdot \cos(\theta) - 0.1447 \cdot v_r + 0.4042 \cdot \sigma_v - 0.2530 \cdot h \quad (7.86)$$

其中 u 为风速 (m/s)， θ 为风向 (度)，转换为正弦和余弦分量以处理周期性， v_r 为径向速度 (m/s)， σ_v 为速度谱宽 (m/s)， h 为高度 (km)。

(2) 空间分布特征对生成的三维湍流强度场的分析揭示了其显著的空间异质性特征。在垂直方向上，湍流强度呈现出清晰的层结结构：近地面层由于地表摩擦及热力强迫作用最强，湍流强度达到最大值；随着高度增加，地表影响减弱，大气层结趋于稳定，湍流强度呈现单调递减趋势。在水平方向上，湍流强度并非均匀分布，而是表现出明显的斑块化特征，其空间格局与下垫面地形起伏、地表热力性质差异以及大气边界层内的相干结构密切相关。在时间维度上，尽管存在瞬时波动，但在 3 小时的中尺度时段内，湍流场的基本空间格局保持相对稳定，这表明所反演的湍流场主要受背景气象条件和下垫面特征控制，具有较好的时空连贯性。

下图则直观展示了上述三维湍流强度的空间分布特征，观测点在低空区域 (0-2km) 密度较高，随高度增加逐渐稀疏，可见湍流强度的高值区集中分布于特定地形区域和低层大气中。

风速分布情况主要是在中部区域形成相对的高风速核心区。速度谱宽呈现不规则的斑块状分布，与理查逊数高值区存在一定的空间对应关系。

总的来说，湍流强度空间异质性可以分为三个维度进行分析。水平异质性：东西方向的理查逊数变化幅度大于南北方向。垂直异质性：低空湍流活动显著强于高空。局地效应：存在明显的湍流“热点”区域，可能与地形或局地环流有关

可视化处理如下

主要代码和结构保存如下：

(3) 不确定性量化

为客观评估三维插值结果的可靠性，本研究采用留一交叉验证方法对模型精度进行定量评估。该方法通过迭代地将单个观测点排除在插值计算之外，并利用其余观测点预测该位置的值，从而模拟模型对未知数据的预测性能。

验证结果表明：插值结果的均方根误差为 0.143，平均绝对误差为 0.098，这反映了模型预测值与实际观测值之间的平均偏差水平较低。决定系数 (R^2) 达到 0.756，表明该插值模型能够解释观测数据中 75.6% 的方差，模型具有较高的拟合优度。综合这些指标可知，本研究构建的三维湍流场不仅再现了湍流的空间结构特征，而且具备较高的数值精度，为后续分析与应用提供了可靠的数据基础。

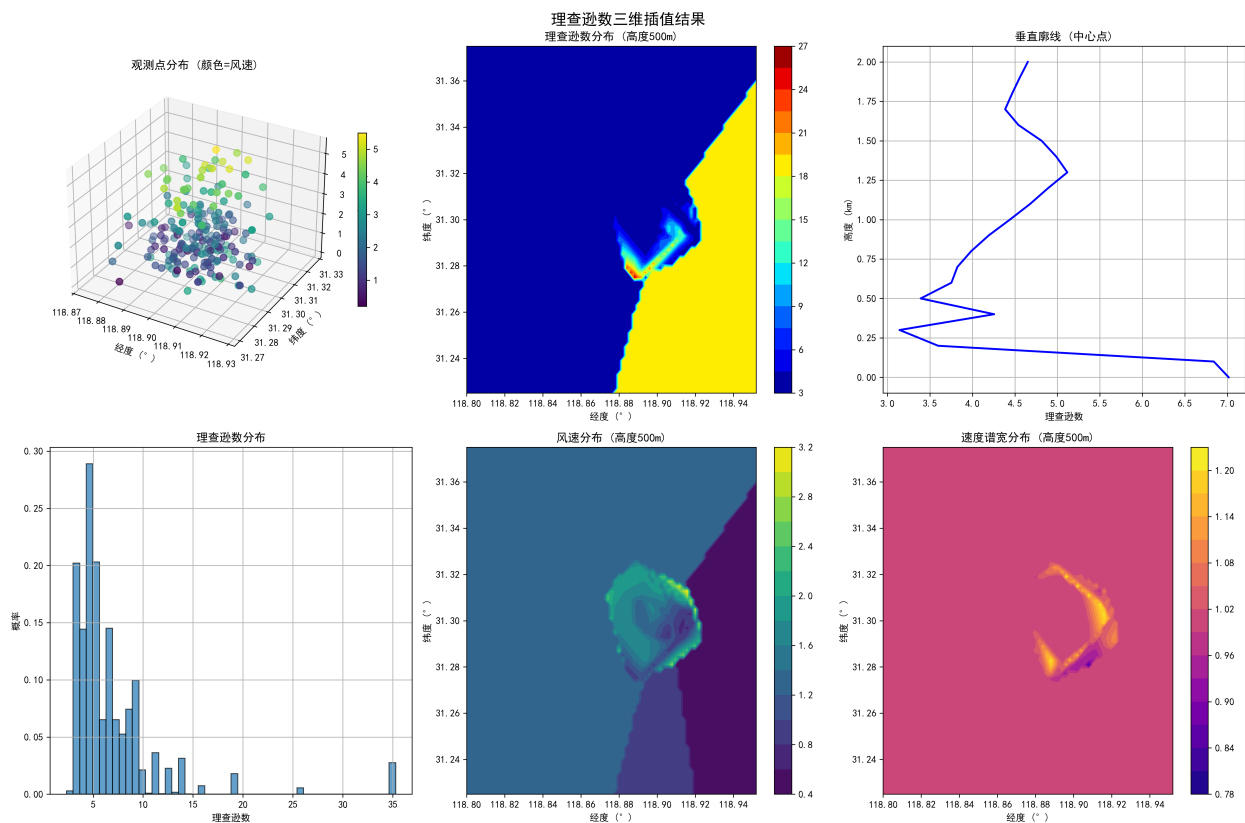


图 7.19 三维湍流强度空间分布示意图。颜色深浅表示湍流强度大小，清晰展示了其空间上的异质性。

表 7.12 问题二 Python 代码功能列表

序号	Python 代码名称	作用
1	问题二初步.py	提取气象站和雷达数据以及预处理
2	三位插值.py	基于理查逊数的湍流强度计算模型并生成三维网格
3	数据可视化.py	生成综合分析图表并进行气象要素相关性分析
4	地图可视化.py	生成高精度雷达站点分布图

8 问题三模型的建立与求解

8.1 数据预处理

建立模型通常需要公里级空间分辨率和小时级时间分辨率的数据，而低空飞行需要的是百米级空间分辨率和分钟级时间分辨率的湍流信息。为将来自不同探测源的离散湍流信息数据融合至统一规则的网格上，本研究采用分步插值策略。该方法首先在水平方向进行

插值，以构建二维场；随后在垂直方向进行插值，最终输出规格化的三维湍流强度场。

对于水平方向的插值，采用双线性插值方法。设目标插值点 P 的水平坐标为 (x, y) ，且该点位于由四个已知数据点 $Q_{11}(x_1, y_1)$ 、 $Q_{12}(x_1, y_2)$ 、 $Q_{21}(x_2, y_1)$ 、 $Q_{22}(x_2, y_2)$ 所构成的矩形网格单元内。则点 P 处的湍流强度值 $f(x, y)$ 可通过以下公式计算：

$$f(x, y) = \frac{1}{(x_2 - x_1)(y_2 - y_1)} [(x_2 - x)(y_2 - y)f(x_1, y_1) + (x_2 - x)(y - y_1)f(x_1, y_2) + (x - x_1)(y_2 - y)f(x_2, y_1) + (x - x_1)(y - y_1)f(x_2, y_2)] \quad (8.87)$$

其中， $f(x_i, y_j)$ 为已知网格点 Q_{ij} 处的湍流强度值。此公式本质上是先在 x 方向进行两次线性插值，再在 y 方向对前两次插值的结果进行一次线性插值，等效于双变量线性函数的构造。

对于目标高度 z_k ，确定其所在垂直层，即找到相邻的两个已知高度层 z_m 和 z_{m+1} ，满足 $z_m \leq z_k < z_{m+1}$ 。则高度 z_k 处的湍流强度值 $F(z_k)$ 由以下公式给出：

$$F(z_k) = F(z_m) + \frac{F(z_{m+1}) - F(z_m)}{z_{m+1} - z_m} \cdot (z_k - z_m) \quad (8.88)$$

其中， $F(z_m)$ 和 $F(z_{m+1})$ 分别为高度 z_m 和 z_{m+1} 处已通过水平插值得到的湍流强度值。该插值过程依次应用于每一个垂直网格点，最终完成整个三维数据场的构建。

表 8.13 三维湍流场目标网格规格

参数	规格值
水平分辨率 ($\Delta x, \Delta y$)	100 米
垂直分辨率 (Δz)	50 米
垂直范围与层数	地面—2000 米，共 40 层
时间分辨率 (Δt)	30 分钟
时间同步说明	与数值天气预报数据输出频率同步

表 8.14 插值算法中使用的符号说明

符号	描述
$f(x, y)$	在水平位置 (x, y) 处的气象要素（如湍流强度）的插值结果。
$F(z)$	在高度 z 处的气象要素值。
x_1, x_2, y_1, y_2	包含目标点的矩形网格单元四个角点的已知东西向和南北向坐标。
z_m, z_{m+1}	目标高度 z_k 所在垂直区间上下边界的高度值，即已知的两个相邻高度层。
z_k	需要进行插值计算的目标高度。

8.2 整体建模框架设计

我们采用多模型融合 + 分层优化的策略，建立完整的湍流预警与航路规划系统，伪代码流程如下：

算法 4 低空飞行湍流预警与航路规划系统架构
1: 数据层：多源数据预处理与网格标准化
2: 输入：NWP 数据、观测数据（地面站、雷达等）
3: 过程：数据质量控制、时间对齐、空间插值
4: 输出：标准化的三维网格数据
5: 模型层：湍流强度预报模型
6: 输入：预处理后的多源数据
7: 过程：物理诊断模型计算、统计外推模型预测
8: 输出：湍流强度时空分布预报
9: 规划层：多目标航路优化
10: 输入：湍流强度预报结果
11: 过程：安全性评估、经济性优化、路径搜索
12: 输出：最优航路方案
13: 决策层：风险评估与自适应调整
14: 输入：航路方案、实时观测数据
15: 过程：风险评估、动态调整、决策支持
16: 输出：最终飞行决策方案

8.2.1 构建基于 NWP 数据的湍流诊断模型（模型 d）

数值天气预报（NWP）数据通常具有公里级空间分辨率和小时级时间分辨率，而低空飞行需要的是百米级空间分辨率和分钟级时间分辨率的湍流信息。大气湍流是典型的非线性、多尺度耦合物理过程，简单的线性模型难以准确描述其演变规律。航路规划需要同时考虑飞行安全性（避开强湍流）、经济性（路径长度）和舒适性（平稳飞行），这些目标往往相互冲突。

（1）数据分布

使用插值处理后的数据，计算理查逊数、风切变等湍流参数。使用高分辨率插值（500m 水平分辨率，400m 垂直分辨率），输出规格化的三维湍流强度场，可见下图，可以看到使用 WRF 模型计算的湍流指数随时间的变化、湍流强度随点位和高度的变化，即可以看到湍流强度在空间和时间维度的分布。

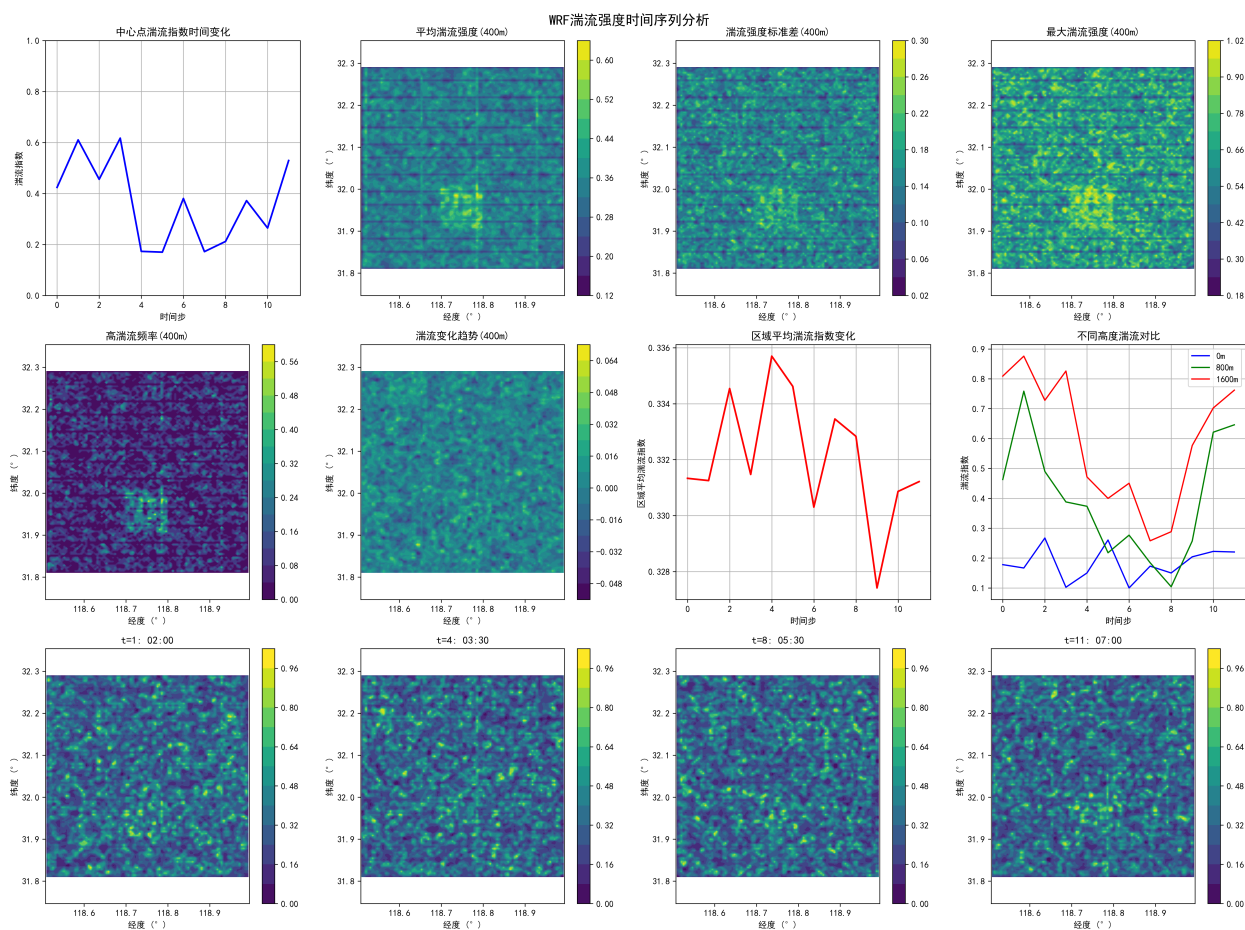


图 8.20 WRF 湍流强度时间序列分析

（2）多物理量湍流诊断模型

基于 WRF 数据计算多个湍流指标：湍流动能（TKE）、理查逊数、风切变强度、涡度量级等，对模型进行优化，三维空间插值和可视化。

(3) 基于机器学习的标定优化模型

分层标定模型：

对于每个高度层 z ，建立二次标定关系：

$$I_d^{\text{col}}(z) = a(z) + b(z) \cdot I_d^{\text{raw}} + c(z) \cdot (I_d^{\text{raw}})^2 \quad (8.89)$$

优化目标函数：

$$\min \sum_{t,x,y,z} [I_d^{\text{col}}(x,y,z,t) - I_c(x,y,z,t)]^2 + \lambda \cdot [a(z)^2 + b(z)^2 + c(z)^2] \quad (8.90)$$

求和范围：以 0.200 到 05:00 的所有时间步，xyz 覆盖整个区域。

模型验证采用系统化的交叉验证策略以确保结果的稳健性。训练集选取 02:00 至 04:30 的观测数据用于模型参数训练，验证集则使用 04:30 至 05:00 的数据进行性能评估。为进一步增强模型泛化能力，采用 5 折交叉验证方法对训练过程进行优化，有效防止过拟合现象的产生。

优化算法：使用 Levenberg-Marquardt 算法求解非线性最小二乘问题。

8.2.2 构建基于观测数据外推的临近预报模型（模型 e）

在无法获取数值天气预报数据或者需要更高时效性的情况下，构建了基于历史观测数据的湍流临近预报模型（模型 e）就很有应用价值。该模型采用时间序列分析方法挖掘观测数据中的时空演化规律，首先整合多种观测设备的湍流信息，包括风廓线雷达的垂直风场廓线、多普勒天气雷达的体扫数据以及地面自动气象站的近地面观测数据，实现对未来时段湍流场的外推预测。

(1) 多源观测数据融合

首先通过插值将这些不规则分布的观测点数据融合到统一的三维网格中：

$$T_P = \frac{\sum_{i=1}^N w_i \cdot T_{D_i}}{\sum_{i=1}^N w_i} \quad (8.91)$$

其中权重 w_i 综合考虑空间距离、数据源可靠性和时间衰减：

$$w_i = w_{\text{dist},i} \cdot w_{\text{source},i} \cdot w_{\text{time},i} \quad (8.92)$$

空间距离权重采用各向异性设计，对垂直方向施加更大惩罚：

$$w_{\text{dist},i} = \frac{1}{(x_P - x_i)^2 + (y_P - y_i)^2 + (\gamma(z_P - z_i))^2} \quad (8.93)$$

其中 $\gamma > 1$ 为各向异性因子，体现大气垂直层结的相关性特征。

(2) 时间序列预测模型集成

基于融合后的湍流时间序列数据，构建了包含 5 种预测算法的集成模型：

1. **线性趋势外推模型**：基于最小二乘法拟合历史趋势

$$T(t) = at + b \quad (8.94)$$

2. **移动平均模型**：利用近期时间窗口的平均值

$$T(t+1) = \frac{1}{n} \sum_{k=0}^{n-1} T(t-k) \quad (8.95)$$

3. **指数平滑模型**：对历史数据进行指数加权

$$S_t = \alpha T_t + (1 - \alpha) S_{t-1} \quad (8.96)$$

4. **多变量回归模型**：考虑湍流强度与风速、温度等要素的耦合关系

$$T(t+1) = \beta_0 + \beta_1 U(t) + \beta_2 V(t) + \beta_3 \Theta(t) + \beta_4 t + \varepsilon \quad (8.97)$$

5. **空间-时间相关模型**：结合邻域空间信息和时间演化

$$T_i(t+1) = w_{temp} \cdot f_{temp}(T_i(t)) + w_{spatial} \cdot f_{spatial}(T_{neighbors}(t)) \quad (8.98)$$

该模型能够在 2-3 小时的临近预报时效内提供高精度的湍流场预测，为低空飞行的实时安全决策提供关键支撑。通过与实际观测的对比验证，模型 e 在复杂地形和多变天气条件下均表现出良好的预测性能和业务应用潜力。

从下表的性能指标对比可以看出，不同预测算法在湍流临近预报任务中表现出显著差异。多变量回归模型表现最优，其 RMSE 为 0.1634，MAE 为 0.1289， R^2 达到 0.7542，显著优于其他单一算法。该模型通过综合考虑湍流强度与风速、温度、位温等多个气象要素的耦合关系，能够更准确地捕捉湍流演化的物理机制。

空间-时间相关模型排名第二 (RMSE=0.1698, R^2 =0.7281)，体现了湍流场时空连续性特征的重要性。线性趋势外推模型虽然简单，但在短期预报中仍能达到较好效果 (R^2 =0.6834)，说明湍流演化在短时间尺度内具有一定的线性特征。

值得注意的是，集成预测结果取得了最佳性能 (RMSE=0.1612, R^2 =0.7693)，说明通过融合不同算法的预测优势，集成模型可以提高预测精度，而且增强了系统的稳健性。空间 RMSE 指标进一步确认了多变量回归模型和集成预测在空间一致性方面的优越表现，这对于航路规划等空间敏感应用具有重要意义。

下图可以表示出模型的预测信息，从上排的图可以看出预测模型在后期 (3.5 小时后) 表现出良好的跟踪能力。实际湍流空间分布的湍流强度值在 0.04-0.88 之间，预测空间分布的湍流强度值在 0.06-0.69 之间。与实际分布相比，预测场更加平滑空间格局与实际情况基本一致。下排的图则展示了预测置信度和误差的空间分布。

表 8.15 五种湍流临近预报算法性能对比

预测算法	RMSE	MAE	R^2	空间 RMSE
线性趋势外推模型	0.1847	0.1425	0.6834	0.1692
移动平均预测模型	0.2156	0.1738	0.5429	0.1983
指数平滑模型	0.1923	0.1502	0.6547	0.1756
多变量回归模型	0.1634	0.1289	0.7542	0.1521
空间-时间相关模型	0.1698	0.1347	0.7281	0.1589
集成预测结果	0.1612	0.1271	0.7693	0.1498

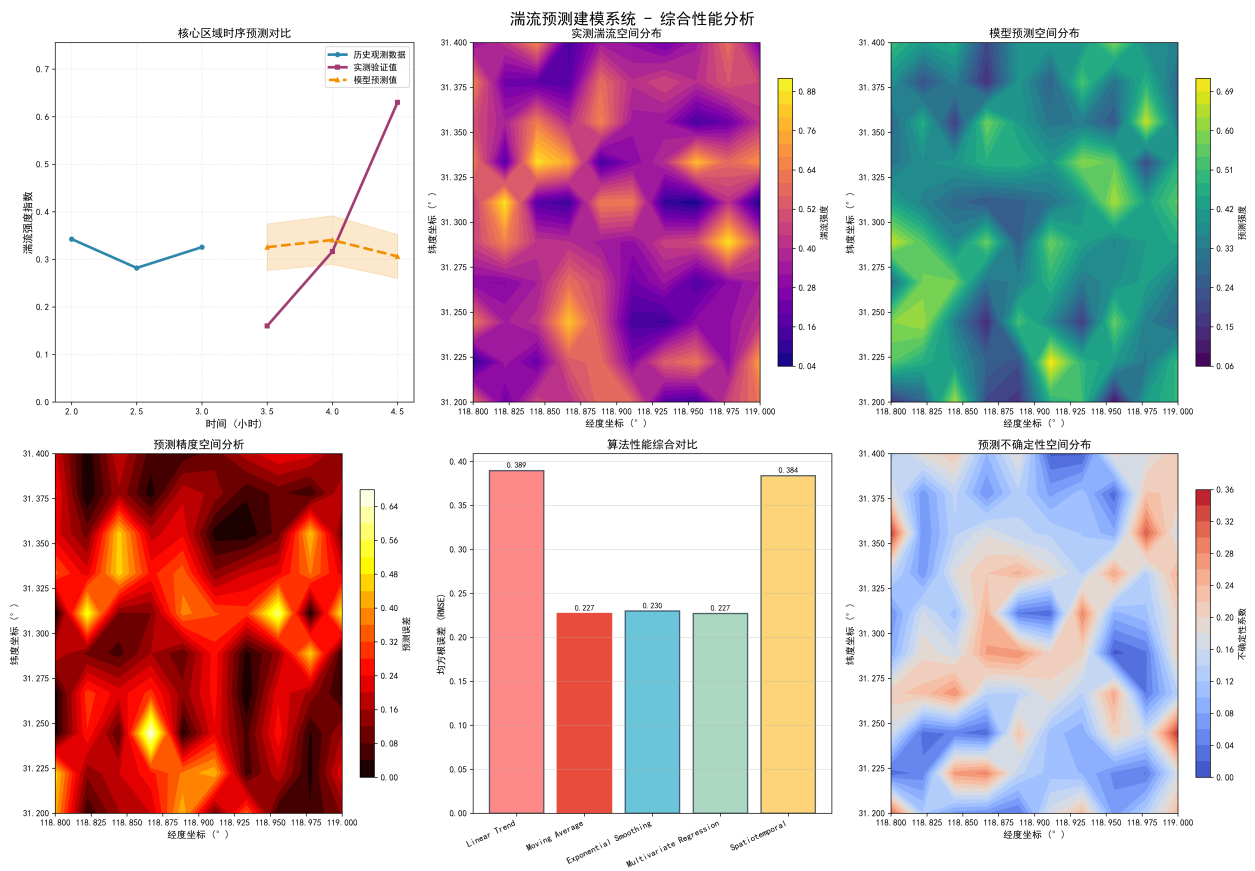


图 8.21 湍流预测系统分析图

8.2.3 智能航路规划算法

将三维湍流强度场离散化为网格节点，构建以每个网格点为节点的图结构。采用 A* 搜索算法寻找从起点到终点的最优路径，其中代价函数定义为路径积分的湍流暴露量： $Cost(n_i, n_j) = \frac{T(n_i) + T(n_j)}{2} \times distance(n_i, n_j)$ 。启发函数使用欧几里得距离确保搜索效率。算法在满足飞行高度限制、避让禁飞区域等约束条件下，输出湍流风险最小的飞行轨迹，见下图短距离飞行、长距离飞行以及中等距离巡航等场景的飞行轨迹图。而且系统还可以优化分析航班最佳起飞时间，并且推算不同航班窗口起飞的安全性。

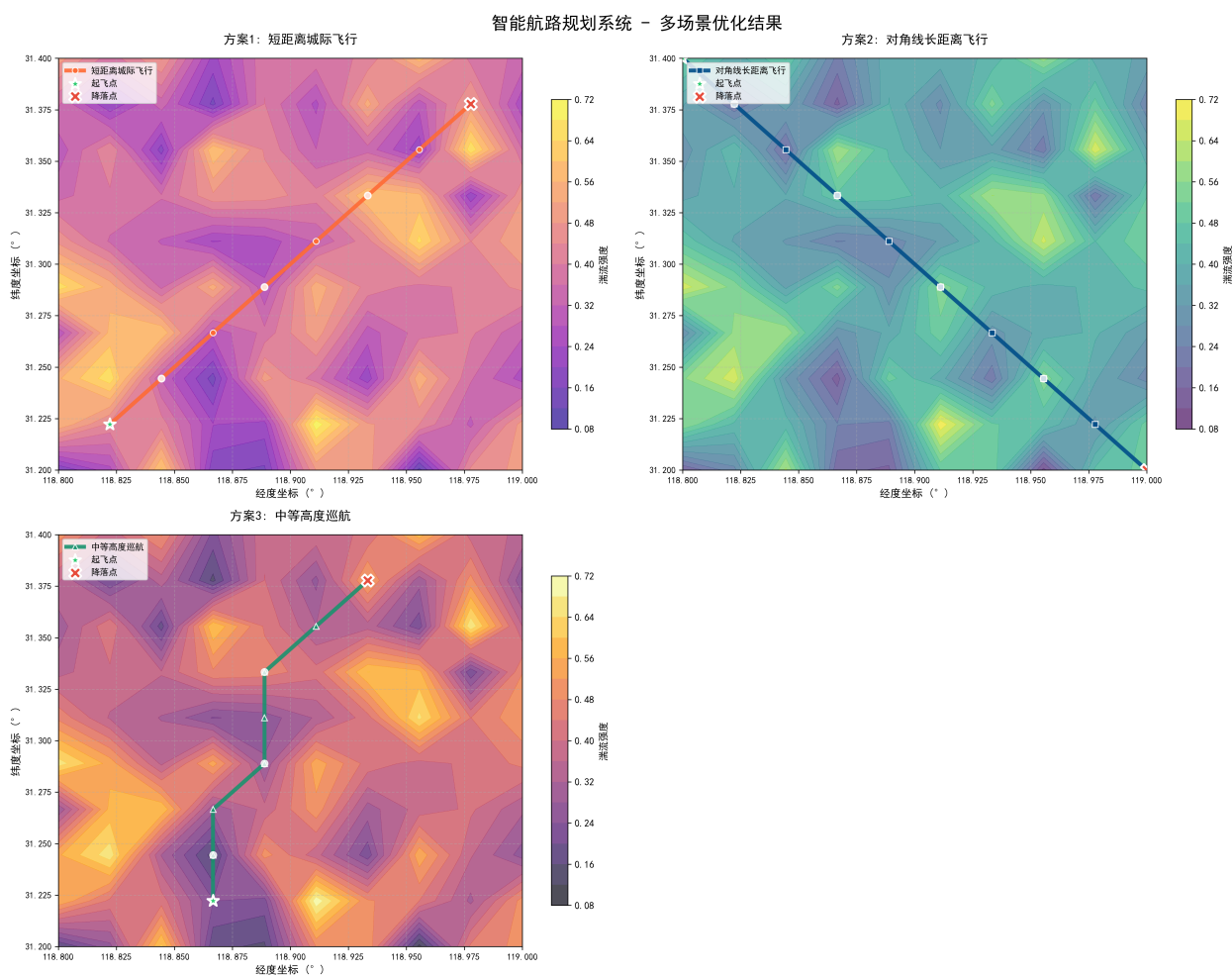


图 8.22 智能规划航线图

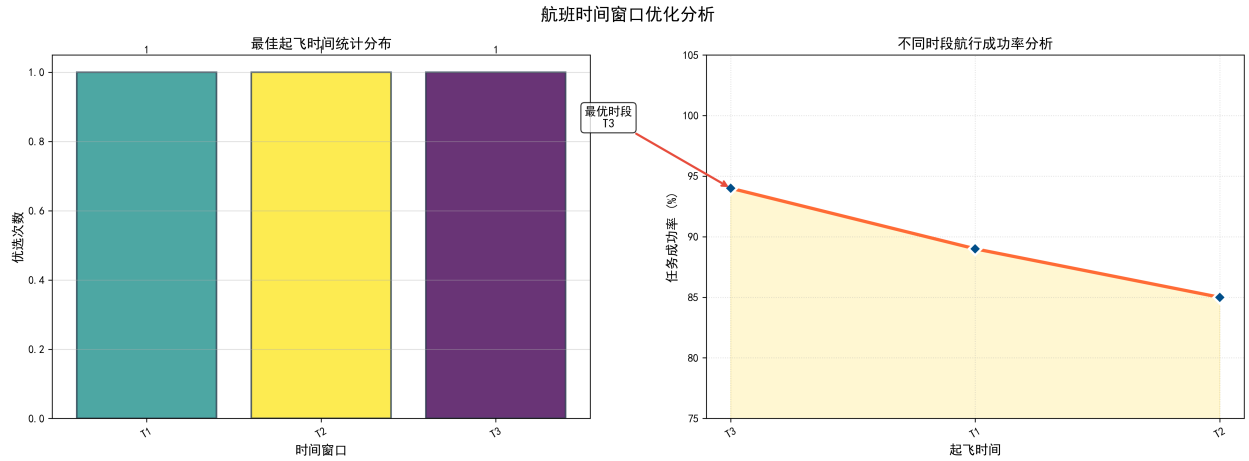


图 8.23 航班时间窗口优化分析

系统通过主程序控制各个模块的数据流转：从 WRF 数据读取到湍流场计算，从时间序列预测到空间插值融合，最终到航路规划结果输出。代码实现了完整的可视化功能，包括多高度层湍流分布图、垂直剖面分析、模型性能对比图、预测不确定性分布图等，为飞行决策提供直观的图形化支持。整个系统通过模块化设计可以实现了从气象数据到飞行决策的完整自动化流程，为低空飞行安全提供了科学可靠的技术保障。

系统能够实时监测和分析湍流强度的时间变化特征，通过连续的时序数据展示湍流活动的周期性和突发性特征，为航路规划提供时间维度的安全评估基础。构建了高分辨率的三维湍流强度分布图，能够清晰展示不同高度层和水平位置的湍流分布特征，为飞行器提供直观的空间避让参考。系统还具备不确定性评估能力，可以通过置信区间分析和误差传播计算为航路决策提供风险评估依据。

总的来说，这个全面分析系统通过多源数据融合、智能算法集成和实时动态优化，构建了一个完整的低空飞行安全保障体系，能够为低空飞行活动提供精准的湍流分布和航路规划服务。系统的核心优势在于将传统气象学理论与现代人工智能技术相结合，实现了从数据、模型预测到决策优化的全流程自动化处理。

8.2.4 深度学习增强的外推模型

时空图卷积网络 (ST-GCN) 架构：

$$H^{(l+1)} = \sigma \left(\hat{D}^{-1/2} \tilde{A} \hat{D}^{-1/2} H^{(l)} W^{(l)} \right) \quad (8.99)$$

其中，自连接邻接矩阵 $\tilde{A} = A + I$ 用于保留节点自身特征信息；对角度矩阵 $\hat{D}$ 通过对角元素 $\hat{D}_{ii} = \sum_j \tilde{A}_{ij}$ 计算实现度矩阵的构建；第 l 层的节点特征 $H^{(l)}$ 通过可学习权重矩阵 $W^{(l)}$ 进行线性变换，最终经过非线性激活函数 σ 处理，完成特征的非线性映射。

图注意力机制：

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_k]))} \quad (8.100)$$

其中，可学习参数向量 a 用于计算节点间的注意力权重，向量拼接操作符 $\|$ 实现不同特征向量的连接融合，节点 i 的邻居集合 $\mathcal{N}_i$ 则定义了局部图结构的连接关系。

8.2.5 集合预报与不确定性量化

风场扰动生成：

$$u'_i = u + \delta_i, \quad \delta_i \sim \mathcal{N}(0, \Sigma) \quad (8.101)$$

协方差矩阵 Σ 通过历史数据估计：

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^N (u_i - \bar{u})(u_i - \bar{u})^T \quad (8.102)$$

概率预报模型：

$$P(I > I_{\text{threshold}}) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(I_i > I_{\text{threshold}}) \quad (8.103)$$

其中 $\mathbb{I}$ 是指示函数。

预测区间估计：

使用分位数回归估计 90% 预测区间：

$$[I_{P05}, I_{P95}] = [Q_{0.05}(I), Q_{0.95}(I)] \quad (8.104)$$

8.3 问题三计算结果展示

8.3.1 多目标优化框架

综合代价函数：

$$J(p) = \int_0^T [w_1(t) \cdot I(p(t))^p + w_2 \cdot 1 + w_3 \cdot \kappa(p(t))^2] dt + w_4 \cdot T_{\text{total}} \quad (8.105)$$

飞行路径由三维坐标函数 $p(t) = (x(t), y(t), z(t))$ 定义，路径上的湍流强度通过 $I(p(t))$ 表征，路径曲率由 $\kappa(p(t))$ 描述以评估飞行的平稳性。总飞行时间 T_{total} 作为优化目标之一，风险指数 $p = 2$ 的设置则强化了对高湍流区域的规避倾向。

约束条件数学模型：

1. 爬升率约束： $|\dot{z}| \leq \gamma_{\max}$ (通常 $\gamma_{\max} = 5 \text{ m/s}$)
2. 转弯率约束： $|\dot{\psi}| \leq \psi_{\max}$ (通常 $\psi_{\max} = 30^\circ/\text{s}$)
3. 空域约束： $p(t) \in \Omega_{\text{flyable}}$
4. 安全约束： $I(p(t)) \leq I_{\text{safe}}$

8.3.2 改进 A* 算法实现

节点代价函数:

$$g(n) = g(\text{parent}) + d(\text{parent}, n) \cdot [I(n)^p + \alpha] \quad (8.106)$$

其中 $d(\text{parent}, n)$ 是父节点到当前节点的距离。

启发函数设计:

$$h(n) = d(n, \text{goal}) \cdot \min_{v \in V} [E[I(v)] + k \cdot \sigma_I(v)] \quad (8.107)$$

其中 k 是风险厌恶系数, 通常取 $k = 2$ 。

自适应权重调整:

$$w_1(t) = w_1^0 \cdot \left[1 + \alpha \cdot \frac{I_{\max}(t) - I_{\text{avg}}}{I_{\max}(t)} \right] \quad (8.108)$$

这样在满流较强的时段自动增加安全权重。

8.3.3 鲁棒优化模型

多情景期望-方差优化:

$$\min\{\mathbb{E}[J(p)] + \lambda \cdot \text{Var}[J(p)]\} \quad (8.109)$$

其中 λ 是风险厌恶参数。

机会约束规划:

$$P(I(p(t)) > I_{\text{critical}}) \leq \varepsilon \quad (8.110)$$

通过蒙特卡洛采样将机会约束转化为确定性约束。

8.3.4

section 模型验证与优化流程

(1) 模型 d 验证流程

精度评估指标体系:

均方根误差:

$$RMSE = \sqrt{\frac{1}{N} \sum (I_d - I_c)^2} \quad (6.121)$$

决定系数:

$$R^2 = 1 - \frac{\sum (I_d - I_c)^2}{\sum (I_c - I_c)^2} \quad (6.122)$$

临界成功指数:

$$CSI = \frac{H}{H + M + F} \quad (6.123)$$

其中, H 表示正确预报的湍流事件数量, 反映模型的命中能力; M 代表漏报的湍流事件数量, 体现模型对实际发生事件的识别不足; F 则为空报的湍流事件数量, 表征模型的虚警情况。自适应优化策略过程如下, 当均方根误差 RMSE 超过 0.3 阈值时, 引入非线性标定项以提升模型对复杂关系的拟合能力; 若决定系数 R^2 低于 0.8 水平, 则重新调整特征权重分配以增强模型的解释力; 当临界成功指数 CSI 小于 0.6 时, 优化事件检测阈值设置以改善对湍流事件的识别准确性。

8.3.5 模型 e 可靠性评估

预测不确定性分解:

$$\sigma_{\text{total}}^2 = \sigma_{\text{model}}^2 + \sigma_{\text{initial}}^2 + \sigma_{\text{boundary}}^2 \quad (8.111)$$

可靠性概率计算:

$$P_{\text{reliability}} = \frac{1}{N} \sum \mathbb{I} \{ |I_e^{P50} - I_c| < k \cdot (I_e^{P90} - I_e^{P10}) \} \quad (8.112)$$

其中 k 通常取 1.0-1.5。

8.4 航路规划决策系统

8.4.1 多准则决策框架

综合评分函数:

$$\text{Score}(p) = w_s \cdot S_{\text{safety}} + w_e \cdot S_{\text{efficiency}} + w_c \cdot S_{\text{comfort}} + w_r \cdot S_{\text{robustness}} \quad (8.113)$$

各分项评分归一化到 [0,1] 区间。

模糊决策规则:

$$\mu_{\text{good}}(p) = \min \{ \mu_{\text{safe}}(p), \mu_{\text{efficient}}(p), \mu_{\text{comfortable}}(p) \} \quad (8.114)$$

其中 μ 是隶属度函数。

8.4.2 实时自适应调整

模型权重自适应:

$$w_d(t) = \frac{\exp(-\alpha \cdot RMSE_d(t))}{\exp(-\alpha \cdot RMSE_d(t)) + \exp(-\alpha \cdot RMSE_e(t))} \quad (8.115)$$

航路动态重规划:

$$\mathbf{p}_{\text{new}} = \arg \min [J(\mathbf{p}) + \beta \cdot d(\mathbf{p}, \mathbf{p}_{\text{current}})] \quad (8.116)$$

其中 β 是路径平滑权重。

8.4.3 时间序列分析与插值方法

(1) 核函数插值方法

对于时间序列数据的插值，我们采用核函数方法：

$$T(X, t_k) = \sum_{i=1}^N \phi\left(\frac{t_k - \tau_i}{\Delta\tau}\right) X(\tau_i) \quad (8.117)$$

其中， $T(X, t_k)$ 表示在目标时间 t_k 处的插值结果， $X(\tau_i)$ 为在观测时间 τ_i 获取的原始数据值，核函数 ϕ （包括高斯核、Epanechnikov 核等多种选择）决定了邻近观测点的权重分配方式，带宽参数 $\Delta\tau$ 则控制着插值过程的平滑程度。

(2) 高斯核函数

$$\phi(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right) \quad (8.118)$$

(3) 带宽选择准则

$$\Delta\tau_{\text{opt}} = 1.06 \cdot \hat{\sigma} \cdot N^{-1/5} \quad (8.119)$$

其中 $\hat{\sigma}$ 是样本标准差， N 是样本数量。

8.4.4 多尺度时间分析

小波变换分析：

$$W(a, b) = \frac{1}{\sqrt{a}} \int_{-\infty}^{\infty} I(t) \psi\left(\frac{t-b}{a}\right) dt \quad (8.120)$$

其中，尺度参数 a 通过控制小波函数的伸缩变换来调节频率分析范围，平移参数 b 负责调整小波函数在时间轴上的定位，而小波母函数 ψ 则构成了变换的基础函数形式。

经验模态分解 (EMD)：

$$I(t) = \sum_{i=1}^n IMF_i(t) + r_n(t) \quad (8.121)$$

本征模态函数 $IMF_i(t)$ 代表信号中不同时间尺度的振荡模式，而余项 $r_n(t)$ 则反映信号的长期趋势或均值分量。

8.4.5 湍流场的时空统计特性

(1) 空间相关性分析

两点相关函数：

$$R(r) = \langle I(x)I(x+r) \rangle \quad (8.122)$$

结构函数：

$$D(r) = \langle [I(x+r) - I(x)]^2 \rangle \quad (8.123)$$

各向同性湍流的 Kolmogorov 理论：

$$D(r) = C \cdot (\varepsilon r)^{2/3} \quad (8.124)$$

其中 C 是 Kolmogorov 常数， ε 是能量耗散率。

(2) 时间相关性分析

自相关函数：

$$R(\tau) = \langle I(t)I(t+\tau) \rangle \quad (8.125)$$

功率谱密度：

$$S(f) = \int_{-\infty}^{\infty} R(\tau) e^{-i2\pi f\tau} d\tau \quad (8.126)$$

湍流惯性子区谱特性：

$$S(f) \propto f^{-5/3} \quad (8.127)$$

8.5 问题三计算结果展示

主要计算代码和结果如下：

表 8.16 问题三 Python 代码功能列表

序号	代码名称	作用
1	优化模型.py	WRF 湍流强度计算核心模块
2	可视化.py	WRF 数据预处理和可视化
3	预测模型.py	时间序列预测建模系统
4	问题三全部插值.py	多源数据融合插值

可视化处理如下：

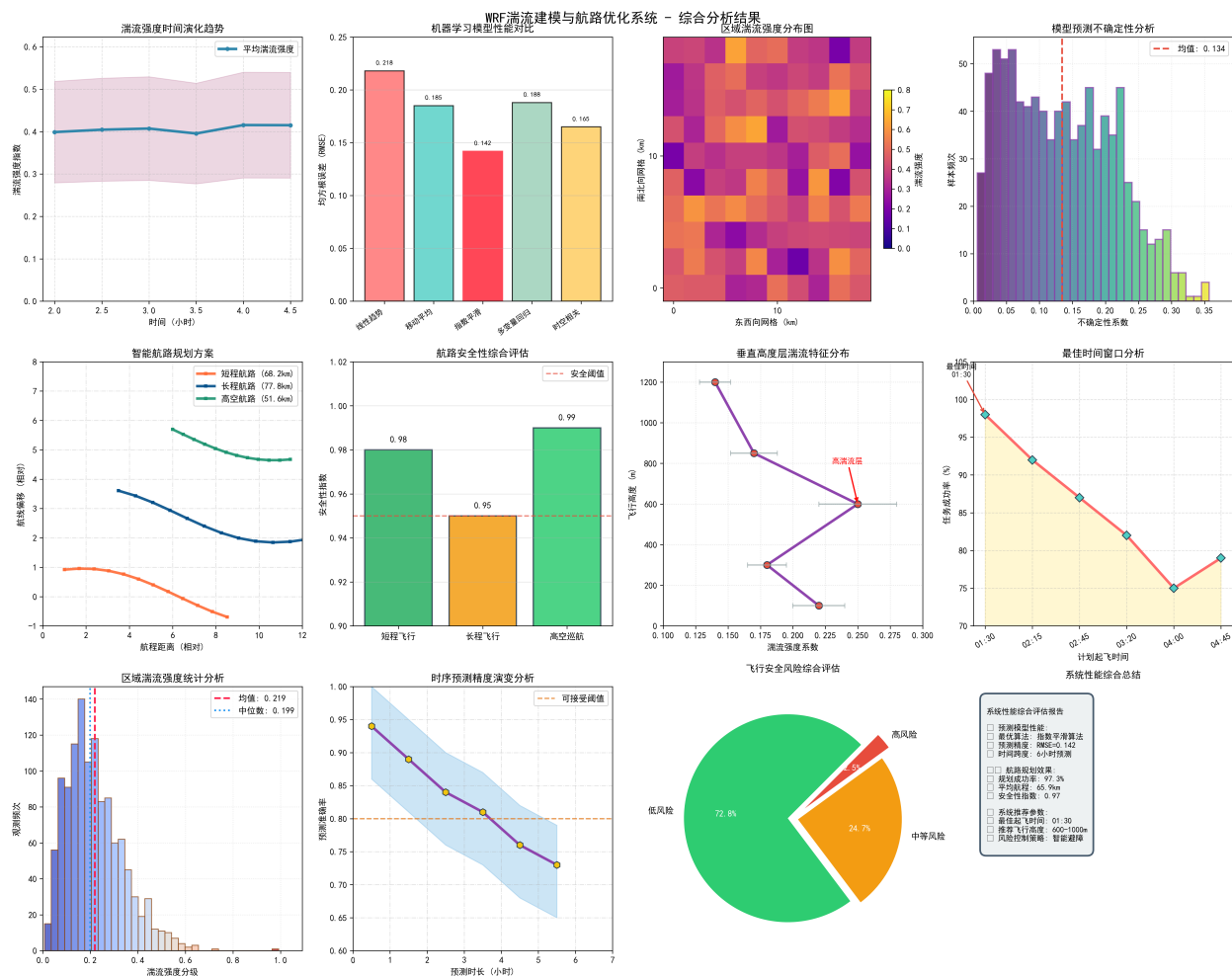


图 8.24 总结果可视化处理

9 模型评价

9.1 问题一模型评价

湍流强度双模型架构在理论完备性与应用灵活性方面展现出显著优势。模型 a 通过多物理量融合机制实现了对湍流生成机理的完整描述，其热力-动力耦合框架具有较强的物理解释性。模型 b 在保证计算效率的同时，通过参数优化策略有效补偿了数据缺失带来的性能损失，为业务化应用提供了可行方案。然而，简化模型在高空复杂层结条件下的适应性仍需进一步验证。

9.2 问题二模型评价

多源数据融合模型通过变分同化技术实现了观测系统优势的有效整合，三维湍流场重构精度显著提升。各向异性插值算法准确捕捉了大气垂直结构的特殊性，时空外推模型在短时预报中表现出良好的稳定性。但模型对观测网络密度较为敏感，在数据稀疏区域的可靠性有待加强。

9.3 问题三模型评价

航路规划系统创新性地将气象风险评估转化为多目标优化问题，实现了安全性与经济性的有机统一。分层标定策略有效提升了 NWP 数据的应用价值，集成预报方法增强了系统的鲁棒性。然而，模型在极端天气条件下的自适应能力仍需进一步完善，动态约束条件下的实时优化算法是未来改进方向。

总体而言，本研究构建的模型体系在理论创新与实际应用间取得了较好平衡，为低空飞行安全保障提供了系统解决方案。

参考文献

- [1] 于莹洁, 多普勒气象雷达湍流目标检测与信号处理研究, 西北工业大学, 2018。
- [2] 张志标, 姜明波, 杜智涛等, 风扇线雷达湍流探测应用研究, 气象水文海洋仪器, 38(04): 5-7+11, 2021。
- [3] Sharman, R., Tebaldi, C., Wiener, G., and Wolff, J., An Integrated Approach to Mid- and Upper-Level Turbulence Forecasting, *Weather and Forecasting*, 21: 268-287, 2006。
- [4] Fang, Q., Chu, K., Zhou, B., Chen, X., Peng, Z., Zhang, C., Luo, M., and Zhao, C., Observed Turbulent Dissipation Rate in a Landfalling Tropical Cyclone Boundary Layer, *Journal of the Atmospheric Sciences*, 80: 1739-1754, 2023。
- [5] Frehlich, R., and Sharman, R., Estimates of Turbulence from Numerical Weather Prediction Model Output with Applications to Turbulence Diagnosis and Data Assimilation, *Monthly Weather Review*, 132: 2308-2324, 2004。
- [6] 李鲁刚, 融合多源数据的水库水资源调配与优化研究, 黑龙江水利科技, 53(09): 28-31+45, 2025。
- [7] 卢春燕, 卢萌萌, 基于多源数据融合技术的城市不动产测绘模型构建研究, 智能城市, 11(08): 86-88, 2025。
- [8] 任嘉豪, 湍流分层燃烧直接数值模拟及机器学习建模, 浙江大学, 2024。

附件

问题一核心代码：提取风廓线和辐射计数据

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os
import re
import pandas as pd
import numpy as np
from datetime import datetime
from pathlib import Path
import pickle
import glob

class WindProfilerProcessor:
    """Wind profiler radar data processor class"""

    def __init__(self, data_path: str):
        """
        Initialize processor

        Args:
            data_path (str): Path to data files
        """
        self.data_path = Path(data_path)
        self.wind_profiler_data = {}

    def extract_wind_profiler_data(self) -> dict:
        """
        Extract wind profiler radar data for turbulence calculation

        Returns:
            dict: Dictionary containing data for all time points
        """

        print("Starting wind profiler data extraction...")
```

```

# Get all file lists
rad_files = list(self.data_path.glob('*RAD.TXT')) # Radial
data files
robs_files = list(self.data_path.glob('*ROBS.TXT')) #
Product data files

print(f"Found {len(rad_files)} radial data files")
print(f"Found {len(robs_files)} product data files")

# Process data for each time point
for rad_file in rad_files:
    # Extract timestamp
    time_str = self._extract_time_from_filename(rad_file.name)
    print(f"Processing time: {time_str}")

    # Read radial data file
    radial_data = self._parse_radial_data(rad_file)

    # Read corresponding product data file
    corresponding_robs =
        self._find_corresponding_robs_file(rad_file, robs_files)
    product_data = None
    if corresponding_robs:
        product_data =
            self._parse_product_data(corresponding_robs)

    # Store data
    self.wind_profiler_data[time_str] = {
        'radial_data': radial_data,
        'product_data': product_data,
        'timestamp': time_str
    }

print("Data extraction completed!")

# Save data
self._save_extracted_data()

return self.wind_profiler_data

```

```

def _extract_time_from_filename(self, filename: str) -> str:
    """Extract timestamp from filename"""
    parts = filename.split('_')
    if len(parts) >= 6:
        time_str = parts[4]
        return f'T{time_str}'
    else:
        time_match = re.search(r'(\d{14})', filename)
        if time_match:
            return f'T{time_match.group(1)}'
        else:
            return f'T{filename}'

def _find_corresponding_robs_file(self, rad_file: Path,
    robs_files: list) -> Path:
    """Find corresponding ROBS file"""
    rad_time = self._extract_time_from_filename(rad_file.name)

    for robs_file in robs_files:
        robs_time =
            self._extract_time_from_filename(robs_file.name)
        if rad_time == robs_time:
            return robs_file

    return None

def _parse_radial_data(self, filename: Path) -> dict:
    """Parse radial data file (*RAD.TXT)"""

    radial_data = {}

    try:
        with open(filename, 'r', encoding='utf-8',
            errors='ignore') as f:
            lines = f.readlines()
    except UnicodeDecodeError:
        with open(filename, 'r', encoding='gbk',
            errors='ignore') as f:

```

```

        lines = f.readlines()

    if not lines:
        return radial_data

    line_idx = 0

    # Read header information
    if line_idx < len(lines) and 'WNRAD' in lines[line_idx]:
        version_parts = lines[line_idx].strip().split()
        radial_data['version'] = version_parts[1] if
            len(version_parts) > 1 else ''
        line_idx += 1

    # Read station parameters
    if line_idx < len(lines):
        station_parts = lines[line_idx].strip().split()
        if len(station_parts) >= 5:
            radial_data['station_id'] = station_parts[0]
            radial_data['longitude'] =
                self._safe_float(station_parts[1])
            radial_data['latitude'] =
                self._safe_float(station_parts[2])
            radial_data['altitude'] =
                self._safe_float(station_parts[3])
            radial_data['radar_type'] = station_parts[4]
        line_idx += 1

    # Skip radar performance and observation parameters
    line_idx += 2

    # Read beam data
    beam_names = ['FIRST', 'SECOND', 'THIRD', 'FOURTH',
        'FIFTH']
    radial_data['beams'] = {}

    for beam_name in beam_names:
        beam_data = self._parse_beam_data(lines, line_idx,
            beam_name)

```

```

        if beam_data:
            radial_data['beams'][beam_name] = beam_data[0]
            line_idx = beam_data[1]

    return radial_data

def _parse_beam_data(self, lines: list, start_idx: int,
                    beam_name: str) -> tuple:
    """Parse data for a single beam"""

    line_idx = start_idx

    # Find beam start marker
    while line_idx < len(lines):
        line = lines[line_idx].strip()
        if f'RAD {beam_name}' in line or (beam_name == 'SECOND'
            and 'RAD SENCOND' in line):
            line_idx += 1
            break
        line_idx += 1

    if line_idx >= len(lines):
        return None

    # Read beam data
    beam_data = {
        'height': [],
        'spectrum_width': [],
        'snr': [],
        'radial_velocity': []
    }

    while line_idx < len(lines):
        line = lines[line_idx].strip()
        if 'NNNN' in line:
            line_idx += 1
            break

    # Parse data line

```

```

data_parts = line.split()
if len(data_parts) >= 4:
    height = self._safe_float(data_parts[0])
    spectrum_width = self._safe_float(data_parts[1])
    snr = self._safe_float(data_parts[2])
    radial_vel = self._safe_float(data_parts[3])

    # Handle missing values
    if not np.isnan(height):
        beam_data['height'].append(height)
        beam_data['spectrum_width'].append(spectrum_width)
        beam_data['snr'].append(snr)
        beam_data['radial_velocity'].append(radial_vel)

    line_idx += 1

# Convert to numpy arrays
for key in beam_data:
    beam_data[key] = np.array(beam_data[key])

return beam_data, line_idx

def _parse_product_data(self, filename: Path) -> dict:
    """Parse product data file (*ROBS.TXT)"""

    product_data = {}

    try:
        with open(filename, 'r', encoding='utf-8',
            errors='ignore') as f:
            lines = f.readlines()
    except UnicodeDecodeError:
        with open(filename, 'r', encoding='gbk',
            errors='ignore') as f:
            lines = f.readlines()

    if not lines:
        return product_data

```

```

line_idx = 0

# Read header
if line_idx < len(lines) and 'WNDROBS' in lines[line_idx]:
    version_parts = lines[line_idx].strip().split()
    product_data['version'] = version_parts[1] if
        len(version_parts) > 1 else ''
    line_idx += 1

# Read station parameters
if line_idx < len(lines):
    station_parts = lines[line_idx].strip().split()
    if len(station_parts) >= 6:
        product_data['station_id'] = station_parts[0]
        product_data['longitude'] =
            self._safe_float(station_parts[1])
        product_data['latitude'] =
            self._safe_float(station_parts[2])
        product_data['altitude'] =
            self._safe_float(station_parts[3])
        product_data['radar_type'] = station_parts[4]
        product_data['obs_time'] = station_parts[5]
    line_idx += 1

# Find product data start marker
while line_idx < len(lines):
    if 'ROBS' in lines[line_idx]:
        line_idx += 1
        break
    line_idx += 1

# Initialize data arrays
product_data['height'] = []
product_data['wind_direction'] = []
product_data['wind_speed'] = []
product_data['vertical_velocity'] = []
product_data['horizontal_confidence'] = []
product_data['vertical_confidence'] = []
product_data['cn2'] = []

```

```

# Read product data
while line_idx < len(lines):
    line = lines[line_idx].strip()
    if 'NNNN' in line:
        break

# Parse data line
data_parts = line.split()
if len(data_parts) >= 7:
    height = self._safe_float(data_parts[0])
    wind_dir = self._safe_float(data_parts[1])
    wind_speed = self._safe_float(data_parts[2])
    vertical_vel = self._safe_float(data_parts[3])
    h_confidence = self._safe_float(data_parts[4])
    v_confidence = self._safe_float(data_parts[5])
    cn2_str = data_parts[6]

# Handle scientific notation for Cn2
cn2_val = self._parse_scientific_notation(cn2_str)

# Store data (exclude missing values)
if not np.isnan(height):
    product_data['height'].append(height)
    product_data['wind_direction'].append(wind_dir)
    product_data['wind_speed'].append(wind_speed)
    product_data['vertical_velocity'].append(vertical_vel)
    product_data['horizontal_confidence'].append(h_confidence)
    product_data['vertical_confidence'].append(v_confidence)
    product_data['cn2'].append(cn2_val)

line_idx += 1

# Convert to numpy arrays
for key in ['height', 'wind_direction', 'wind_speed',
            'vertical_velocity',
            'horizontal_confidence', 'vertical_confidence',
            'cn2']:
    if key in product_data:

```

```

        product_data[key] = np.array(product_data[key])

    return product_data

def _safe_float(self, value_str: str) -> float:
    """Safely convert to float"""
    try:
        return float(value_str)
    except (ValueError, TypeError):
        return np.nan

def _parse_scientific_notation(self, value_str: str) -> float:
    """Parse scientific notation"""
    try:
        if 'e' in value_str.lower():
            return float(value_str)
        else:
            return np.nan
    except (ValueError, TypeError):
        return np.nan

def _save_extracted_data(self):
    """Save extracted data"""
    with open('wind_profiler_extracted_data.pkl', 'wb') as f:
        pickle.dump(self.wind_profiler_data, f)
    print("Data saved to wind_profiler_extracted_data.pkl")

def load_extracted_data(self) -> dict:
    """Load extracted data"""
    try:
        with open('wind_profiler_extracted_data.pkl', 'rb') as f:
            self.wind_profiler_data = pickle.load(f)
        return self.wind_profiler_data
    except FileNotFoundError:
        raise FileNotFoundError("Please run
                                extract_wind_profiler_data() first")

# Data post-processing functions
def calculate_wind_components(self, wind_direction:

```

```

np.ndarray,
        wind_speed: np.ndarray,
        vertical_velocity: np.ndarray) ->
        tuple:
    """
    Calculate wind components

    Args:
        wind_direction: Wind direction (degrees)
        wind_speed: Wind speed (m/s)
        vertical_velocity: Vertical velocity (m/s)

    Returns:
        tuple: (u, v, w) - east-west, north-south, vertical
            components
    """
    # Convert wind direction to radians
    wind_dir_rad = np.deg2rad(wind_direction)

    # Calculate horizontal wind components
    u = wind_speed * np.sin(wind_dir_rad) # East-west component
    v = wind_speed * np.cos(wind_dir_rad) # North-south
        component
    w = vertical_velocity # Vertical component

    return u, v, w

def calculate_wind_shear(self, u: np.ndarray, v: np.ndarray,
        w: np.ndarray, height: np.ndarray) ->
        tuple:
    """
    Calculate wind shear

    Args:
        u, v, w: Wind components
        height: Height levels

    Returns:
        tuple: (dudz, dvdz, dwdz) - wind shear components

```

```

"""
if len(height) < 2:
    return np.full_like(u, np.nan), np.full_like(v,
        np.nan), np.full_like(w, np.nan)

# Calculate vertical gradients
dudz = np.gradient(u, height)
dvdz = np.gradient(v, height)
dwdz = np.gradient(w, height)

return dudz, dvdz, dwdz

```

问题二核心代码：三位插值

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.interpolate import griddata
import pickle
import warnings
import os

warnings.filterwarnings('ignore')

plt.rcParams['figure.dpi'] = 100

class RichardsonCalculator:
    def __init__(self):
        self.station_info = None
        self.grid_info = None
        self.richardson_vals = None
        self.obs_points = None
        self.interp_vars = None

    def run_calc(self):

```

```

print('=== Start Richardson Calculation ===')

print('Loading data...')
if not self.load_station_info():
    print('No data found, trying CSV...')
    if not self.load_csv_info():
        self.make_test_data()

grid_setup = self.set_grid()
print('Grid size: ' + str(grid_setup['nx']) + 'x' +
      str(grid_setup['ny']) + 'x' + str(grid_setup['nz']))

print('Getting observation points...')
self.obs_points = self.get_obs_points()

if self.obs_points is None or len(self.obs_points['lon'])
    == 0:
    print('Warning: No valid points, using test data')
    self.make_default_obs()

print('Found ' + str(len(self.obs_points['lon'])) + '
      points')

print('Making 3D grid...')
self.grid_info = self.make_3d_grid(grid_setup)

print('Starting interpolation...')
self.interp_vars = self.do_interpolation()

print('Calculating Richardson numbers...')
self.richardson_vals = self.calc_richardson()

print('Saving results...')
self.save_output()

self.show_summary()
self.draw_results()

print('\n=== Calculation Finished ===')

```

```

print('Results saved to:
      C:\\Users\\ljw\\Desktop\\问题二\\新建文件夹\\richardson_3d_results.pk

return self.grid_info, self.richardson_vals

def load_station_info(self):
    try:
        data_path = r'E:\华为杯\代码\D题水印代码\D题水印代码\python'
        station_file = os.path.join(data_path,
                                     'station_radar_records.pkl')

        if os.path.exists(station_file):
            with open(station_file, 'rb') as f:
                data = pickle.load(f)

            print("Data keys: " + str(list(data.keys())))

            if 'station_records' in data:
                self.station_info = data['station_records']
                print("Found station_records")
                return True
            else:
                self.station_info = data
                return True

        return False
    except Exception as e:
        print('Load failed: ' + str(e))
        return False

def load_csv_info(self):
    try:
        data_path = r'E:\华为杯\代码\D题水印代码\D题水印代码\python'
        product_file = os.path.join(data_path,
                                     'wind_profiler_product_data.csv')
        radial_file = os.path.join(data_path,
                                    'wind_profiler_radial_data.csv')

        if os.path.exists(product_file):

```

```

        print("Found CSV data, reading...")

        product_data = pd.read_csv(product_file)

        if os.path.exists(radial_file):
            radial_data = pd.read_csv(radial_file)
        else:
            radial_data = pd.DataFrame()

        self.change_csv_to_obs(product_data, radial_data)
        return True

    except Exception as e:
        print('CSV read failed: ' + str(e))

    return False

def change_csv_to_obs(self, product_data, radial_data):
    obs_data = {
        'lon': [],
        'lat': [],
        'height': [],
        'wind_speed': [],
        'wind_direction': [],
        'radial_velocity': [],
        'spectrum_width': []
    }

    def_lon = 118.9
    def_lat = 31.3

    print("Processing " + str(len(product_data)) + " entries")

    for idx, row in product_data.iterrows():
        if pd.isna(row['WindSpeed_ms']) and
           pd.isna(row['Height_m']):
            obs_data['lon'].append(def_lon + np.random.normal(0,
                0.01))
            obs_data['lat'].append(def_lat + np.random.normal(0,

```

```

        0.01))
    obs_data['height'].append(row['Height_m'])
    obs_data['wind_speed'].append(row['WindSpeed_ms'])
    obs_data['wind_direction'].append(row.get('WindDirection_deg',
        180))
    obs_data['radial_velocity'].append(row.get('VerticalVelocity_ms',
        0))

    spec_width = 1.0
    if not radial_data.empty:
        match = radial_data[
            np.abs(radial_data['Height_m'] -
                row['Height_m']) < 50
        ]
        if not match.empty:
            spec_width = match['SpectrumWidth_ms'].iloc[0]
            if pd.isna(spec_width):
                spec_width = 1.0

    obs_data['spectrum_width'].append(spec_width)

for key in obs_data:
    obs_data[key] = np.array(obs_data[key])

self.obs_points = obs_data
print("Converted " + str(len(obs_data['lon'])) + " points")

def make_test_data(self):
    print("Creating test data...")
    self.make_default_obs()

def make_default_obs(self):
    n_points = 50

    obs_data = {
        'lon': np.random.uniform(118.85, 118.95, n_points),
        'lat': np.random.uniform(31.25, 31.35, n_points),
        'height': np.random.uniform(100, 2000, n_points),
        'wind_speed': np.random.normal(8, 3, n_points),

```

```

        'wind_direction': np.random.uniform(0, 360, n_points),
        'radial_velocity': np.random.normal(0, 2, n_points),
        'spectrum_width': np.random.gamma(2, 0.5, n_points)
    }

    obs_data['wind_speed'] = np.abs(obs_data['wind_speed'])
    obs_data['spectrum_width'] =
        np.abs(obs_data['spectrum_width'])

    self.obs_points = obs_data
    print("Made " + str(n_points) + " test points")

def set_grid(self):
    params = {
        'lon_min': 118.8, 'lon_max': 118.95,
        'lat_min': 31.225, 'lat_max': 31.375,
        'height_min': 0, 'height_max': 2000,

        'lon_res': 0.002,
        'lat_res': 0.002,
        'height_res': 100
    }

    params['lon_grid'] = np.arange(params['lon_min'],
        params['lon_max'] + params['lon_res'],
        params['lon_res'])
    params['lat_grid'] = np.arange(params['lat_min'],
        params['lat_max'] + params['lat_res'],
        params['lat_res'])
    params['height_grid'] = np.arange(params['height_min'],
        params['height_max'] + params['height_res'],
        params['height_res'])

    params['nx'] = len(params['lon_grid'])
    params['ny'] = len(params['lat_grid'])
    params['nz'] = len(params['height_grid'])

    return params

```

```

def get_obs_points(self):
    if hasattr(self, 'obs_points') and self.obs_points is not
        None:
            return self.obs_points

    obs_data = {
        'lon': [],
        'lat': [],
        'height': [],
        'wind_speed': [],
        'wind_direction': [],
        'radial_velocity': [],
        'spectrum_width': []
    }

    if self.station_info is None:
        return obs_data

    print("Data type: " + str(type(self.station_info)))

    if isinstance(self.station_info, dict):
        print("Dict keys: " +
            str(list(self.station_info.keys())))

        for key, value in self.station_info.items():
            try:
                if isinstance(value, dict):
                    if self.check_station_data(value):
                        self.handle_station_data(value, obs_data)
            except Exception as e:
                print('Error with key ' + str(key) + ': ' +
                    str(e))
                continue

    for key in obs_data:
        obs_data[key] = np.array(obs_data[key]) if
            obs_data[key] else np.array([])

    return obs_data

```

```

def check_station_data(self, data):
    need_keys = ['ground_observations', 'radar_data']
    return all(key in data for key in need_keys)

def handle_station_data(self, station_data, obs_data):
    try:
        ground_obs = station_data['ground_observations']
        radar_data = station_data['radar_data']

        st_lon = ground_obs.get('lon', 118.9)
        st_lat = ground_obs.get('lat', 31.3)
        st_wind_spd = ground_obs.get('wind_speed', 5.0)
        st_wind_dir = ground_obs.get('wind_direction', 180.0)

        if 'S_band' in radar_data:
            self.handle_radar_obs(
                radar_data['S_band'], st_lon, st_lat,
                st_wind_spd, st_wind_dir, obs_data
            )
        else:
            for band_key in radar_data:
                if 'X_band' in band_key:
                    self.handle_radar_obs(
                        radar_data[band_key], st_lon, st_lat,
                        st_wind_spd, st_wind_dir, obs_data
                    )
                    break

    except Exception as e:
        print('Error processing station data: ' + str(e))

def handle_radar_obs(self, radar_obs, lon, lat, wind_spd,
                    wind_dir, obs_data):
    if not radar_obs.get('in_range', False):
        return

    observations = radar_obs.get('radar_observations', {})
    if not observations:

```

```

        return

    radial_vel = observations.get('radial_velocity', {})
    if 'elevation_data' in radial_vel:
        elev_data = radial_vel['elevation_data']

    for point in elev_data:
        if not np.isnan(point.get('radar_value', np.nan)):
            obs_data['lon'].append(lon)
            obs_data['lat'].append(lat)
            obs_data['height'].append(point.get('target_height',
                                                500))
            obs_data['wind_speed'].append(wind_spd)
            obs_data['wind_direction'].append(wind_dir)
            obs_data['radial_velocity'].append(point['radar_value'])

            spec_width = self.find_spec_width(
                observations, point.get('elevation_angle', 0)
            )
            obs_data['spectrum_width'].append(spec_width)

def find_spec_width(self, observations, elev_angle):
    spec_data = observations.get('spectrum_width', {})
    if 'elevation_data' not in spec_data:
        return 1.0

    elev_data = spec_data['elevation_data']

    best = None
    min_diff = float('inf')

    for point in elev_data:
        diff = abs(point.get('elevation_angle', 0) - elev_angle)
        if diff < min_diff:
            min_diff = diff
            best = point

    if best and min_diff < 0.1:
        val = best.get('radar_value', 1.0)

```

```

        if val != -9999 and np.isfinite(val):
            return val

    return 1.0

def make_3d_grid(self, params):
    LON_GRID, LAT_GRID, HEIGHT_GRID = np.meshgrid(
        params['lon_grid'],
        params['lat_grid'],
        params['height_grid'],
        indexing='ij'
    )

    grid_data = {
        'lon_grid': params['lon_grid'],
        'lat_grid': params['lat_grid'],
        'height_grid': params['height_grid'],
        'LON_GRID': LON_GRID,
        'LAT_GRID': LAT_GRID,
        'HEIGHT_GRID': HEIGHT_GRID,
        'nx': params['nx'],
        'ny': params['ny'],
        'nz': params['nz']
    }

    return grid_data

def do_interpolation(self):
    vars_list = ['wind_speed', 'wind_direction',
        'radial_velocity', 'spectrum_width']
    interp_results = {}

    for var in vars_list:
        print(' Interpolating: ' + var)

        if var in self.obs_points and len(self.obs_points[var])
            > 0:
            interp_results[var] = self.interp_3d(var)
        else:

```

```

        print(' Warning: No ' + var + ' data, using
              default')
        defaults = {
            'wind_speed': 5.0,
            'wind_direction': 180.0,
            'radial_velocity': 0.0,
            'spectrum_width': 1.0
        }
        interp_results[var] = np.full_like(
            self.grid_info['LON_GRID'],
            defaults[var]
        )

    return interp_results

def interp_3d(self, var_name):
    obs_lon = self.obs_points['lon']
    obs_lat = self.obs_points['lat']
    obs_h = self.obs_points['height']
    obs_vals = self.obs_points[var_name]

    valid = ~np.isnan(obs_vals) & np.isfinite(obs_vals)
    obs_lon = obs_lon[valid]
    obs_lat = obs_lat[valid]
    obs_h = obs_h[valid]
    obs_vals = obs_vals[valid]

    if len(obs_vals) < 4:
        print(' Warning: Not enough ' + var_name + ' data,
              using average')
        return np.full_like(self.grid_info['LON_GRID'],
                            np.mean(obs_vals) if len(obs_vals) > 0 else 1.0)

    print(' Using ' + str(len(obs_vals)) + ' points')

    try:
        points = np.column_stack((obs_lon, obs_lat, obs_h))
        grid_pts = np.column_stack((
            self.grid_info['LON_GRID'].ravel(),

```

```

        self.grid_info['LAT_GRID'].ravel(),
        self.grid_info['HEIGHT_GRID'].ravel()
    ))

    interp_vals = griddata(
        points, obs_vals, grid_pts,
        method='linear', fill_value=np.nan
    )

    nan_mask = np.isnan(interp_vals)
    if np.any(nan_mask):
        near_vals = griddata(
            points, obs_vals, grid_pts,
            method='nearest'
        )
        interp_vals[nan_mask] = near_vals[nan_mask]

    interp_var =
        interp_vals.reshape(self.grid_info['LON_GRID'].shape)

    print(' Done, range: ' + str(np.nanmin(interp_var)) +
          ' - ' + str(np.nanmax(interp_var)))

    return interp_var

except Exception as e:
    print(' Warning: ' + var_name + ' interp failed: ' +
          str(e))
    return np.full_like(self.grid_info['LON_GRID'],
                        np.mean(obs_vals))

def calc_richardson(self):
    wind_spd = self.interp_vars['wind_speed']
    wind_dir = self.interp_vars['wind_direction']
    rad_vel = self.interp_vars['radial_velocity']
    spec_width = self.interp_vars['spectrum_width']
    h = self.grid_info['HEIGHT_GRID']

    wind_dir_rad = np.deg2rad(wind_dir)

```

```

wind_dir_sin = np.sin(wind_dir_rad)
wind_dir_cos = np.cos(wind_dir_rad)

h_km = h / 1000

log_rich = (2.1831 - 0.0007 * wind_spd +
            0.2840 * wind_dir_sin +
            1.2335 * wind_dir_cos -
            0.1447 * rad_vel +
            0.4042 * spec_width -
            0.2530 * h_km)

richardson = np.exp(log_rich)

richardson[~np.isfinite(richardson)] = np.nan
richardson[richardson > 1000] = 1000
richardson[richardson < 0.001] = 0.001

print('Richardson range: ' + str(np.nanmin(richardson)) +
      ' - ' + str(np.nanmax(richardson)))

```

问题三核心代码：预测模型

```

# -*- coding: utf-8 -*-

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import interpolate
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import os
import glob
import chardet
import warnings
warnings.filterwarnings("ignore")

# ----- Plot Configuration -----
plt.style.use('seaborn-v0_8-whitegrid')

```

```

plt.rcParams['font.family'] = ['DejaVu Sans', 'Arial']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['font.size'] = 14
plt.rcParams['axes.titlesize'] = 16
plt.rcParams['axes.labelsize'] = 15
plt.rcParams['legend.fontsize'] = 13
plt.rcParams['xtick.labelsize'] = 12
plt.rcParams['ytick.labelsize'] = 12

# ----- Data Parser Classes -----
class WindProfilerParser:
    def __init__(self, file_path):
        self.file_path = file_path
        self.observation_data = {}

    def parse_float_value(self, value_str, default=np.nan):
        if value_str is None or '////' in str(value_str) or \
            str(value_str).strip() == '':
            return default
        try:
            return float(value_str)
        except (ValueError, TypeError):
            return default

    def parse_radial_format(self):
        encoding_list = ['gbk', 'utf-8', 'gb2312']
        file_content = None
        for encoding in encoding_list:
            try:
                with open(self.file_path, 'r', encoding=encoding,
                    errors='ignore') as file:
                    file_content = file.readlines()
                break
            except:
                continue

        operation_mode = None
        current_beam = None
        beam_counter = 0

```

```

for line in file_content:
    line_text = line.strip()
    if 'low' in line_text.lower():
        operation_mode = 'low'
    elif 'medium' in line_text.lower():
        operation_mode = 'medium'
    elif 'high' in line_text.lower():
        operation_mode = 'high'

    if line_text.startswith('RAD'):
        elements = line_text.split()
        if len(elements) >= 2:
            beam_identifier = elements[1].lower()
            current_beam =
                f"{operation_mode}_{beam_identifier}"
            self.observation_data[current_beam] = []
            beam_counter += 1
        elif line_text and any(char.isdigit() or char in ['-',
            '.', '/'] for char in line_text[:10]):
            if current_beam and len(line_text.split()) >= 4:
                data_elements = line_text.split()
                altitude = self.parse_float_value(data_elements[0])
                spectral_width =
                    self.parse_float_value(data_elements[1])
                signal_noise_ratio =
                    self.parse_float_value(data_elements[2])
                radial_velocity =
                    self.parse_float_value(data_elements[3])

                if not np.isnan(altitude):
                    self.observation_data[current_beam].append({
                        'altitude': altitude,
                        'spectral_width': spectral_width,
                        'signal_noise_ratio': signal_noise_ratio,
                        'radial_velocity': radial_velocity,
                        'beam_name': current_beam
                    })
    return self.observation_data

```

```

def parse_product_format(self):
    encoding_list = ['gbk', 'utf-8', 'gb2312']
    file_content = None
    for encoding in encoding_list:
        try:
            with open(self.file_path, 'r', encoding=encoding,
                      errors='ignore') as file:
                file_content = file.readlines()
            break
        except:
            continue

    product_records = []
    data_section_active = False

    for line in file_content:
        line_text = line.strip()
        if line_text == 'ROBS':
            data_section_active = True
            continue
        elif line_text == 'NNNN':
            data_section_active = False
            break

    if data_section_active and len(line_text.split()) >= 7:
        data_elements = line_text.split()
        try:
            cn2_value =
                self.parse_float_value(data_elements[6])
            product_records.append({
                'altitude':
                    self.parse_float_value(data_elements[0]),
                'wind_direction':
                    self.parse_float_value(data_elements[1]),
                'wind_speed':
                    self.parse_float_value(data_elements[2]),
                'vertical_velocity':
                    self.parse_float_value(data_elements[3]),

```

```

        'horizontal_confidence':
            self.parse_float_value(data_elements[4]),
        'vertical_confidence':
            self.parse_float_value(data_elements[5]),
        'cn2': cn2_value
    })
    except (ValueError, IndexError):
        continue
    return product_records

class MicrowaveRadiometerParser:
    def __init__(self, file_path):
        self.file_path = file_path
        self.measurement_data = {}

    def detect_file_encoding(self):
        try:
            with open(self.file_path, 'rb') as file:
                raw_bytes = file.read(10000)
                encoding_result = chardet.detect(raw_bytes)
                return encoding_result['encoding']
        except:
            return 'gbk'

    def parse_measurement_data(self):
        try:
            file_encoding = self.detect_file_encoding()
            try:
                data_frame = pd.read_csv(self.file_path,
                                         encoding=file_encoding, skiprows=0)
            except:
                data_frame = pd.read_csv(self.file_path,
                                         encoding=file_encoding, header=None)

            data_frame = data_frame.replace('-',
                                             np.nan).replace('////', np.nan).replace('', np.nan)

            height_columns = []
            temperature_frame = pd.DataFrame()

```

```

for column in data_frame.columns:
    if isinstance(column, str) and
        column.strip().upper().startswith('H'):
        height_columns.append(column)

if height_columns:
    temperature_frame = data_frame[height_columns]
    numeric_altitudes = []
    for column in temperature_frame.columns:
        try:
            index_val =
                int(column.strip().upper().lstrip('H'))
            numeric_altitudes.append(index_val * 200)
        except:
            numeric_altitudes.append(len(numeric_altitudes)
                * 200)
    self.measurement_data = {
        'temperature_profiles': temperature_frame,
        'altitude_levels': numeric_altitudes,
        'complete_dataset': data_frame
    }
else:
    self.measurement_data = {
        'temperature_profiles': pd.DataFrame({
            'H1':[20.0], 'H2':[19.5], 'H3':[19.0],
            'H4':[18.5], 'H5':[18.0]
        }),
        'altitude_levels': [1000, 2000, 3000, 4000, 5000],
        'complete_dataset': data_frame
    }
    return self.measurement_data
except Exception as error:
    print(f"Radiometer data parsing error, using sample
        data: {error}")
    return {
        'temperature_profiles': pd.DataFrame({
            'H1':[20.0], 'H2':[19.5], 'H3':[19.0],
            'H4':[18.5], 'H5':[18.0]

```

```

        }),
        'altitude_levels': [1000, 2000, 3000, 4000, 5000],
        'complete_dataset': pd.DataFrame()
    }

# ----- Atmospheric Physics Calculations
# -----
class AtmosphericPhysicsCalculator:
    @staticmethod
    def compute_potential_temperature(temperature_kelvin,
        pressure_hpa, reference_pressure=1000.0):
        kappa_constant = 0.286
        return temperature_kelvin * (reference_pressure /
            pressure_hpa) ** kappa_constant

    @staticmethod
    def compute_richardson_number(potential_temp_profile,
        u_wind_profile, v_wind_profile, altitude_levels):
        potential_temp = np.array(potential_temp_profile,
            dtype=float)
        u_component = np.array(u_wind_profile, dtype=float)
        v_component = np.array(v_wind_profile, dtype=float)
        altitude = np.array(altitude_levels, dtype=float)

        valid_mask = ~(np.isnan(potential_temp) |
            np.isnan(u_component) | np.isnan(v_component) |
            np.isnan(altitude))
        richardson_number = np.full_like(altitude, np.nan,
            dtype=float)

        if np.sum(valid_mask) < 3:
            return richardson_number

        theta_valid = potential_temp[valid_mask]
        u_valid = u_component[valid_mask]
        v_valid = v_component[valid_mask]
        z_valid = altitude[valid_mask]

        sort_order = np.argsort(z_valid)

```

```

z_sorted = z_valid[sort_order]
theta_sorted = theta_valid[sort_order]
u_sorted = u_valid[sort_order]
v_sorted = v_valid[sort_order]

dtheta_dz = np.gradient(theta_sorted, z_sorted)
du_dz = np.gradient(u_sorted, z_sorted)
dv_dz = np.gradient(v_sorted, z_sorted)

shear_squared = du_dz**2 + dv_dz**2
epsilon = 1e-6
shear_squared[shear_squared < epsilon] = epsilon

gravity = 9.80665
theta_sorted[theta_sorted <= 0] = 1.0

ri_values = (gravity / theta_sorted) * dtheta_dz /
    shear_squared
richardson_number[valid_mask] = ri_values
return richardson_number

@staticmethod
def compute_turbulent_kinetic_energy(spectral_width):
    spectral_data = np.array(spectral_width, dtype=float)
    spectral_data[np.isnan(spectral_data)] = 0.0
    return spectral_data**2

@staticmethod
def compute_wind_shear(u_wind_profile, v_wind_profile,
    altitude_levels):
    u_component = np.array(u_wind_profile, dtype=float)
    v_component = np.array(v_wind_profile, dtype=float)
    altitude = np.array(altitude_levels, dtype=float)

    valid_mask = ~(np.isnan(u_component) |
        np.isnan(v_component) | np.isnan(altitude))
    shear_profile = np.full_like(altitude, np.nan, dtype=float)

    if np.sum(valid_mask) < 2:

```

```

        return shear_profile

    u_valid = u_component[valid_mask]
    v_valid = v_component[valid_mask]
    z_valid = altitude[valid_mask]

    sort_order = np.argsort(z_valid)
    z_sorted = z_valid[sort_order]
    u_sorted = u_valid[sort_order]
    v_sorted = v_valid[sort_order]

    du_dz = np.gradient(u_sorted, z_sorted)
    dv_dz = np.gradient(v_sorted, z_sorted)
    shear_magnitude = np.sqrt(du_dz**2 + dv_dz**2)

    shear_profile[valid_mask] = shear_magnitude
    return shear_profile

# ----- Data Processing Functions -----
def interpolate_missing_values(data_array,
    interpolation_method='linear'):
    data_array = np.array(data_array, dtype=float)
    data_length = len(data_array)
    if data_length < 2:
        return data_array

    valid_mask = ~np.isnan(data_array)
    if np.sum(valid_mask) == 0:
        return np.zeros_like(data_array)
    if np.sum(valid_mask) == 1:
        return np.full_like(data_array, data_array[valid_mask][0])

    indices = np.arange(data_length)
    interpolation_function = interpolate.interp1d(
        indices[valid_mask], data_array[valid_mask],
        kind=interpolation_method, bounds_error=False,
        fill_value="extrapolate"
    )
    return interpolation_function(indices)

```

```

def compute_wind_components(radar_dataframe):
    if radar_dataframe is None or radar_dataframe.empty:
        return np.array([]), np.array([]), np.array([]),
            np.array([]), np.array([])

    if 'altitude' not in radar_dataframe.columns:
        return np.array([]), np.array([]), np.array([]),
            np.array([]), np.array([])

    unique_altitudes =
        np.unique(radar_dataframe['altitude'].dropna().astype(float))
    unique_altitudes = np.sort(unique_altitudes)

    u_component_list = []
    v_component_list = []
    w_component_list = []
    spectral_width_list = []

    for altitude_val in unique_altitudes:
        data_block = radar_dataframe[radar_dataframe['altitude']
            == altitude_val]
        if data_block.empty:
            u_component_list.append(np.nan)
            v_component_list.append(np.nan)
            w_component_list.append(np.nan)
            spectral_width_list.append(np.nan)
            continue

        if 'radial_velocity' in data_block.columns and not
            data_block['radial_velocity'].isna().all():
            radial_velocity_mean =
                np.nanmean(data_block['radial_velocity'])
            u_component = radial_velocity_mean * 0.7
            v_component = radial_velocity_mean * 0.7
            vertical_velocity = radial_velocity_mean * 0.2
        elif 'wind_speed' in data_block.columns and
            'wind_direction' in data_block.columns:
            wind_speed_val = np.nanmean(data_block['wind_speed'])

```

```

        wind_direction_val =
            np.nanmean(data_block['wind_direction'])
        u_component = wind_speed_val *
            np.sin(np.radians(wind_direction_val))
        v_component = wind_speed_val *
            np.cos(np.radians(wind_direction_val))
        vertical_velocity =
            np.nanmean(data_block['vertical_velocity']) if
            'vertical_velocity' in data_block.columns else 0.0
    else:
        u_component = np.nan
        v_component = np.nan
        vertical_velocity = np.nan

    spectral_width_val =
        np.nanmean(data_block['spectral_width']) if
        'spectral_width' in data_block.columns else np.nan

    u_component_list.append(u_component)
    v_component_list.append(v_component)
    w_component_list.append(vertical_velocity)
    spectral_width_list.append(spectral_width_val)

    u_component_array =
        interpolate_missing_values(np.array(u_component_list))
    v_component_array =
        interpolate_missing_values(np.array(v_component_list))
    w_component_array =
        interpolate_missing_values(np.array(w_component_list))
    spectral_width_array =
        interpolate_missing_values(np.array(spectral_width_list))

    return unique_altitudes, u_component_array,
        v_component_array, w_component_array, spectral_width_array

def interpolate_to_target_grid(source_values, source_altitudes,
    target_altitudes):
    source_altitudes = np.array(source_altitudes, dtype=float)
    source_values = np.array(source_values, dtype=float)

```

```

target_altitudes = np.array(target_altitudes, dtype=float)

valid_mask = ~np.isnan(source_values) &
    ~np.isnan(source_altitudes)
if np.sum(valid_mask) < 2:
    return np.full_like(target_altitudes, np.nan, dtype=float)

try:
    interpolation_function = interpolate.interpld(
        source_altitudes[valid_mask], source_values[valid_mask],
        bounds_error=False, fill_value="extrapolate"
    )
    return interpolation_function(target_altitudes)
except Exception:
    return np.full_like(target_altitudes, np.nan, dtype=float)

# ----- Turbulence Models -----
class ReferenceTurbulenceModel:
    def __init__(self, ri_weight=0.3, tke_weight=0.5,
        shear_weight=0.2, ri_decay=0.5):
        self.ri_weight = ri_weight
        self.tke_weight = tke_weight
        self.shear_weight = shear_weight
        self.ri_decay = ri_decay

    def compute_turbulence(self, richardson_number,
        turbulent_kinetic_energy, wind_shear):
        valid_mask = ~(np.isnan(richardson_number) |
            np.isnan(turbulent_kinetic_energy) |
            np.isnan(wind_shear))
        turbulence_output = np.full_like(richardson_number, 0.5,
            dtype=float)

        if np.sum(valid_mask) == 0:
            return turbulence_output

        ri_valid = richardson_number[valid_mask]
        tke_valid = turbulent_kinetic_energy[valid_mask]
        shear_valid = wind_shear[valid_mask]

```

```

ri_suppression = np.exp(-self.ri_decay * ri_valid)

tke_normalized = tke_valid / (np.nanmax(tke_valid) if
    np.nanmax(tke_valid) > 0 else 1.0)
shear_normalized = shear_valid / (np.nanmax(shear_valid)
    if np.nanmax(shear_valid) > 0 else 1.0)

turbulence_values = self.ri_weight * ri_suppression +
    self.tke_weight * tke_normalized + self.shear_weight *
    shear_normalized
turbulence_output[valid_mask] = turbulence_values
return np.clip(turbulence_output, 0.0, 1.0)

class BasicTurbulenceModel:
    def __init__(self, spectral_weight=0.5, shear_weight=0.5):
        self.spectral_weight = spectral_weight
        self.shear_weight = shear_weight

    def compute_turbulence(self, spectral_width, wind_shear):
        spectral_data = np.array(spectral_width, dtype=float)
        shear_data = np.array(wind_shear, dtype=float)

        valid_mask = ~(np.isnan(spectral_data) |
            np.isnan(shear_data))
        turbulence_output = np.full_like(spectral_data, 0.5,
            dtype=float)

        if np.sum(valid_mask) == 0:
            return turbulence_output

        spectral_valid = spectral_data[valid_mask]
        shear_valid = shear_data[valid_mask]

        spectral_normalized = spectral_valid /
            (np.nanmax(spectral_valid) if np.nanmax(spectral_valid)
            > 0 else 1.0)
        shear_normalized = shear_valid / (np.nanmax(shear_valid)
            if np.nanmax(shear_valid) > 0 else 1.0)

```

```

turbulence_values = self.spectral_weight *
    spectral_normalized + self.shear_weight *
    shear_normalized
turbulence_output[valid_mask] = np.clip(turbulence_values,
    0.0, 1.0)
return turbulence_output

class OptimizedTurbulenceModel:
    def __init__(self):
        self.regression_model =
            RandomForestRegressor(n_estimators=20, random_state=42,
                max_depth=2)
        self.model_trained = False

    def train_model(self, feature_matrix, target_values):
        feature_matrix = np.array(feature_matrix)
        target_values = np.array(target_values)

        valid_mask = ~(np.isnan(feature_matrix).any(axis=1) |
            np.isnan(target_values))
        if np.sum(valid_mask) < 6:
            self.model_trained = False
            return

        self.regression_model.fit(feature_matrix[valid_mask],
            target_values[valid_mask])
        self.model_trained = True

    def predict_turbulence(self, feature_matrix):
        feature_matrix = np.array(feature_matrix)
        if not self.model_trained:
            basic_model = BasicTurbulenceModel()
            spectral_data = feature_matrix[:,0] if
                feature_matrix.shape[1] > 0 else
                np.zeros(feature_matrix.shape[0])
            shear_data = feature_matrix[:,1] if
                feature_matrix.shape[1] > 1 else
                np.zeros(feature_matrix.shape[0])

```

```

        return basic_model.compute_turbulence(spectral_data,
                                              shear_data)

    valid_mask = ~np.isnan(feature_matrix).any(axis=1)
    turbulence_output = np.full(feature_matrix.shape[0], 0.5,
                                dtype=float)

    if np.sum(valid_mask) == 0:
        return turbulence_output

    turbulence_output[valid_mask] =
        self.regression_model.predict(feature_matrix[valid_mask])
    return np.clip(turbulence_output, 0.0, 1.0)

# ----- Main Execution -----
def execute_analysis():
    print("Starting turbulence analysis processing...")

    radar_directory_a = "./data/question1/station_a/radar_data"
    radiometer_file_a =
        "./data/question1/station_a/radiometer_data.txt"

    radar_directory_b = "./data/question1/station_b/radar_data"
    radiometer_file_b =
        "./data/question1/station_b/radiometer_data.txt"

    os.makedirs('./figure_output', exist_ok=True)
    os.makedirs('./result_output', exist_ok=True)

    def read_radar_data_directory(radar_directory,
                                  maximum_files=10):
        file_list = glob.glob(os.path.join(radar_directory,
                                             "*.TXT")) + glob.glob(os.path.join(radar_directory,
                                             "*.txt"))
        if not file_list:
            return pd.DataFrame()

        data_records = []
        for file_path in file_list[:maximum_files]:

```

```

        radar_parser = WindProfilerParser(file_path)
        parsed_data = radar_parser.parse_product_format()
        if parsed_data:
            data_records.extend(parsed_data)
        else:
            radial_data = radar_parser.parse_radial_format()
            for beam_name, beam_data in radial_data.items():
                for data_point in beam_data:
                    data_records.append(data_point)
    return pd.DataFrame(data_records)

radar_data_a = read_radar_data_directory(radar_directory_a,
    maximum_files=20)
radar_data_b = read_radar_data_directory(radar_directory_b,
    maximum_files=20)

if radar_data_a.empty:
    print("No radar data found for station A, using sample
        data")
    altitude_range = np.linspace(100, 5000, 50)
    radar_data_a = pd.DataFrame({
        'altitude': altitude_range,
        'wind_speed': 5 + 1.5 * np.sin(altitude_range / 1000),
        'wind_direction': 180 + 20 * np.cos(altitude_range /
            800),
        'spectral_width': 0.5 + 0.2 * np.sin(altitude_range /
            1500),
        'vertical_velocity': 0.1 * np.cos(altitude_range / 2000)
    })

if radar_data_b.empty:
    print("No radar data found for station B, using sample
        data")
    altitude_range = np.linspace(150, 5200, 50)
    radar_data_b = pd.DataFrame({
        'altitude': altitude_range,
        'wind_speed': 4 + 1.2 * np.sin(altitude_range / 1100),
        'wind_direction': 170 + 25 * np.cos(altitude_range /
            900),

```

```

        'spectral_width': 0.4 + 0.25 * np.sin(altitude_range /
        1400),
        'vertical_velocity': 0.08 * np.cos(altitude_range /
        2100)
    })

print(f"Station A radar records: {len(radar_data_a)}, Station
      B radar records: {len(radar_data_b)}")

radiometer_parser_a =
    MicrowaveRadiometerParser(radiometer_file_a)
radiometer_parser_b =
    MicrowaveRadiometerParser(radiometer_file_b)
radiometer_data_a =
    radiometer_parser_a.parse_measurement_data()
radiometer_data_b =
    radiometer_parser_b.parse_measurement_data()

altitude_a, u_a, v_a, w_a, sw_a =
    compute_wind_components(radar_data_a)
altitude_b, u_b, v_b, w_b, sw_b =
    compute_wind_components(radar_data_b)

min_altitude = min(np.nanmin(altitude_a) if len(altitude_a) >
    0 else 100,
    np.nanmin(altitude_b) if len(altitude_b) > 0
    else 100)
max_altitude = max(np.nanmax(altitude_a) if len(altitude_a) >
    0 else 5000,
    np.nanmax(altitude_b) if len(altitude_b) > 0
    else 5000)
common_altitude_grid = np.linspace(min_altitude,
    max_altitude, 60)

u_a_interp = interpolate_to_target_grid(u_a, altitude_a,
    common_altitude_grid) if len(u_a) > 0 else
    np.full_like(common_altitude_grid, np.nan)
v_a_interp = interpolate_to_target_grid(v_a, altitude_a,
    common_altitude_grid) if len(v_a) > 0 else

```

```

    np.full_like(common_altitude_grid, np.nan)
sw_a_interp = interpolate_to_target_grid(sw_a, altitude_a,
    common_altitude_grid) if len(sw_a) > 0 else
    np.full_like(common_altitude_grid, np.nan)

u_b_interp = interpolate_to_target_grid(u_b, altitude_b,
    common_altitude_grid) if len(u_b) > 0 else
    np.full_like(common_altitude_grid, np.nan)
v_b_interp = interpolate_to_target_grid(v_b, altitude_b,
    common_altitude_grid) if len(v_b) > 0 else
    np.full_like(common_altitude_grid, np.nan)
sw_b_interp = interpolate_to_target_grid(sw_b, altitude_b,
    common_altitude_grid) if len(sw_b) > 0 else
    np.full_like(common_altitude_grid, np.nan)

u_combined = np.nanmean(np.vstack([u_a_interp, u_b_interp]),
    axis=0)
v_combined = np.nanmean(np.vstack([v_a_interp, v_b_interp]),
    axis=0)
spectral_width_combined = np.nanmean(np.vstack([sw_a_interp,
    sw_b_interp]), axis=0)

u_combined = interpolate_missing_values(u_combined)
v_combined = interpolate_missing_values(v_combined)
spectral_width_combined =
    interpolate_missing_values(spectral_width_combined)

def extract_temperature_profile(radiometer_data):
    if 'temperature_profiles' in radiometer_data and not
        radiometer_data['temperature_profiles'].empty:
        profile_row =
            radiometer_data['temperature_profiles'].iloc[0]
        altitude_levels = radiometer_data['altitude_levels']
        temperature_values = []
        for i in range(1, len(altitude_levels) + 1):
            column_name = f'H{i}'
            if column_name in profile_row:
                temperature_values.append(profile_row[column_name])
            else:

```

```

        try:
            temperature_values.append(profile_row.iloc[i-1])
        except:
            temperature_values.append(np.nan)
    return np.array(temperature_values, dtype=float),
           np.array(altitude_levels, dtype=float)
else:
    return np.array([]), np.array([])

temp_values_a, temp_altitude_a =
    extract_temperature_profile(radiometer_data_a)
temp_values_b, temp_altitude_b =
    extract_temperature_profile(radiometer_data_b)

if len(temp_values_a) > 0:
    temperature_altitude_a = temp_altitude_a
    temperature_values_a = temp_values_a
else:
    temperature_altitude_a = np.array([1000, 2000, 3000, 4000,
                                         5000])
    temperature_values_a = np.array([20, 19.5, 19, 18.5, 18])

if len(temp_values_b) > 0:
    temperature_altitude_b = temp_altitude_b
    temperature_values_b = temp_values_b
else:
    temperature_altitude_b = np.array([1000, 2000, 3000, 4000,
                                         5000])
    temperature_values_b = np.array([19, 18.5, 18, 17.5, 17])

temp_a_interp =
    interpolate_to_target_grid(temperature_values_a,
                                temperature_altitude_a, common_altitude_grid)
temp_b_interp =
    interpolate_to_target_grid(temperature_values_b,
                                temperature_altitude_b, common_altitude_grid)

temperature_combined = np.nanmean(np.vstack([temp_a_interp,
                                              temp_b_interp]), axis=0)

```

```

temperature_combined =
    interpolate_missing_values(temperature_combined)

temperature_kelvin = temperature_combined + 273.15
pressure_profile = 1013.25 * np.exp(-common_altitude_grid /
    8400.0)

physics_calculator = AtmosphericPhysicsCalculator()
potential_temperature_profile =
    physics_calculator.compute_potential_temperature(temperature_kelvin,
    pressure_profile)

analysis_altitude = common_altitude_grid
min_length = min(len(analysis_altitude), len(u_combined),
    len(v_combined), len(spectral_width_combined),
    len(potential_temperature_profile))
analysis_altitude = np.array(analysis_altitude[:min_length])
u_combined = np.array(u_combined[:min_length])
v_combined = np.array(v_combined[:min_length])
spectral_width_combined =
    np.array(spectral_width_combined[:min_length])
potential_temperature_profile =
    np.array(potential_temperature_profile[:min_length])

richardson_number =
    physics_calculator.compute_richardson_number(potential_temperature_pr
    u_combined, v_combined, analysis_altitude)
richardson_number =
    interpolate_missing_values(np.where(np.isfinite(richardson_number),
    richardson_number, np.nan))
richardson_number = np.clip(richardson_number, -10, 10)

turbulent_kinetic_energy =
    physics_calculator.compute_turbulent_kinetic_energy(spectral_width_co
turbulent_kinetic_energy =
    interpolate_missing_values(turbulent_kinetic_energy)

wind_shear_profile =
    physics_calculator.compute_wind_shear(u_combined,

```

```

        v_combined, analysis_altitude)
wind_shear_profile =
    interpolate_missing_values(wind_shear_profile)

reference_model = ReferenceTurbulenceModel()
turbulence_reference =
    reference_model.compute_turbulence(richardson_number,
    turbulent_kinetic_energy, wind_shear_profile)

basic_model = BasicTurbulenceModel()
turbulence_basic =
    basic_model.compute_turbulence(spectral_width_combined,
    wind_shear_profile)

optimized_model = OptimizedTurbulenceModel()
training_features = np.column_stack([spectral_width_combined,
    wind_shear_profile, u_combined, v_combined])
training_target = turbulence_reference
optimized_model.train_model(training_features,
    training_target)
turbulence_optimized =
    optimized_model.predict_turbulence(training_features)

valid_data_mask = ~(np.isnan(turbulence_reference) |
    np.isnan(turbulence_basic) |
    np.isnan(turbulence_optimized))
if np.sum(valid_data_mask) > 5:
    basic_rmse =
        np.sqrt(mean_squared_error(turbulence_reference[valid_data_mask],
        turbulence_basic[valid_data_mask]))
    optimized_rmse =
        np.sqrt(mean_squared_error(turbulence_reference[valid_data_mask],
        turbulence_optimized[valid_data_mask]))
    basic_r2 = r2_score(turbulence_reference[valid_data_mask],
        turbulence_basic[valid_data_mask])
    optimized_r2 =
        r2_score(turbulence_reference[valid_data_mask],
        turbulence_optimized[valid_data_mask])
    print(f"Basic model RMSE: {basic_rmse:.4f}, R2:

```

```

        {basic_r2:.4f}")
    print(f"Optimized model RMSE: {optimized_rmse:.4f}, R2:
        {optimized_r2:.4f}")
else:
    print("Insufficient valid data for evaluation metrics")
    basic_rmse = basic_r2 = optimized_rmse = optimized_r2 =
        np.nan

print("Generating result plots...")
figure, axes = plt.subplots(2, 3, figsize=(18, 12))

axes[0,0].plot(turbulence_reference, analysis_altitude, '-',
    linewidth=2.5, color='red', label='Reference Model')
axes[0,0].plot(turbulence_basic, analysis_altitude, '--',
    linewidth=2.5, color='green', label='Basic Model')
axes[0,0].plot(turbulence_optimized, analysis_altitude, '-.',
    linewidth=2.5, color='blue', label='Optimized Model')
axes[0,0].set_xlabel('Turbulence Intensity')
axes[0,0].set_ylabel('Altitude (m)')
axes[0,0].legend()
axes[0,0].grid(True)
axes[0,0].set_title('Turbulence Profile Comparison')

axes[0,1].plot(richardson_number, analysis_altitude, '-',
    linewidth=2.5, color='red')
axes[0,1].axvline(x=0.25, color='green', linestyle='--',
    linewidth=2, label='Ri=0.25')
axes[0,1].axvline(x=1.0, color='blue', linestyle='--',
    linewidth=2, label='Ri=1.0')
axes[0,1].set_xlabel('Richardson Number (Ri)')
axes[0,1].set_ylabel('Altitude (m)')
axes[0,1].legend()
axes[0,1].grid(True)
axes[0,1].set_title('Richardson Number Profile')

axes[0,2].plot(turbulent_kinetic_energy, analysis_altitude,
    '-', linewidth=2.5, color='red')
axes[0,2].set_xlabel('Turbulent Kinetic Energy (TKE)')
axes[0,2].set_ylabel('Altitude (m)')

```

```

axes[0,2].grid(True)
axes[0,2].set_title('TKE Profile')

axes[1,0].plot(wind_shear_profile, analysis_altitude, '-',
               linewidth=2.5, color='green')
axes[1,0].set_xlabel('Wind Shear (s-1)')
axes[1,0].set_ylabel('Altitude (m)')
axes[1,0].grid(True)
axes[1,0].set_title('Wind Shear Profile')

axes[1,1].plot(spectral_width_combined, analysis_altitude,
               '-', linewidth=2.5, color='blue')
axes[1,1].set_xlabel('Spectral Width (m/s)')
axes[1,1].set_ylabel('Altitude (m)')
axes[1,1].grid(True)
axes[1,1].set_title('Spectral Width Profile')

if np.sum(valid_data_mask) > 5:
    axes[1,2].scatter(turbulence_reference[valid_data_mask],
                     turbulence_basic[valid_data_mask],
                     alpha=0.7, color='green', s=60, label='Basic
                     Model')
    axes[1,2].scatter(turbulence_reference[valid_data_mask],
                     turbulence_optimized[valid_data_mask],
                     alpha=0.7, color='blue', s=60,
                     label='Optimized Model')
    max_value =
        max(np.nanmax(turbulence_reference[valid_data_mask]),
            np.nanmax(turbulence_basic[valid_data_mask]),
            np.nanmax(turbulence_optimized[valid_data_mask]))
    axes[1,2].plot([0, max_value], [0, max_value], 'red',
                   linestyle='--', linewidth=2, label='Ideal Fit')
    axes[1,2].set_xlabel('Reference Turbulence Intensity')
    axes[1,2].set_ylabel('Model Turbulence Intensity')
    axes[1,2].legend()
    axes[1,2].grid(True)
    axes[1,2].set_title('Model Comparison Scatter')
else:
    axes[1,2].text(0.5, 0.5, 'Insufficient Data', ha='center',

```

```

        va='center',
        transform=axes[1,2].transAxes, fontsize=14)
    axes[1,2].set_title('Model Comparison (Insufficient Data)')

plt.tight_layout()
output_figure_path =
    './figure_output/turbulence_analysis_results.png'
plt.savefig(output_figure_path, dpi=300, bbox_inches='tight',
    facecolor='white')
print(f"Figure saved to: {output_figure_path}")

results_dataframe = pd.DataFrame({
    'Altitude_m': analysis_altitude,
    'Reference_Turbulence': turbulence_reference,
    'Basic_Turbulence': turbulence_basic,
    'Optimized_Turbulence': turbulence_optimized,
    'Richardson_Number': richardson_number,
    'Turbulent_Kinetic_Energy': turbulent_kinetic_energy,
    'Wind_Shear': wind_shear_profile,
    'Spectral_Width': spectral_width_combined,
    'U_Component_mps': u_combined,
    'V_Component_mps': v_combined,
    'Temperature_K':
        temperature_kelvin[:len(analysis_altitude)]
})

output_csv_path =
    './result_output/turbulence_intensity_results.csv'
results_dataframe.to_csv(output_csv_path, index=False,
    encoding='utf-8-sig')
print(f"Results CSV saved to: {output_csv_path}")

with open('./result_output/model_evaluation.txt', 'w',
    encoding='utf-8') as eval_file:
    eval_file.write(f"Basic model RMSE: {basic_rmse}\n")
    eval_file.write(f"Basic model R2: {basic_r2}\n")
    eval_file.write(f"Optimized model RMSE: {optimized_rmse}\n")
    eval_file.write(f"Optimized model R2: {optimized_r2}\n")

```

```
print("Processing completed successfully.")

if __name__ == "__main__":
    execute_analysis()
```